



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that Ansh Sarfare of D20A semester VIII, have successfully completed necessary experiments in the ITC801: Blockchain & DLT Lab under my supervision in **VES Institute of Technology** during the academic year 2025-2026.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher Name: Kajal

Joseph Signature:



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Experiment -1

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Theory:

1. Cryptographic Hash Function in Blockchain

A cryptographic hash function is a mathematical function that takes an input string of any length and converts it into a fixed-length output string known as a **hash value**. These hash functions are designed to be secure, fast, and irreversible, making them suitable for blockchain applications.

To be cryptographically secure and useful, a hash function must satisfy the following properties:

- **Collision Resistance:**
It should be computationally difficult to find two different messages m_1 and m_2 such that $\text{hash}(k, m_1) = \text{hash}(k, m_2)$ where k is a key value.
- **Preimage Resistance:**
Given a hash value h , it should be infeasible to determine the original input m such that $h = \text{hash}(k, m)$. The original data cannot be derived from the hash.
- **Second Preimage Resistance:**
Given a message m_1 , it should be difficult to find another message m_2 such that both produce the same hash.
- **Large Output Space:**
Hash functions produce a large number of possible outputs, making brute-force collision attacks impractical.
- **Deterministic:**
For the same input, the hash function must always generate the same output.
- **Avalanche Effect:**
A very small change in input results in a completely different hash output.
- **Puzzle Friendliness:**
Even if a portion of the hash is known, it should be impossible to predict the remaining bits.

- **Fixed-Length Mapping:**
Regardless of input size, the output hash is always of fixed length.

Role of Hash Functions in Blockchain

- Ensures data integrity
- Links blocks securely using hashes
- Used in Merkle Trees
- Used in block headers and digital signatures

2. Merkle Tree (Hash Tree)

A Merkle Tree, also known as a **Hash Tree**, is a data structure used for efficient data verification and synchronization. It is a binary tree in which each non-leaf node is the cryptographic hash of its child nodes.

All leaf nodes are located at the same depth and positioned as far left as possible. The hash at the top of the tree is called the **Merkle Root**, which represents the entire dataset.

Key characteristics:

- Enables efficient handling of large datasets
- Any change in data can be easily detected
- The root hash acts as a fingerprint for the complete data

3. Structure of a Merkle Tree

A Merkle Tree is an **inverted binary tree** constructed from the bottom up using cryptographic hashes. It consists of the following components:

- **Leaf Nodes:**
The lowest level of the tree. Each leaf node contains the hash of an individual data block or transaction.
- **Internal Nodes:**
Each internal node is generated by concatenating the hashes of its two child nodes and hashing the combined value.
- **Merkle Root:**
The topmost node of the tree. It uniquely represents all the data below it. Any modification in the underlying data results in a different Merkle Root, indicating

tampering.

4. Merkle Root

The **Merkle Root** is the final hash at the top of a Merkle Tree. It serves as a concise summary of all transactions within a block. In blockchains like Bitcoin, the Merkle Root is stored in the block header.

Importance of Merkle Root

1. Acts as a unique digital fingerprint of all transactions in a block
2. Stored in the block header instead of the full transaction list
3. Any transaction modification changes the Merkle Root
4. Enables fast and efficient transaction verification
5. Ensures data integrity, security, and immutability

5. Working of Merkle Tree

A Merkle Tree is built in a **bottom-up manner** by repeatedly hashing pairs of nodes until a single hash remains.

Example:

Consider four transactions: T_1 , T_2 , T_3 , T_4

Step 1: Compute individual hashes

- $H_1 = \text{Hash}(T_1)$
- $H_2 = \text{Hash}(T_2)$
- $H_3 = \text{Hash}(T_3)$
- $H_4 = \text{Hash}(T_4)$

Step 2: Form parent nodes

- $H_{12} = \text{Hash}(H_1 + H_2)$
- $H_{34} = \text{Hash}(H_3 + H_4)$

Step 3: Compute Merkle Root

- $H_{1234} = \text{Hash}(H_{12} + H_{34})$

The final hash H_{1234} is the **Merkle Root**.

6. Use of Merkle Tree in Blockchain

In distributed blockchain networks, every node maintains its own copy of the ledger. Without Merkle Trees, validating transactions would require transferring and comparing entire ledgers, which is inefficient and resource-intensive.

Merkle Trees solve this problem by:

- Allowing verification using only a small subset of hashes
- Reducing network bandwidth usage
- Eliminating the need to download full transaction data

Thus, Merkle Trees enable scalable, secure, and efficient blockchain operation.

Program and Output:

```
import hashlib

# 1. SHA-256 Hash Function
def generate_hash(message):
    return hashlib.sha256(message.encode()).hexdigest()

user_text = input("Enter a message to hash: ")
hash_result = generate_hash(user_text)
print("Generated Hash:", hash_result)
```

Enter a message to hash: Hello Ansh
Generated Hash: c35f50e7afce646b6211b5487fdae273159b01ce3d7cdd864aff00aa86e914c5

#2. Execution: Hash with Nonce

```
def generate_hash_with_nonce(message, nonce):  
    combined = message + str(nonce)  
    return generate_hash(combined)  
  
nonce_value = int(input("Enter a nonce value: "))  
nonce_hash = generate_hash_with_nonce(user_text, nonce_value)  
print("Hash with Nonce:", nonce_hash)
```

Enter a nonce value: 3

Hash with Nonce: d51352c6698752911f09fff6252c715e6271a5ee599605aeddff69e1659a3c7c

3. Proof of Work

```
def proof_of_work(message, difficulty):  
    print("\nMining started...")  
    prefix = "0" * difficulty  
    nonce = 0  
  
    while True:  
        new_hash = generate_hash_with_nonce(message, nonce)  
        if new_hash.startswith(prefix):  
            return nonce, new_hash  
        nonce += 1  
  
difficulty_level = int(input("Enter difficulty (number of leading zeros): "))  
valid_nonce, valid_hash = proof_of_work(user_text, difficulty_level)  
  
print("Nonce Found:", valid_nonce)  
print("Valid Hash:", valid_hash)
```

Enter difficulty (number of leading zeros): 4

Mining started...

Nonce Found: 67746

Valid Hash: 00008d13c735d52906bf7e7ab2c8953e1cc80276305bae1d36baaebd506f0a8f

```

# 4. Merkle Tree & Merkle Root
def calculate_merkle_root(transactions):
    hash_list = [generate_hash(tx) for tx in transactions]

    while len(hash_list) > 1:
        if len(hash_list) % 2 != 0:
            hash_list.append(hash_list[-1])
        temp_list = []
        for i in range(0, len(hash_list), 2):
            combined_hash = hash_list[i] + hash_list[i + 1]
            temp_list.append(generate_hash(combined_hash))
        hash_list = temp_list
    return hash_list[0]

print("Enter transactions (type 'stop' to finish):")
transactions = []
while True:
    tx = input()
    if tx.lower() == "stop":
        break
    transactions.append(tx)
if len(transactions) == 0:
    print("⚠ No transactions entered.")
else:
    root_hash = calculate_merkle_root(transactions)
    print("Merkle Root Hash:")
    print(root_hash)

```

```

Enter transactions (type 'stop' to finish):
1
2
stop
Merkle Root Hash:
33b675636da5dcc86ec847b38c08fa49ff1cace9749931e0a5d4dfdbdedd808a

```

Conclusion

This experiment successfully demonstrated the foundational cryptographic concepts used in blockchain technology. SHA-256 hashing and Proof-of-Work illustrated how data integrity and consensus are achieved, while the Merkle Tree demonstrated efficient and secure verification of large transaction datasets. Together, these mechanisms ensure the security, immutability, and scalability of blockchain systems.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Experiment -2

Aim: Create a Blockchain using Python

Theory:

1. What is Blockchain

Blockchain is an innovative technology that works as a shared, tamper-resistant digital ledger. Its name comes from the way data is structured — information is stored in blocks, each connected to the previous one, forming a continuous chain.

Each block contains important information such as a list of transactions, a timestamp, and a cryptographic hash, which uniquely identifies the block. The hash is generated from both the block's content and the previous block's hash, which links the blocks together securely.

- This linked design ensures that any attempt to alter data becomes immediately noticeable, as it breaks the chain.
- Blockchain serves as a distributed database, storing transaction data across all network participants.
- Transactions are verified collectively by the network, maintaining legitimacy and trust.
- Because it is decentralized, no single entity can manipulate the data.

In a blockchain network, multiple computers (called nodes) hold copies of the ledger. Once a transaction is added and confirmed, it becomes practically immutable, meaning it cannot be changed or deleted without consensus from the network.

2. What is a Block in Blockchain

A block is a digital container that securely stores verified transactions. It is a key building block of the blockchain. Each block contains:

- Transaction Data – details of the transaction such as sender, receiver, and amount.
- Timestamp – records the exact creation time of the block.
- Unique Hash – a cryptographic fingerprint of the block, ensuring its integrity.

- Previous Block Hash – links the block to the one before it, maintaining an unbroken chain.

By storing this information, blocks create a chronological and transparent record of all transactions across the network.

3. Components of a Blockchain Block

Key Components Include:

- Transaction Data: Captures details of who made the transaction, what was exchanged, and when.
- Timestamp: Marks when the block was created, maintaining chronological order.
- Hash: A unique identifier for the block, produced from its contents. Any change in data changes the hash.
- Previous Hash: Connects this block to the preceding one, forming a secure chain.

4. What is Mining

Mining is the process of validating transactions and adding new blocks to the blockchain. It is crucial for network security, decentralization, and transparency.

In cryptocurrencies like Bitcoin, mining uses Proof of Work (PoW), where miners solve complex computational puzzles. Miners compete to validate transactions and append blocks, receiving rewards for success.

Steps in Mining Process

4.1 Transaction Initiation

- Users submit transactions to the blockchain network.
- These transactions are broadcast to all nodes.

4.2 Transaction Pool (Mempool)

- Unconfirmed transactions are collected in a temporary area called the mempool.
- Miners select transactions from this pool to include in new blocks.

4.3 Block Creation

A new block is formed containing:

- Selected transactions
- Hash of the previous block
- Timestamp
- Nonce (a random number used for hashing)

4.4 Hash Generation

- Miners compute a cryptographic hash (e.g., SHA-256) of the block.
- The hash must satisfy the network's difficulty rules, such as starting with a certain number of zeros.

4.5 Proof of Work

- Miners repeatedly adjust the nonce and recompute the hash until a valid hash is found.
- The first miner to find a valid hash proves they have completed the PoW.

4.6 Block Verification

- The mined block is broadcast to all network nodes.
- Nodes verify:
 - Hash correctness
 - Transaction authenticity
 - Proper block structure

4.7 Adding Block to Blockchain

- Once verified, the block is permanently added to the chain.
- The chain remains tamper-proof and synchronized across all nodes.

4.8 Reward Distribution

- The miner who successfully adds the block receives:
 - New coins (block reward)
 - Transaction fees from transactions in the block

5. How to Check Block Validity

Block validity ensures that a block adheres to all blockchain rules before it is accepted. Key checks include:

- **Block Structure Verification:** Confirm the block has all required elements like header, previous hash, timestamp, nonce, and transactions.
- **Previous Hash Validation:** Compare the block's previous hash with the hash of the latest block to maintain the chain's integrity.
- **Hash and Proof of Work Verification:** Recompute the block hash and ensure it meets the network's difficulty conditions.
- **Transaction Validation:** Ensure every transaction has valid digital signatures, sufficient balance, and no double-spending.
- **Merkle Root Verification:** Recalculate the Merkle root from all transactions and compare it with the stored root.
- **Timestamp Verification:** Ensure the block's timestamp is valid and sequential relative to the previous block.
- **Block Size and Rule Compliance:** Check that the block follows size limits and other protocol rules.
- **Consensus Rule Verification:** Ensure the block adheres to the network's consensus mechanisms, confirming it is legitimate.

Program:

Module 1 - Create a Simple Blockchain (Fully Commented)

Required installation:(run cmd as administrator)

pip install flask==2.2.5

Importing libraries

import datetime # To generate timestamps for blocks

```

import hashlib    # To generate SHA256 hashes
import json       # To convert block data into JSON
from flask import Flask, jsonify # To create API endpoints

# Part 1 - Building the Blockchain

class Blockchain:

    def __init__(self):
        # This list will store the entire chain of blocks
        self.chain = []
        # Create the genesis block (first block)
        # proof = 1 → arbitrary
        # previous_hash = '0' → since no previous block exists
        self.create_block(proof=1, previous_hash='0')

    def create_block(self, proof, previous_hash):
        Creates a new block and adds it to the chain.
        Each block contains:
        - index
        - timestamp
        - proof (PoW output)
        - previous
        block = {
            'index': len(self.chain) + 1,          # Position of block in chain
            'timestamp': str(datetime.datetime.now()), # Current timestamp
            'proof': proof,                        # PoW result
            'previous_hash': previous_hash         # Hash of the previous block
        }
        # Add block to the chain
        self.chain.append(block)
        return block

    def get_previous_block(self):

```

Return the last block in the chain.

```
return self.chain[-1]
```

```
def proof_of_work(self, previous_proof):
```

Simple Proof of Work (PoW) algorithm:

- Find a number (new_proof)
- Such that the SHA256 hash of (new_proof² - previous_proof²) starts with '0000'

This is a computational puzzle to secure the network.

```
new_proof = 1
```

```
check_proof = False
```

```
while check_proof is False:
```

```
    # Cryptographic puzzle: hash difference of squares
```

```
    hash_operation = hashlib.sha256(
```

```
        str(new_proof**2 - previous_proof**2).encode()
```

```
    ).hexdigest()
```

```
    # Check if hash begins with 4 leading zeros
```

```
    if hash_operation[:4] == '0000':
```

```
        check_proof = True
```

```
    else:
```

```
        new_proof += 1
```

```
return new_proof
```

```
def hash(self, block):
```

Creates a SHA256 hash of a block.

- Sort keys to ensure consistent hashing """

```
encoded_block = json.dumps(block, sort_keys=True).encode()
```

```
return hashlib.sha256(encoded_block).hexdigest()
```

```
def is_chain_valid(self, chain):
```

"""

Validates the blockchain by checking:

1. The previous_hash field matches the actual hash of the previous block.
2. The PoW condition (hash starting with '0000') is satisfied for each block.

```
previous_block = chain[0] # Genesis block
block_index = 1          # Start checking from block 2
while block_index < len(chain):
    block = chain[block_index]
    # Check previous hash correctness
    if block['previous_hash'] != self.hash(previous_block):
        return False
    # Validate Proof of Work
    previous_proof = previous_block['proof']
    proof = block['proof']
    hash_operation = hashlib.sha256(
        str(proof**2 - previous_proof**2).encode()
    ).hexdigest()
    if hash_operation[:4] != '0000':
        return False
    # Move to next block
    previous_block = block
    block_index += 1
return True
```

Part 2 - Mining our Blockchain (Flask API)

Initialize Flask web application

```
app = Flask(__name_)
```

Create an instance of the Blockchain


```

blockchain = Blockchain()

# API Endpoint: Mine a new block
@app.route('/mine_block', methods=['GET'])
def mine_block():
    # Get the previous block
    previous_block = blockchain.get_previous_block()

    # Extract previous proof
    previous_proof = previous_block['proof']

    # Run Proof of Work algorithm
    proof = blockchain.proof_of_work(previous_proof)

    # Get hash of previous block
    previous_hash = blockchain.hash(previous_block)

    # Create the new block
    block = blockchain.create_block(proof, previous_hash)

    # Prepare response
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash']
    }

    return jsonify(response), 200

# API Endpoint: Get the full blockchain
@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length':
            len(blockchain.chain)
    }

```

```

        return jsonify(response), 200

# API Endpoint: Check if blockchain is
valid @app.route('/is_valid',
methods=['GET']) def is_valid():

    is_valid = blockchain.is_chain_valid(blockchain.chain)

    if is_valid:

        response = {'message': 'All good. The Blockchain is valid.'}

    else:

        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}

    return jsonify(response), 200

# Run the Flask app
app.run(host='0.0.0.0', port=5000)

```

Output:

```

C:\Users\STUDENT.INFT505-03\Desktop>cd blockchain lab

C:\Users\STUDENT.INFT505-03\Desktop\blockchain lab>python -m venv venv

C:\Users\STUDENT.INFT505-03\Desktop\blockchain lab>venv\Scripts\activate.bat

(venv) C:\Users\STUDENT.INFT505-03\Desktop\blockchain lab>pip install flask==2.2.5
Collecting flask==2.2.5
  Downloading Flask-2.2.5-py3-none-any.whl.metadata (3.9 kB)
Collecting Werkzeug>=2.2.2 (from flask==2.2.5)
  Downloading werkzeug-3.1.5-py3-none-any.whl.metadata (4.0 kB)

```

```

[notice] To update, run: python.exe -m pip install --upgrade pip

(venv) C:\Users\STUDENT.INFT505-03\Desktop\blockchain lab>
(venv) C:\Users\STUDENT.INFT505-03\Desktop\blockchain lab>python.exe -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\student.inft505-03\desktop\blockchain lab\venv\lib\site-packages (25.1.1)
Collecting pip
  Using cached pip-25.3-py3-none-any.whl.metadata (4.7 kB)
Using cached pip-25.3-py3-none-any.whl (1.8 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.1.1
    Uninstalling pip-25.1.1:
      Successfully uninstalled pip-25.1.1
Successfully installed pip-25.3

(venv) C:\Users\STUDENT.INFT505-03\Desktop\blockchain lab>python blockchain.py

```

```
(venv) C:\Users\STUDENT.INFT505-03\Desktop\blockchain lab>python blockchain.py
* Serving Flask app 'blockchain'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.32.230:5001
Press CTRL+C to quit
127.0.0.1 - - [30/Jan/2026 12:47:19] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:47:39] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:47:44] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:47:47] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:48:04] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:48:09] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:48:11] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:48:25] "GET /is_valid HTTP/1.1" 200 -
```

← → ↻ ⓘ 127.0.0.1:5001/mine_block

Pretty-print ☒

```
{
  "index": 4,
  "message": "Ansh, you just mined a block!",
  "previous_hash": "d0863156188d5c027df2bd30b848824245c3ce19176588e2e6debbb2df65450d",
  "proof": 21391,
  "timestamp": "2026-01-30 12:48:09.371936"
}
```

← → ↻ ⓘ 127.0.0.1:5001/get_chain

Pretty-print ☒

```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-01-30 12:47:07.313020"
    },
    {
      "index": 2,
      "previous_hash": "5ab6acdb01cd2d0c7bcb1ca286b4d820348d71c7db288bd6f1aeb7f4a80bfc29",
      "proof": 533,
      "timestamp": "2026-01-30 12:47:19.626099"
    },
    {
      "index": 3,
      "previous_hash": "12f847513780c8368e276efff21a4e98d611d32e9d52f03796a2f036534d4c5d",
      "proof": 45293,
      "timestamp": "2026-01-30 12:47:44.855424"
    },
    {
      "index": 4,
      "previous_hash": "d0863156188d5c027df2bd30b848824245c3ce19176588e2e6debbb2df65450d",
      "proof": 21391,
      "timestamp": "2026-01-30 12:48:09.371936"
    }
  ],
  "length": 4
}
```

A screenshot of a web browser window. The address bar shows the URL '127.0.0.1:5001/is_valid'. Below the address bar, there is a toggle switch labeled 'Pretty-print' which is currently turned on. The main content area of the browser displays a JSON object:

```
{  "message": "All good. The Blockchain is valid."}
```

```
← → ↻ ⓘ 127.0.0.1:5001/is_valid
Pretty-print ☒
{
  "message": "All good. The Blockchain is valid."
}
```

Conclusion

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles decentralization, immutability, and consensus in a simplified environment, providing a strong foundation for building more advanced blockchain systems.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Blockchain Exp - 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints
pip install Flask
- **Requests** – For HTTP communication between nodes
pip install requests==2.18.4
- **Postman** – For testing API requests
- Python Standard Libraries:
 - datetime
 - jsonify
 - hashlib
 - uuid4
 - urlparse
 - request

Code :

```
# Module 2 - Create a Cryptocurrency

# To be installed:
# Flask==0.12.2: pip install Flask==0.12.2
# Postman HTTP Client: https://www.getpostman.com/
# requests==2.18.4: pip install requests==2.18.4

# Importing the
libraries import
datetime
import hashlib
import json
from flask import Flask, jsonify, request
import requests
from uuid import uuid4
hex from urllib.parse import urlparse

# Generate a unique id that is in
# To parse url of the nodes

# Part 1 - Building a Blockchain
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        are added to a block
        self.create_block(proof = 1, previous_hash = '0')
        self.nodes = set()
        # Set is used as there is no order to
        be maintained as the nodes can be from all around the globe

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                  'timestamp': str(datetime.datetime.now()),
                  'proof': proof,
                  'previous_hash': previous_hash,
                  'transactions': self.transactions}
        # Adding transactions to make
        the blockchain a cryptocurrency
        self.transactions = []
        # The list of transaction should
        become empty after they are added to a block
        self.chain.append(block)
        return block
```



```

def get_previous_block(self):
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    new_proof = 1
    check_proof = False
    while check_proof is False:
        hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys = True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] != '0000':
            return False
        previous_block = block
        block_index += 1
    return True

```

- This method will add the transaction to the list of transactions
- ```

def add_transaction(self, sender, receiver, amount):
 self.transactions.append({'sender': sender,
 'receiver': receiver,
 'amount': amount})
 previous_block = self.get_previous_block()

```

```
 return previous_block['index'] + 1 # It will return the block index
 to which the transaction should be added
```

• This function will add the node containing an address to the set of nodes created in init function

```
def add_node(self, address):
 parsed_url = urlparse(address) # urlparse will parse the url from
 the address # Add is used and not append as
 self.nodes.add(parsed_url.netloc) # it's a set. Netloc will only return '127.0.0.1:5000'
```

• Consensus Protocol. This function will replace all the shorter chain with the longer chain in all the nodes on the network

```
def replace_chain(self):
 network = self.nodes # network variable is the set of nodes
 all around the globe
 longest_chain = None # It will hold the longest chain when
 we scan the network
 max_length = len(self.chain) # This will hold the length of the
 chain held by the node that runs this function
 for node in network:
 response = requests.get(f'http://{node}/get_chain') # Use get chain
 method already created to get the length of the chain
 if response.status_code == 200:
 length = response.json()['length'] # Extract the length of the chain
 from get_chain function
 chain = response.json()['chain']
 if length > max_length and self.is_chain_valid(chain): # We check if the length
 is bigger and if the chain is valid then
 max_length = length # We update the max length
 longest_chain = chain # We update the longest chain
 if longest_chain: # If longest_chain is not none that means
 it was replaced
 self.chain = longest_chain # Replace the chain of the current
 node with the longest chain
 return True
 return False # Return false if current chain is the longest
 one
```

# Part 2 - Mining our Blockchain

# Creating a Web App

```

app = Flask(_name_)

Creating an address for the node on Port 5000. We will create some other nodes as well on
different ports
node_address = str(uuid4()).replace('-', '') #

Creating a Blockchain
blockchain = Blockchain()

Mining a new block
@app.route('/mine_block', methods = ['GET'])
def mine_block():
 previous_block = blockchain.get_previous_block()
 previous_proof = previous_block['proof']
 proof = blockchain.proof_of_work(previous_proof)
 previous_hash = blockchain.hash(previous_block)
 blockchain.add_transaction(sender = node_address, receiver = 'Richard', amount = 1) #
 Hadcoins to mine the block (A Reward). So the node gives 1 hadcoin to Abcde for mining the
 block
 block = blockchain.create_block(proof, previous_hash)
 response = {'message': 'Congratulations, you just mined a block!',
 'index': block['index'],
 'timestamp': block['timestamp'],
 'proof': block['proof'],
 'previous_hash': block['previous_hash'],
 'transactions': block['transactions']}
 return jsonify(response), 200

Getting the full Blockchain
@app.route('/get_chain', methods = ['GET'])
def get_chain():
 response = {'chain': blockchain.chain,
 'length': len(blockchain.chain)}
 return jsonify(response), 200

Checking if the Blockchain is valid
@app.route('/is_valid', methods = ['GET'])
def is_valid():
 is_valid = blockchain.is_chain_valid(blockchain.chain)
 if is_valid:
 response = {'message': 'All good. The Blockchain is valid.'}
 else:

```

```
 response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
 return jsonify(response), 200
```

```
Adding a new transaction to the Blockchain
```

```
@app.route('/add_transaction', methods = ['POST']) # Post method as we
have to pass something to get something in return
```

```
def add_transaction():
```

```
 json = request.get_json() # This will get the json file from
postman. In Postman we will create a json file in which we will pass the values for the keys in
the json file
```

```
 transaction_keys = ['sender', 'receiver', 'amount']
```

```
 if not all(key in json for key in transaction_keys): # Checking if all keys
are available in json
```

```
 return 'Some elements of the transaction are missing', 400
```

```
 index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])
```

```
 response = {'message': f'This transaction will be added to Block {index}'}
 return jsonify(response), 201 # Code 201 for creation
```

```
Part 3 - Decentralizing our Blockchain
```

```
Connecting new nodes
```

```
@app.route('/connect_node', methods = ['POST']) # POST request to
register the new nodes from the json file
```

```
def connect_node():
```

```
 json = request.get_json()
```

```
 nodes = json.get('nodes') # Get the nodes from json
```

```
 file if nodes is None:
```

```
 return "No node", 400
```

```
 for node in nodes:
```

```
 blockchain.add_node(node)
```

```
 response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now
contains the following nodes:',
```

```
 'total_nodes': list(blockchain.nodes)}
```

```
 return jsonify(response), 201
```

```
Replacing the chain by the longest chain if needed
```

```
@app.route('/replace_chain', methods = ['GET'])
```

```
def replace_chain():
```

```
 is_chain_replaced = blockchain.replace_chain()
```

```
 if is_chain_replaced:
```

```
 response = {'message': 'The nodes had different chains so the chain was replaced by the
longest one.',
```

```
 'new_chain': blockchain.chain}
 else:
 response = {'message': 'All good. The chain is the largest one.',
 'actual_chain': blockchain.chain}
 return jsonify(response), 200

Running the app
app.run(host = '0.0.0.0', port = 500*)
```

Output :

### 1. Connecting nodes: Node 1 -> 2,3

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:5001/connect_node`. The request body is a JSON object: `{ "nodes": [ "http://127.0.0.1:5002", "http://127.0.0.1:5003" ] }`. The response status is 201 CREATED, and the response body is: `{ "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:", "total_nodes": [ "127.0.0.1:5002" ] }`.

```
POST http://127.0.0.1:5001/connect_node
```

```
{
 "nodes": [
 "http://127.0.0.1:5002",
 "http://127.0.0.1:5003"
]
}
```

Status: 201 CREATED Time: 9 ms Size: 308 B

```
{
 "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
 "total_nodes": [
 "127.0.0.1:5002"
]
}
```

### Node 2 -> 1,3

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:5002/connect_node`. The request body is a JSON object: `{ "nodes": [ "http://127.0.0.1:5001", "http://127.0.0.1:5003" ] }`. The response status is 201 CREATED, and the response body is: `{ "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:", "total_nodes": [ "127.0.0.1:5001", "127.0.0.1:5003" ] }`.

```
POST http://127.0.0.1:5002/connect_node
```

```
{
 "nodes": [
 "http://127.0.0.1:5001",
 "http://127.0.0.1:5003"
]
}
```

Status: 201 CREATED Time: 5 ms Size: 325 B

```
{
 "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
 "total_nodes": [
 "127.0.0.1:5001",
 "127.0.0.1:5003"
]
}
```

## Node 3 -> 1,2

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:5003/connect_node`. The request body is a JSON object:

```
{ "nodes": ["http://127.0.0.1:5001", "http://127.0.0.1:5002"]}
```

The response status is 201 CREATED. The response body is a JSON object:

```
{ "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:", "total_nodes": ["127.0.0.1:5001", "127.0.0.1:5002"]}
```

## 2. Adding transaction from node 1

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:5001/add_transaction`. The request body is a JSON object:

```
{ "sender": "a", "receiver": "b", "amount": 5}
```

The response status is 201 CREATED. The response body is a JSON object:

```
{ "message": "This transaction will be added to Block 2"}
```

### 3. Mining a Block (Only on 5001)

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:5001/mine_block`. The response is a JSON object with the following structure:

```
{
 "index": 3,
 "message": "Congratulations Ansh, you just mined a block!",
 "previous_hash": "9a8ebca8604836891376ba868d5364a1cfaf416181f1649e282f6e4d970b8",
 "proof": 45293,
 "timestamp": "2026-02-28 12:06:24.482439",
 "transactions": [
 {
 "amount": 1,
 "receiver": "Richard",
 "sender": "d0052a0c97154aac90270efc0d5926ca"
 }
]
}
```

### 4. Checking chain length difference: Node 1

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:5001/get_chain`. The response is a JSON object with the following structure:

```
{
 "chain": [
 {
 "index": 1,
 "previous_hash": "8",
 "proof": 1,
 "timestamp": "2026-02-28 12:08:28.223268",
 "transactions": []
 }
],
 "length": 1
}
```



## Node 2

Postman interface showing a GET request to `http://127.0.0.1:5002/get_chain`. The request is successful (Status: 200 OK, Time: 14 ms, Size: 290 B). The response body is a JSON object representing a block in a chain.

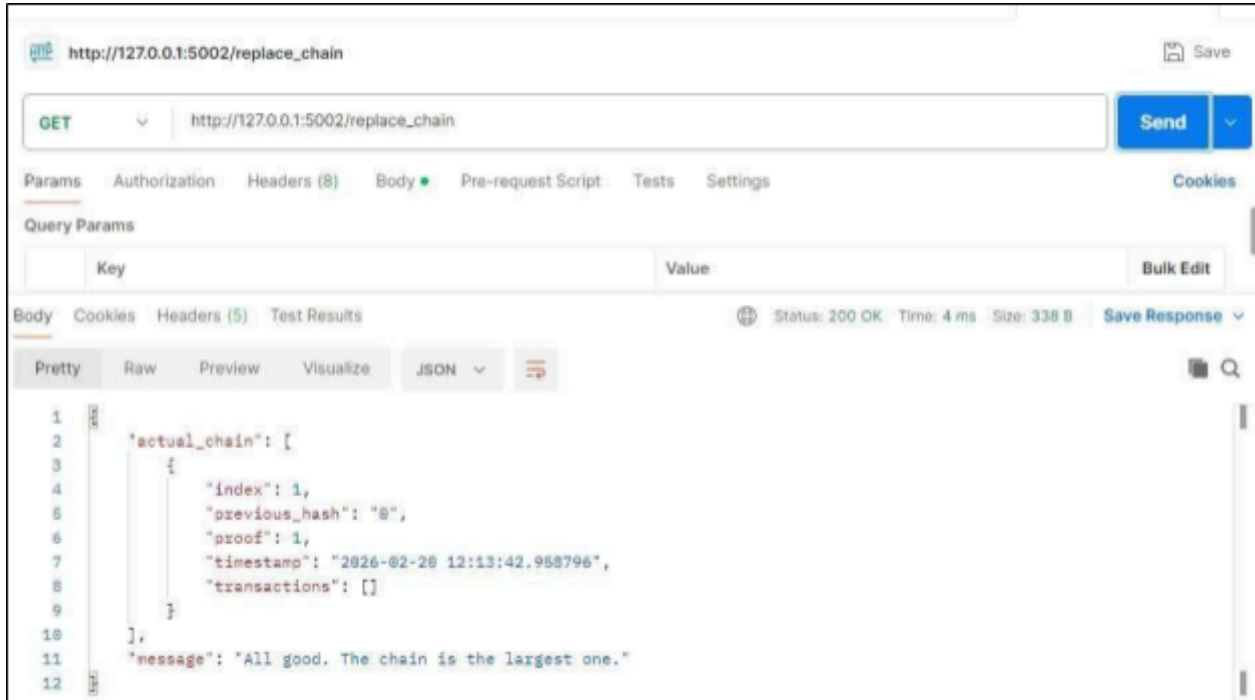
```
1 {
2 "sender": "a",
3 "receiver": "b",
4 "chain": [
5 {
6 "index": 1,
7 "previous_hash": "0",
8 "proof": 1,
9 "timestamp": "2026-02-20 12:11:06.701988",
10 "transactions": []
11 }
12],
13 "length": 1
14 }
```

## Node 3

Postman interface showing a GET request to `http://127.0.0.1:5003/get_chain`. The request is successful (Status: 200 OK, Time: 5 ms, Size: 290 B). The response body is a JSON object representing a block in a chain, similar to Node 2 but with a different timestamp.

```
1 {
2 "sender": "a",
3 "receiver": "b",
4 "amount": 5,
5 "chain": [
6 {
7 "index": 1,
8 "previous_hash": "0",
9 "proof": 1,
10 "timestamp": "2026-02-20 12:12:08.344375",
11 "transactions": []
12 }
13],
14 "length": 1
15 }
```

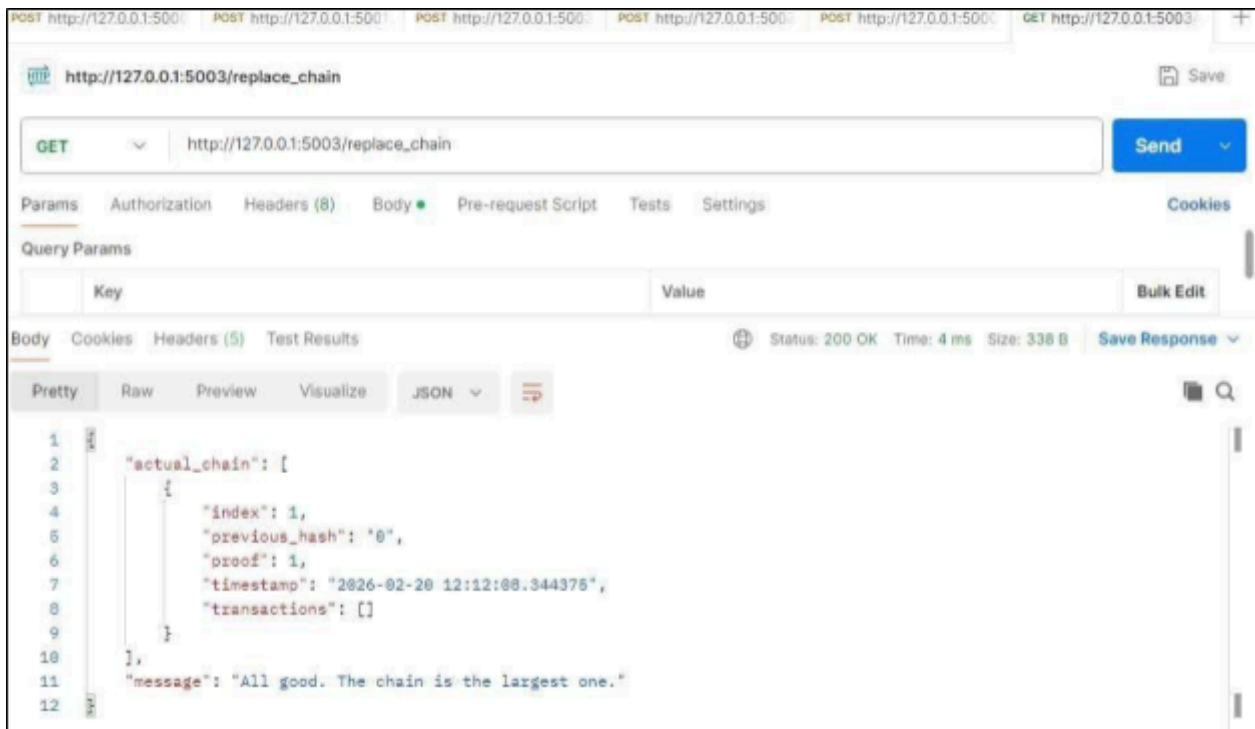
## 5. Replace Chain (Consensus Mechanism) : Node 2



REST client interface showing a GET request to `http://127.0.0.1:5002/replace_chain`. The response is a JSON object with the following structure:

```
1 {
2 "actual_chain": [
3 {
4 "index": 1,
5 "previous_hash": "0",
6 "proof": 1,
7 "timestamp": "2026-02-20 12:13:42.958796",
8 "transactions": []
9 }
10],
11 "message": "All good. The chain is the largest one."
12 }
```

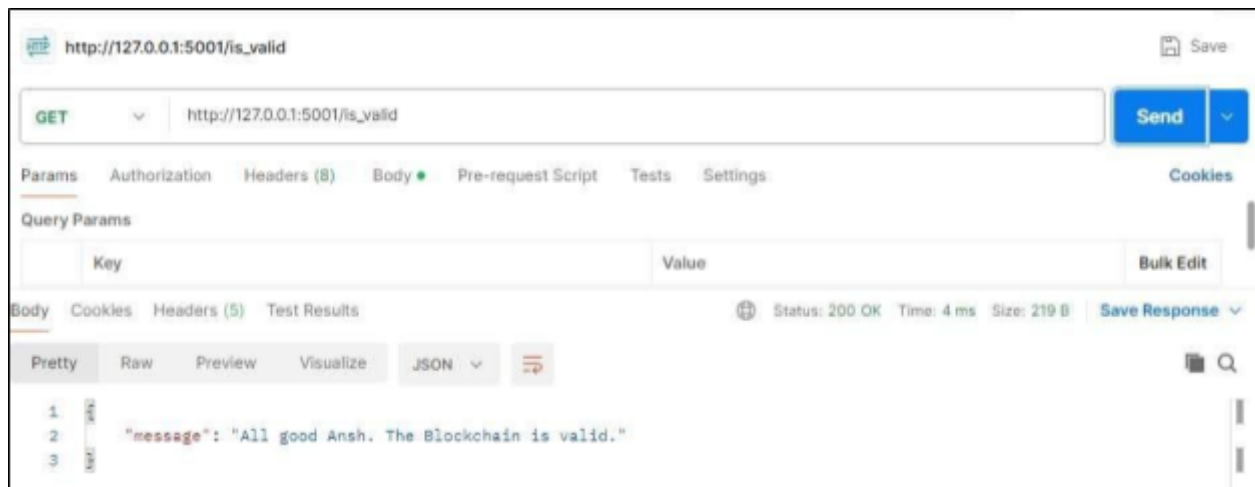
## Node 3



REST client interface showing a GET request to `http://127.0.0.1:5003/replace_chain`. The response is a JSON object with the following structure:

```
1 {
2 "actual_chain": [
3 {
4 "index": 1,
5 "previous_hash": "0",
6 "proof": 1,
7 "timestamp": "2026-02-20 12:12:08.344375",
8 "transactions": []
9 }
10],
11 "message": "All good. The chain is the largest one."
12 }
```

## 6. Final Validation



## 7. Terminal Activity

```
(venv) PS C:\Users\Student\Documents\Lab_3_Create a Cryptocurrency> python hadcoin_node_5003.py
WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5003
* Running on http://192.168.36.127:5003
Press CTRL+C to quit
127.0.0.1 - - [20/Feb/2026 12:12:15] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [20/Feb/2026 12:13:06] "GET /replace_chain HTTP/1.1" 200 -
(venv) PS C:\Users\Student\Documents\Lab_3_Create a Cryptocurrency> python hadcoin_node_5002.py
* Serving Flask app 'hadcoin_node_5002'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on http://192.168.36.127:5002
Press CTRL+C to quit
127.0.0.1 - - [20/Feb/2026 12:13:48] "GET /replace_chain HTTP/1.1" 200 -
(venv) PS C:\Users\Student\Documents\Lab_3_Create a Cryptocurrency> python hadcoin_node_5001.py
* Serving Flask app 'hadcoin_node_5001'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.36.127:5001
```

**Conclusion:** In this experiment, we successfully created a basic cryptocurrency using Python and Flask and implemented mining in a multi-node blockchain network. We demonstrated how transactions are added, stored temporarily, and included in a block during the mining process using a Proof-of-Work mechanism. By running three separate nodes, we simulated a peer-to-peer network and applied the Longest Chain Rule to maintain consensus across all nodes. The experiment helped us understand blockchain structure, decentralized networking, mining rewards, and chain synchronization in a practical manner.



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801 : Blockchain & DLT Lab**  
**Experiment 04**

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

|           |                                                                                   |
|-----------|-----------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                |
| Name      | Ansh Sarfare                                                                      |
| Class     | D20A                                                                              |
| Subject   | ITC801: Blockchain and DLT Lab                                                    |
| CO Mapped | CO3:Implement smart contracts in Ethereum using different development frameworks. |

## Blockchain Exp - 4

**AIM:** Hands on Solidity Programming Assignments for creating Smart Contracts.

### Theory:

#### 1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

#### 2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

#### 3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:
  - o **public:** available both inside and outside the contract.
  - o **private:** only accessible within the same contract.
  - o **internal:** accessible within the contract and its child contracts.
  - o **external:** can be called only by external accounts or other contracts.

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

### 3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

### 5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays:** Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings:** Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs:** Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums:** Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

### 6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

## 7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether =  $10^{18}$  Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

### Implementation:

#### Tutorial - 1 (compile)

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract Counter {
5 uint public count;
6
7 // Function to get the current count
8 function get() public view returns (uint) { 2453 gas
9 return count;
10 }
11
12 // Function to increment count by 1
13 function inc() public { infinite gas
14 count += 1;
15 }
16
17 // Function to decrement count by 1
18 function dec() public { infinite gas
19 count -= 1;
20 }
21 }
```

Welcome to Remix 1.5.1

Your files are stored in indexedDB, 2.65 KB / 108.86 GB used

You can use this terminal to:

- Check transactions details and start debugging.
- Execute JavaScript scripts:
  - Input a script directly in the command line interface
  - Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.executeCurrent()` in the command line interface
  - Right-click on a JavaScript file in the file explorer and then click 'Run'

The following libraries are accessible:

- [ethers.js](#)

Type the library name to see available commands.

creation of Counter pending...



[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei  
data: 0x608...f0033 logs: 0 hash: 0x955...0d83c

Debug





## Tutorial - 1 (get)

COUNTER AT 0XD91...39138 (MEMORY)

Balance: 0 ETH

dec

inc

count

get

0: uint256: 0

Low level interactions

CALLDATA

Transact

You can use this terminal to:

- Check transactions details and start debugging.
- Execute JavaScript scripts:
  - Input a script directly in the command line interface
  - Select a JavaScript file in the file explorer and then run `remix.execute()` or `remix.executeCurrent()` in the command line interface
  - Right-click on a JavaScript file in the file explorer and then click "Run"

The following libraries are accessible:

- [ethers.js](#)

Type the library name to see available commands.

creation of Counter pending...

[vm] from: 0x583...eddC4 to: Counter.(constructor) value: 0 wei  
data: 0x608...f0033 logs: 0 hash: 0x955...0d83c

call to Counter.get

[call] from: 0x58380a6a701c568545dCfc803Fc8875f56beddC4 to: Counter.get()  
data: 0xb6d4...ce63c

## Tutorial - 1 (inc)

COUNTER AT 0XD91...39138 (MEMORY)

Balance: 0 ETH

dec

inc

count

get

0: uint256: 0

Low level interactions

CALLDATA

Transact

Right-click on a JavaScript file in the file explorer and then click "Run"

The following libraries are accessible:

- [ethers.js](#)

Type the library name to see available commands.

creation of Counter pending...

[vm] from: 0x583...eddC4 to: Counter.(constructor) value: 0 wei  
data: 0x608...f0033 logs: 0 hash: 0x955...0d83c

call to Counter.get

[call] from: 0x58380a6a701c568545dCfc803Fc8875f56beddC4 to: Counter.get()  
data: 0xb6d4...ce63c

transact to Counter.inc pending ...

[vm] from: 0x583...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei  
data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5

## Tutorial - 1 (dec)

COUNTER AT 0XD91...39138 (MEMORY)

Balance: 0 ETH

dec

inc

count

get

0: uint256: 0

Low level interactions

CALLDATA

Transact

Type the library name to see available commands.

creation of Counter pending...

[vm] from: 0x583...eddC4 to: Counter.(constructor) value: 0 wei  
data: 0x608...f0033 logs: 0 hash: 0x955...0d83c

call to Counter.get

[call] from: 0x58380a6a701c568545dCfc803Fc8875f56beddC4 to: Counter.get()  
data: 0xb6d4...ce03c

transact to Counter.inc pending ...

[vm] from: 0x583...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei  
data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5

transact to Counter.dec pending ...

[vm] from: 0x583...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei  
data: 0xb3b...cfa82 logs: 0 hash: 0xc0c...79d46



## Tutorial - 2

LEARNETH

Tutorials list

Syllabus

2. Basic Syntax  
2 / 19

variable when you declare it. In this case, `greet` is a `string`.

We also define the *visibility* of the variable, which specifies from where you can access it. In this case, it's a `public` variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

[Watch a video tutorial on Basic Syntax.](#)

### ★ Assignment

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

basicSyntax.sol

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and
3 pragma solidity ^0.8.3;
4 //Ansh Sarfare D20A51
5 contract Mycontract {
6 string public name = "Alice";
7 }
```

## Tutorial - 3

LEARNETH

Tutorials list

Syllabus

3. Primitive Data Types  
3 / 19

You can learn more about these data types as well as *Fixed Point Numbers*, *Byte Arrays*, *Strings*, and more in the [Solidity documentation](#).

Later in the course, we will look at data structures like **Mappings**, **Arrays**, **Enums**, and **Structs**.

[Watch a video tutorial on Primitive Data Types.](#)

### ★ Assignment

1. Create a new variable `newAddr` that is a `public` `address` and give it a value that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

basicSyntax.sol

primitiveD

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract Primitives {
5 bool public boo = true;
6
7 /*
8 * uint stands for unsigned integer, meaning non negative integers
9 * different sizes are available
10 * uint8 ranges from 0 to 2 ** 8 - 1
11 * uint16 ranges from 0 to 2 ** 16 - 1
12 * ...
13 * uint256 ranges from 0 to 2 ** 256 - 1
14 */
15 uint8 public u8 = 1;
16 uint public u256 = 456;
17 uint public u = 123; // uint is an alias for uint256
18
19 /*
20 * Negative numbers are allowed for int types.
21 * Like uint, different ranges are available from int8 to int256
22 */
23 int8 public i8 = -1;
24 int public i256 = 456;
25 int public i = -123; // int is same as int256
26
27 address public addr = 0xCA35b7d915458Ef540aDe6068dFe2F44E8fa733c;
28
29 // Default values
30 // Unassigned variables have a default value
31 bool public defaultBool; // false
32 uint public defaultuint; // 0
```

## Tutorial - 4

LEARNETH

[Tutorials list](#)

[Syllabus](#)

4. Variables

4 / 19

Global variables, also called *special variables*, exist in the global namespace. They don't need to be declared but can be accessed from within your contract. Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

### ★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

Compile

introduction.sol

variables.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D28A51
4 contract Variables {
5 // State variables are stored on the blockchain.
6 string public text = "Hello";
7 uint public num = 123;
8 uint public blockNumber;
9
10 function doSomething() public { 22334 gas
11 // Local variables are not saved to the blockchain
12 uint i = 456;
13
14 // Here are some global variables
15 uint timestamp = block.timestamp; // Current block
16 address sender = msg.sender; // address of the caller
17 blockNumber = block.number;
18 }
19 }
```

Explain contract

## Tutorial - 5

LEARNETH

[Tutorials list](#)

[Syllabus](#)

5.1 Functions - Reading and Writing to a State Variable

5 / 19

To define a function, use the `function` keyword followed by a unique name.

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video [tutorial on Functions](#).

### ★ Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

introduction.sol

readAndWrite.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D28A51
4 contract SimpleStorage {
5 // State variable to store a number
6 uint public num;
7 bool public b = true;
8
9 // You need to send a transaction to write to a state variable
10 function set(uint _num) public { 22536 gas
11 num = _num;
12 }
13
14 // You can read from a state variable without sending a transaction
15 function get() public view returns (uint) { 2475 gas
16 return num;
17 }
18
19 function get_b() public view returns (bool) { 2536 gas
20 return b;
21 }
22 }
```

Explain contract

## Tutorial - 6

LEARNETH

Tutorials list

Syllabus

5.2 Functions - View and Pure

6 / 19

4. Calling any function not marked pure.

5. Using inline assembly that contains certain opcodes."

From the Solidity documentation,

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

★ Assignment

Create a function called `addTwoX` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

viewAndPure.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract ViewAndPure {
5 uint public x = 1;
6
7 // Promise not to modify the state.
8 function addTwoX(uint y) public view returns (uint) { Infinite gas
9 return x + y;
10 }
11
12 // Promise not to modify or read from the state.
13 function add(uint i, uint j) public pure returns (uint) { Infinite gas
14 return i + j;
15 }
16
17 function addTwoX2(uint y) public { Infinite gas
18 x = x + y;
19 }
20 }
```

## Tutorial - 7

LEARNETH

Tutorials list

Syllabus

5.3 Functions - Modifiers and Constructors

7 / 19

Constructor

A constructor function is executed upon the creation of a contract. You can use it to run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

★ Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.

2. Make sure that `x` can only be increased.

3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

visibility.sol

modifiersAndConstructors.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract FunctionModifier {
5 // We will use these variables to demonstrate how to use
6 // modifiers.
7 address public owner;
8 uint public x = 10;
9 bool public locked;
10
11 constructor() {
12 // Set the transaction sender as the owner of the contract.
13 owner = msg.sender;
14 }
15
16 // modifier to check that the caller is the owner of
17 // the contract.
18 modifier onlyOwner() {
19 require(msg.sender == owner, "Not owner");
20 // Underscore is a special character only used inside
21 // a function modifier and it tells Solidity to
22 // execute the rest of the code.
23 _;
24 }
25
26 // Modifiers can take inputs. This modifier checks that the
27 // address passed in is not the zero address.
28 modifier validAddress(address _addr) {
29 require(_addr != address(0), "Not valid address");
30 _;
31 }
32 }
```



## Tutorial - 8

LEARNETH

Tutorials list

Syllabus

5.4 Functions - Inputs and Outputs

8 / 19

### Input and Output restrictions

There are a few restrictions and best practices for the input and output parameters of contract functions.

*"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible."* From the Solidity documentation.

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

[Watch a video tutorial on Function Outputs.](#)

### ★ Assignment

Create a new function called `returnTwo` that returns the values `2` and `true`, without using a return statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

visibility.sol

inputsAndOutputs.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract Function {
5 // Functions can return multiple values.
6 function returnMany() infinite gas
7 {
8 public
9 pure
10 returns (
11 uint,
12 bool,
13 uint
14)
15 {
16 return (1, true, 2);
17 }
18
19 // Return values can be named.
20 function named() infinite gas
21 {
22 public
23 pure
24 returns (
25 uint x,
26 bool b,
27 uint y
28)
29 {
30 return (1, true, 2);
31 }
32
33 // Return values can be assigned to their name.
34 // In this case the return statement can be omitted.
```

## Tutorial - 9

LEARNETH

Tutorials list

Syllabus

6. Visibility

9 / 19

### external

- Can be called from other contracts or transactions
- State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

[Watch a video tutorial on Visibility.](#)

### ★ Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

visibility.sol

X

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract Base {
5 // Private function can only be called
6 // - inside this contract
7 // - contracts that inherit this contract cannot call this function.
8 function privateFunc() private pure returns (string memory) { infinite gas
9 return "private function called";
10 }
11
12 function testPrivateFunc() public pure returns (string memory) { infinite gas
13 return privateFunc();
14 }
15
16 // Internal function can be called
17 // - inside this contract
18 // - inside contracts that inherit this contract
19 function internalFunc() internal pure returns (string memory) { infinite gas
20 return "internal function called";
21 }
22
23 function testInternalFunc() public pure virtual returns (string memory) { infinite gas
24 return internalFunc();
25 }
26
27 // Public functions can be called
28 // - inside this contract
29 // - inside contracts that inherit this contract
30 // - by other contracts and accounts
31 function publicFunc() public pure returns (string memory) { infinite gas
32 return "public function called";
```

## Tutorial - 10

LEARNETH

Tutorials list

Syllabus

7.1 Control Flow - If/Else

10 / 19

in this contract the `gas` function uses the `gas` statement (line 10) to return `1` if none of the other conditions are met.

**else if**

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

**★ Assignment**

Create a new function called `evenCheck` in the `ifElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

ifElse.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract IfElse {
5 function foo(uint x) public pure returns (uint) {
6 if (x < 10) {
7 return 0;
8 } else if (x < 20) {
9 return 1;
10 } else {
11 return 2;
12 }
13 }
14
15 function ternary(uint _x) public pure returns (uint) {
16 // if (x < 10) {
17 // return 1;
18 // }
19 // return 2;
20
21 // shorthand way to write if / else statement
22 return _x < 10 ? 1 : 2;
23 }
24
25 function evenCheck(uint y) public pure returns (bool) {
26 return y%2 == 0 ? true : false;
27 }
28 }
```

## Tutorial - 11

LEARNETH

Tutorials list

Syllabus

7.2 Control Flow - Loops

11 / 19

executed at least once, before checking on the condition.

**continue**

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second `if` statement (line 12) from being executed.

**break**

The `break` statement is used to exit a loop. In this contract, the `break` statement (line 14) will cause the `for` loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

**★ Assignment**

1. Create a public `uint` state variable called `count` in the `Loop` contract.

2. At the end of the `for` loop, increment the `count` variable by 1.

3. Try to get the `count` variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

loops.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract Loop {
5 uint public count;
6 function loop() public {
7 // for loop
8 for (uint i = 0; i < 10; i++) {
9 if (i == 5) {
10 // skip to next iteration with continue
11 continue;
12 }
13 if (i == 5) {
14 // exit loop with break
15 break;
16 }
17 count++;
18 }
19
20 // while loop
21 uint j;
22 while (j < 10) {
23 j++;
24 }
25 }
26 }
27
```

## Tutorial - 12

LEARNETH

[Tutorials list](#)

[Syllabus](#)

8.1 Data Structures - Arrays

12 / 19

Using the `pop()` member function, we delete the last element of a dynamic array (line 31).

We can use the `delete` operator to remove an element with a specific index from an array (line 42). When we remove an element with the `delete` operator all other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

### Array length

Using the `length` member, we can read the number of elements that are stored in an array (line 35).

[Watch a video tutorial on Arrays.](#)

### ★ Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

arrays.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20AS1
4 contract Array {
5 // Several ways to initialize an array
6 uint[] public arr;
7 uint[] public arr2 = [1, 2, 3];
8 // Fixed sized array, all elements initialize to 0
9 uint[10] public myFixedSizeArr;
10 uint[3] public arr3 = [0, 1, 2];
11
12 function get(uint i) public view returns (uint) { Infinite gas
13 return arr[i];
14 }
15
16 // solidity can return the entire array.
17 // But this function should be avoided for
18 // arrays that can grow indefinitely in length.
19 function getArr() public view returns (uint[3] memory) { Infinite
20 return arr3;
21 }
22
23 function push(uint i) public { 46820 gas
24 // Append to array
25 // This will increase the array length by 1.
26 arr.push(i);
27 }
28
29 function pop() public { 29462 gas
30 // Remove last element from array
31 // This will decrease the array length by 1
32 arr.pop();
```

## Tutorial - 13

LEARNETH

[Tutorials list](#)

[Syllabus](#)

8.2 Data Structures - Mappings

13 / 19

### Setting values

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

### Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

[Watch a video tutorial on Mappings.](#)

### ★ Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

mappings.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20AS1
4 contract Mapping {
5 // Mapping from address to uint
6 mapping(address => uint) public balances;
7
8 function get(address _addr) public view returns (uint) { 2872 gas
9 // Mapping always returns a value.
10 // If the value was never set, it will return the default value
11 return balances[_addr];
12 }
13
14 function set(address _addr) public { 25250 gas
15 // Update the value at this address
16 balances[_addr] = _addr.balance;
17 }
18
19 function remove(address _addr) public { 5566 gas
20 // Reset the value to the default value.
21 delete balances[_addr];
22 }
23 }
24
25 contract NestedMapping {
26 // Nested mapping (mapping from address to another mapping)
27 mapping(address => mapping(uint => bool)) public nested;
28
29 function get(address _addr1, uint _i) public view returns (bool) {
30 // You can get values from a nested mapping
31 // even when it is not initialized
32 return nested[_addr1][_i];
```



## Tutorial - 14

LEARNETH

Tutorials list

Syllabus

8.3 Data Structures - Structs

14 / 19

members as parameters in parentheses (line 16).

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct. We initialize an empty struct first and then update its member by assigning it a new value (line 23).

### Accessing structs

To access a member of a struct we can use the dot operator (line 33).

### Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

[Watch a video tutorial on Structs.](#)

### ★ Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

structs.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract Todos {
5 struct Todo {
6 string text;
7 bool completed;
8 }
9
10 // An array of 'Todo' structs
11 Todo[] public todos;
12
13 function create(string memory _text) public {
14 // 3 ways to initialize a struct
15 // - calling it like a function
16 todos.push(Todo(_text, false));
17
18 // key value mapping
19 todos.push(Todo({text: _text, completed: false}));
20
21 // initialize an empty struct and then update it
22 Todo memory todo;
23 todo.text = _text;
24 // todo.completed initialized to false
25
26 todos.push(todo);
27 }
28
29 // Solidity automatically created a getter for 'todos' so
30 // you don't actually need this function.
31 function get(uint _index) public view returns (string memory text, bool completed) {
32 Todo storage todo = todos[_index];
```

## Tutorial - 15

LEARNETH

Tutorials list

Syllabus

8.4 Data Structures - Enums

15 / 19

### Updating an enum value

We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

### Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

### ★ Assignment

- Define an enum type called `Size` with the members `S`, `M`, and `L`.
- Initialize the variable `sizes` of the enum type `Size`.
- Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

enums.sol

```
17
18
19
20 // Default value is the first element listed in
21 // definition of the type, in this case "Pending"
22 status public status;
23 size public sizes;
24
25 function get() public view returns (Status) {
26 return status;
27 }
28
29 function getSize() public view returns (Size) {
30 return sizes;
31 }
32
33 // Update status by passing uint into input
34 function set(Status _status) public {
35 status = _status;
36 }
37
38 // You can update to a specific enum like this
39 function cancel() public {
40 status = Status.Canceled;
41 }
42
43 // delete resets the enum to its first value, 0
44 function reset() public {
45 delete status;
46 }
47 }
```

## Tutorial - 16

LEARNETH

Tutorials list

Syllabus

9. Data Locations

16 / 19

function f (line 12) and assign it the value of myStruct, changes in myMemStruct3 would not affect the values stored in the mapping myStructs (line 10).

As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ Assignment

- Change the value of the myStruct member foo, inside the function f, to 4.
- Create a new struct myMemStruct2 with the data location memory inside the function f and assign it the value of myMemStruct. Change the value of the myMemStruct2 member foo to 1.
- Create a new struct myMemStruct3 with the data location memory inside the function f and assign it the value of myStruct. Change the value of the myMemStruct3 member foo to 3.
- Let the function f return myStruct, myMemStruct2, and myMemStruct3.

Tip: Make sure to create the correct return types for the function f.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

dataLocations.sol 2 X

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract DataLocations {
5 uint[] public arr;
6 mapping(uint => address) map;
7 struct MyStruct {
8 uint foo;
9 }
10 mapping(uint => MyStruct) public myStructs;
11
12 function f() public returns (MyStruct memory, MyStruct memory, MyStr
13 // call f with state variables
14 _f(arr, map, myStructs[1]);
15 // get a struct from a mapping
16 MyStruct storage myStruct = myStructs[1];
17 myStruct.foo = 4;
18 // create a struct in memory
19 MyStruct memory myMemStruct = MyStruct(0);
20 MyStruct memory myMemStruct2 = myMemStruct;
21 myMemStruct2.foo = 1;
22
23 MyStruct memory myMemStruct3 = myStruct;
24 myMemStruct3.foo = 3;
25 return (myStruct, myMemStruct2, myMemStruct3);
26 }
27
28 function _f(
29 uint[] storage _arr,
30 mapping(uint => address) storage _map,
31 MyStruct storage _myStruct
32) internal {
```

## Tutorial - 17

LEARNETH

Tutorials list

Syllabus

10.1 Transactions - Ether and Wei

17 / 19

to specify a unit of Ether, we can add the suffixes wei, gwei, or ether to a literal number.

wei

Wei is the smallest subunit of Ether, named after the cryptographer Wei Dai. Ether numbers without a suffix are treated as wei (line 7).

gwei

One gwei (giga-wei) is equal to 1,000,000,000 (10^9) wei.

ether

One ether is equal to 1,000,000,000,000,000,000 (10^18) wei (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

- Create a public uint called oneGwei and set it to 1 gwei.
- Create a public bool called isOneGwei and set it to the result of a comparison operation between 1 gwei and 10^9.

Tip: Look at how this is written for gwei and ether in the contract.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

etherAn

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare D20A51
4 contract EtherUnits {
5 uint public oneWei = 1 wei;
6 // 1 wei is equal to 1
7 bool public isOneWei = 1 wei == 1;
8
9 uint public oneEther = 1 ether;
10 // 1 ether is equal to 10^18 wei
11 bool public isOneEther = 1 ether == 1e18;
12
13 uint public oneGwei = 1 gwei;
14 // 1 ether is equal to 10^9 wei
15 bool public isOneGwei = 1 gwei == 1e9;
16 }
```



## Tutorial - 18

The screenshot shows the LearnNETH interface for Tutorial 18, titled "10.2 Transactions - Gas and Gas Price". The left sidebar contains a "Tutorials list" and a "Syllabus" button. The main content area explains that gas prices are denoted in gwei and defines the "Gas limit" as the maximum amount of gas a sender is willing to pay for. It includes a tip about checking transaction details in the Remix terminal and using the Gas Profiler. Below the text is an "Assignment" section with instructions to create a new public state variable named `cost` of type `uint` in the `Gas` contract. At the bottom, there are "Check Answer" and "Show answer" buttons, a "Next" button, and a green status bar indicating "Well done! No errors."

LEARNETH

Tutorials list Syllabus

10.2 Transactions - Gas and Gas Price 18 / 19

Gas prices are denoted in gwei.

### Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about *gas* on [ethereum.org](https://ethereum.org).

Watch a video tutorial on Gas and Gas Price.

### ★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare 028A51
4 contract Gas {
5 uint public i = 0;
6 uint public cost = 170367;
7
8 // Using up all of the gas that you send causes your trans
9 // State changes are undone.
10 // Gas spent are not refunded.
11 function forever() public { // Infinite gas
12 // Here we run a loop until all of the gas are spent.
13 // and the transaction fails.
14 while (true) {
15 i += 1;
16 }
17 }
18 }
```

## Tutorial - 19

The screenshot shows the LearnNETH interface for Tutorial 19, titled "10.3 Transactions - Sending Ether". The left sidebar contains a "Tutorials list" and a "Syllabus" button. The main content area explains that changing the parameter type for `sendViaTransfer` and `sendViaSend` from `payable address` to `address` allows the use of `transfer()` or `send()`. It includes a tip about testing the contract by deploying it and sending Ether. Below the text is an "Assignment" section with instructions to build a charity contract that receives Ether and can be withdrawn by a beneficiary. The assignment steps are: 1. Create a contract called `Charity`. 2. Add a public state variable called `owner` of the type `address`. 3. Create a donate function that is public and payable without any parameters or function code. 4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address. At the bottom, there are "Check Answer" and "Show answer" buttons, a "Next" button, and a green status bar indicating "Well done! No errors."

LEARNETH

Tutorials list Syllabus

10.3 Transactions - Sending Ether 19 / 19

If you change the parameter type for the functions `sendViaTransfer` and `sendViaSend` (line 33 and 38) from `payable address` to `address`, you won't be able to use `transfer()` (line 35) or `send()` (line 41).

Watch a video tutorial on Sending Ether.

### ★ Assignment

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Ansh Sarfare 028A51
4 contract ReceiveEther {
5 /*
6 * Which function is called, fallback() or receive()?
7 *
8 * send Ether
9 * |
10 * msg.data is empty?
11 * / \
12 * yes no
13 * / \
14 * receive() exists? fallback()
15 * / \
16 * yes no
17 * / \
18 * receive() fallback()
19 */
20
21 // Function to receive Ether. msg.data must be empty
22 receive() external payable {} // undefined gas
23
24 // Fallback function is called when msg.data is not empty
25 fallback() external payable {} // undefined gas
26
27 function getBalance() public view returns (uint) { // 312 gas
28 return address(this).balance;
29 }
30 }
31
32 contract SendEther {
```

**Conclusion:** Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801 : Blockchain & DLT Lab**  
**Experiment 05**

Aim: Deploying a Voting/Ballot Smart Contract

|           |                                                                                                                                                                                                                                                                   |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                                                                                                                                                                                                |
| Name      | Ansh Sarfare                                                                                                                                                                                                                                                      |
| Class     | D20A                                                                                                                                                                                                                                                              |
| Subject   | ITC801: Blockchain and DLT Lab                                                                                                                                                                                                                                    |
| CO Mapped | CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology.<br>CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions<br>CO3:Implement smart contracts in Ethereum using different development frameworks. |

## Blockchain Exp- 5

**AIM:** Deploying a Voting/Ballot Smart Contract

Tasks:

1. Open [Remix IDE](#)
2. Under **Workspaces**, open **contracts** folder
3. Open **Ballot.sol**, contract.
4. Understand **Ballot.sol** contract.
5. Deploy the contract by changing the Proposal name from **bytes32** → **string**

### **THEORY:**

#### **1. Relevance of require Statements in Solidity Programs**

In Solidity, the `require` statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside `require` is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a Voting Smart Contract, `require` can be used to check:

- Whether the person calling the function has the right to vote  
(`require(voters[msg.sender].weight > 0, "Has no right to vote");`).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, `require` statements enforce security, correctness, and reliability in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

#### **2. Keywords: mapping, storage, and**

##### **memory mapping:**

- A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is `mapping(keyType => valueType)`. For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like Ballot, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them gas efficient for lookups but limited for enumeration.

### **storage:**

- In Solidity, storage refers to the permanent memory of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

### **memory:**

- In contrast, memory is temporary storage, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must balance between storage and memory to ensure efficiency and cost-effectiveness.

- **bytes32** is a fixed-size type, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a dynamically sized type, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the Web3 Type Converter help developers easily switch between these two types for deployment and testing.

## CODE:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {

 struct Voter {
 uint
 weight;
 bool voted;
 address
 delegate; uint
 vote;
 }

 struct Proposal {
 string name; // CHANGED from bytes32 →
 string uint voteCount;
 }

 address public chairperson;
 mapping(address => Voter) public voters;
 Proposal[] public proposals;

 /**
 * @dev Create a new ballot
 * @param proposalNames names of proposals
 */
 constructor(string[] memory proposalNames)
 {
 chairperson = msg.sender;
 voters[chairperson].weight = 1;
 }
}
```

```

for (uint i = 0; i < proposalNames.length; i++) {
 proposals.push(
 Proposal({
 name: proposalNames[i],
 voteCount: 0
 })
);
}
}

```

```

function giveRightToVote(address voter) external {
 require(msg.sender == chairperson, "Only chairperson can give right to vote");
 require(!voters[voter].voted, "The voter already voted");
 require(voters[voter].weight == 0, "Voter already has right");

 voters[voter].weight = 1;
}

```

```

function delegate(address to) external {
 Voter storage sender =
 voters[msg.sender];

 require(sender.weight != 0, "No right to vote");
 require(!sender.voted, "Already voted");
 require(to != msg.sender, "Self-delegation not allowed");

 while (voters[to].delegate != address(0))
 {
 to = voters[to].delegate;
 require(to != msg.sender, "Delegation loop detected");
 }

 Voter storage delegate_ = voters[to];
 require(delegate_.weight >= 1, "Delegate has no right");

 sender.voted = true;
}

```

```

sender.delegate = to;

if (delegate_.voted) {
 proposals[delegate_.vote].voteCount +=
 sender.weight;
} else {
 delegate_.weight += sender.weight;
}
}

function vote(uint proposal) external {
 Voter storage sender = voters[msg.sender];

 require(sender.weight != 0, "No right to vote");
 require(!sender.voted, "Already voted");

 sender.voted = true;
 sender.vote = proposal;

 proposals[proposal].voteCount += sender.weight;
}

function winningProposal() public view returns (uint winningProposal_) {
 uint winningVoteCount = 0;

 for (uint p = 0; p < proposals.length; p++) {
 if (proposals[p].voteCount > winningVoteCount) {
 winningVoteCount = proposals[p].voteCount;
 winningProposal_ = p;
 }
 }
}

function winnerName() external view returns (string memory) {
 return proposals[winningProposal()].name;
}

```

```
}
}
}
```

## OUTPUT:

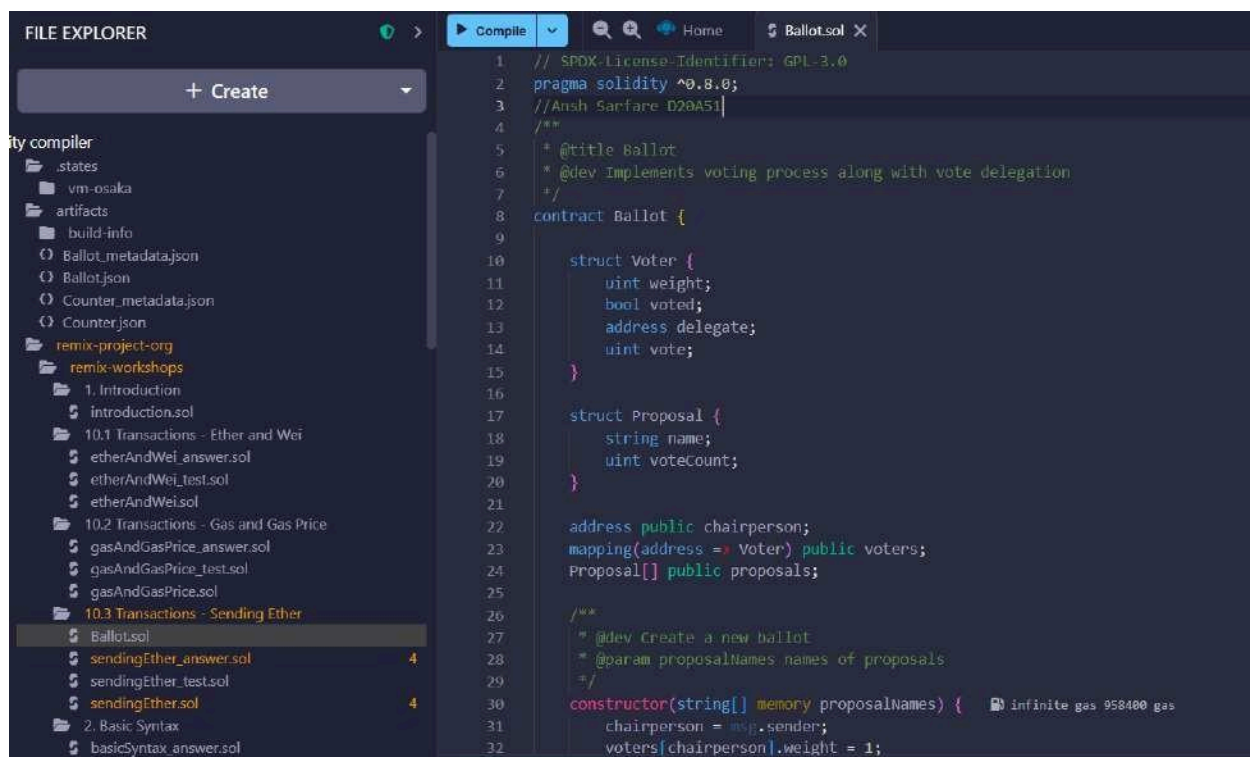
### Step 1: Open Remix

Go to --- <https://remix.ethereum.org>

No install needed.

### Step 2: Create the file

- In File Explorer



**Ballot.sol**

### Step 3: Compile the contract

1. Open Solidity Compiler (left sidebar)
2. Click Compile Ballot.sol
3. Make sure green tick appears (no

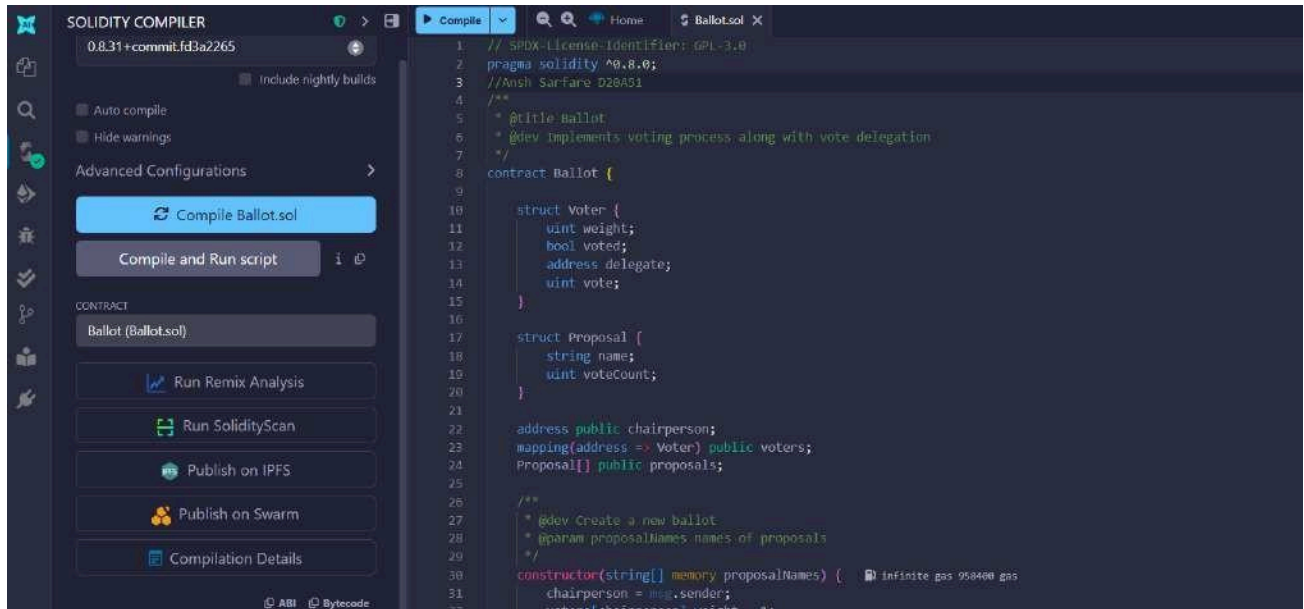
errors) Step 4: Deploy the contract



## 1. Open Deploy & Run Transactions

## 2. Set:

- Environment → **Remix VM**
- Account → keep default
- Contract → **Ballot**



## Step 5: Enter constructor input

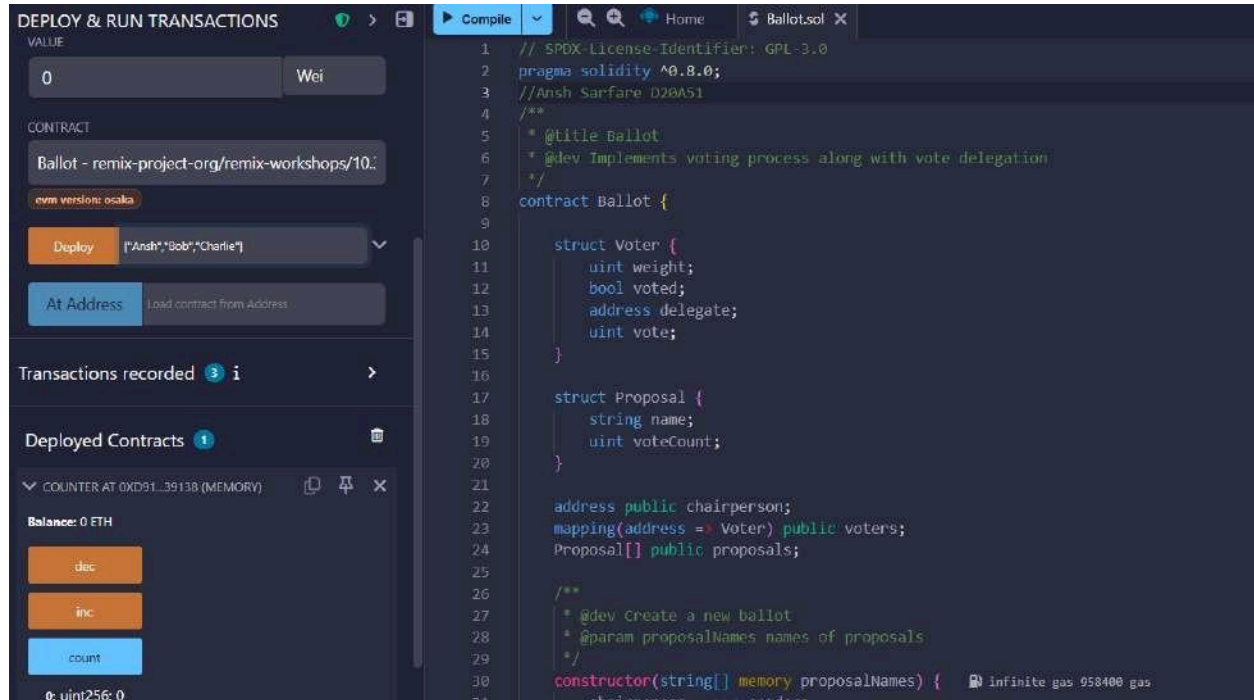
(IMPORTANT) Your constructor is:

**constructor(string[] memory proposalNames)**

So input must be a string array **)r)**

Example:

**["Ansh", "Bob", "Charlie"]**



## Step 6: Click Deploy

- **Click Deploy**
- **Contract appears under Deployed Contracts**
- **The deployer is automatically the**

**chairperson** **giveRightToVote** (address voter)

### What to add:

**Paste a Remix account address (NOT the chairperson).**

**Important:**

- **You must be using Account 1 (chairperson) when clicking this**
- **Switch account at the top if**

needed Click Transact

Then vote

2 vote (uint256 proposal)

What to add:

A number, based on proposal index:

0 → Ansh

1 → Bob

2 → Charlie

Example:

0

Important:

- Switch to the voter account (the one you gave rights to)
- Then click Transact

The screenshot displays a web application interface for a voting system. On the left, the 'Deployed Contracts' panel shows two contracts: 'COUNTER AT 0x091...39138 (MEMORY)' and 'BALLOT AT 0xA2E...DCB97 (MEMORY)'. The 'BALLOT' contract is selected, showing its 'Balance: 0 ETH' and a 'delegate' button. Below this, the 'GIVERIGHTTOVOTE' section shows a 'voter' field with the address '0xA0483F6d9C6d1E1F96846A657dD315E' and a 'transact' button. The 'VOTE' section shows a 'proposal' field with the value 'uint256' and a 'transact' button. On the right, the transaction log shows a series of transactions, including the creation of the Counter, a call to Counter.get(), a transaction to Counter.inc pending, a call to Counter.inc(), a transaction to Counter.dec pending, a call to Counter.dec(), a call to Counter.count, a transaction to Counter.count pending, a call to Counter.count(), a transaction to Counter.count pending, a call to Counter.count(), a transaction to Counter.count pending, a call to Counter.count(), a transaction to Counter.count pending, and a call to Counter.count(). Each transaction is accompanied by a 'Debug' button.

DEPLOY & RUN TRANSACTIONS

COUNTER AT 0x091...39138 (MEMORY)

BALLOT AT 0xA3B...DCB97 (MEMORY)

Balance: 0 ETH

delegate

address to

GIVERIGHTTOVOTE

vote: 0xA1B8483F6449C6d1fcF9b849Ae677d03151

CallData

Parameters

transact

VOTE

proposal: 0

CallData

Parameters

transact

chairperson

proposals: 0x4254

votes: address

winnerName

winningProposal

Low level Interactions

Listen on all transactions

Filter with transaction hash or address

[vm] from: 0x583...ed8C4 to: Counter.(constructor) value: 0 wei data: 0x08...f0033 logs: 0 hash: 0x955...0383c

call to Counter.get

OK [call] from: 0x58380a6a701c568545dcfc803fc8075f56beddC4 to: Counter.get() data: 0x04...ce63c

transact to Counter.inc pending ...

[vm] from: 0x583...ed8C4 to: Counter.inc() 0x091...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee505

transact to Counter.dec pending ...

[vm] from: 0x583...ed8C4 to: Counter.dec() 0x091...39138 value: 0 wei data: 0xb3b...cfab3 logs: 0 hash: 0xccc...79d46

call to Counter.count

OK [call] from: 0x58380a6a701c568545dcfc803fc8075f56beddC4 to: Counter.count() data: 0x066...51ab

creation of Ballot pending...

[vm] from: 0x583...ed8C4 to: Ballot.(constructor) value: 0 wei data: 0x68...00000 logs: 0 hash: 0xc27...4f088

transact to Ballot.giveRightToVote pending ...

[vm] from: 0x583...ed8C4 to: Ballot.giveRightToVote(address) 0xa2e...DCB97 value: 0 wei data: 0xb07...35cb2 logs: 0 hash: 0xf4b...ed2fa

transact to Ballot.vote pending ...

[vm] from: 0xA08...35cb2 to: Ballot.vote(uint256) 0xa2e...DCB97 value: 0 wei data: 0x012...00000 logs: 0 hash: 0xf44...06c65

DEPLOY & RUN TRANSACTIONS

GIVERIGHTTOVOTE

vote: 0xA1B8483F6449C6d1fcF9b849Ae677d03151

CallData

Parameters

transact

VOTE

proposal: 0

CallData

Parameters

transact

chairperson

PROPOSALS

0

CallData

Parameters

call

0 string: name Ansh

1 uint256: voteCount 1

votes

address

winnerName

winningProposal

Low level Interactions

Listen on all transactions

Filter with transaction hash or address

OK [call] from: 0x58380a6a701c568545dcfc803fc8075f56beddC4 to: Counter.get() data: 0xb0d...ce63c

transact to Counter.inc pending ...

[vm] from: 0x583...ed8C4 to: Counter.inc() 0x091...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee505

transact to Counter.dec pending ...

[vm] from: 0x583...ed8C4 to: Counter.dec() 0x091...39138 value: 0 wei data: 0xb3b...cfab3 logs: 0 hash: 0xccc...79d46

call to Counter.count

OK [call] from: 0x58380a6a701c568545dcfc803fc8075f56beddC4 to: Counter.count() data: 0x066...51ab

creation of Ballot pending...

[vm] from: 0x583...ed8C4 to: Ballot.(constructor) value: 0 wei data: 0x68...00000 logs: 0 hash: 0xc27...4f088

transact to Ballot.giveRightToVote pending ...

[vm] from: 0x583...ed8C4 to: Ballot.giveRightToVote(address) 0xa2e...DCB97 value: 0 wei data: 0xb07...35cb2 logs: 0 hash: 0xf4b...ed2fa

transact to Ballot.vote pending ...

[vm] from: 0xA08...35cb2 to: Ballot.vote(uint256) 0xa2e...DCB97 value: 0 wei data: 0x012...00000 logs: 0 hash: 0xf44...06c65

call to Ballot.proposals

OK [call] from: 0xA1B8483F6449C6d1fcF9b849Ae677d03151835cb2 to: Ballot.proposals(uint256) data: 0x813...00000

## proposals (uint256)

**What to add:**

**Proposal index:**

**0**

**Returns proposal details.**

The screenshot displays the Remix IDE interface. On the left, the 'VOTE' contract is shown with state variables: `proposal: 0`, `chairperson`, `PROPOSALS`, `0`, `0 string: name Ansh`, `1: uint256: voteCount 1`, `voters`, `winnerName`, `0 string: Ansh`, and `winningProposal`. The main area shows the transaction log with the following entries:

- `ou: [call] from: 0x58380a6a701c568545dCfc983Fc8875f56beddC4 to: Counter.count() data: 0x066...61abd`
- `creation of Ballot pending...`
- `[vm] from: 0x583...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0xc27...4f808`
- `transact to Ballot.giveRightToVote pending...`
- `[vm] from: 0x583...eddC4 to: Ballot.giveRightToVote(address) 0xa2e...DCb97 value: 0 wei data: 0x9e7...35c02 logs: 0 hash: 0x9a0...ed2fa`
- `transact to Ballot.vote pending...`
- `[vm] from: 0xab8...35cb2 to: Ballot.vote(uint256) 0xa2e...DCb97 value: 0 wei data: 0x012...00000 logs: 0 hash: 0xf44...06c65`
- `call to Ballot.proposals`
- `ou: [call] from: 0xab8483f64d9C6d1FcF9b849Ae677d03315835cb2 to: Ballot.proposals(uint256) data: 0x013...00000`
- `call to Ballot.winnerName`
- `ou: [call] from: 0xab8483f64d9C6d1FcF9b849Ae677d03315835cb2 to: Ballot.winnerName() data: 0xe2b...a53f8`

## Conclusion:

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.





**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801 : Blockchain & DLT Lab**  
**Experiment 06**

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

|           |                                                                                                                                                                                                                                                                                                                                  |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                                                                                                                                                                                                                                                               |
| Name      | Ansh Sarfare                                                                                                                                                                                                                                                                                                                     |
| Class     | D20A                                                                                                                                                                                                                                                                                                                             |
| Subject   | ITC801: Blockchain and DLT Lab                                                                                                                                                                                                                                                                                                   |
| CO Mapped | CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology.<br>CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions<br>CO3:Implement smart contracts in Ethereum using different development frameworks.<br>CO5:Interpret different Crypto assets and Crypto currencies |

## **BC Experiment-6**

**AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE**

### **THEORY:**

**What is a Smart Contract?** A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity and executed on the Ethereum Virtual Machine (EVM).

### **Significance of Smart Contracts in the Ethereum Blockchain:**

- **Automation:** Executes actions automatically when conditions are satisfied.
- **Transparency:** Contract code and transactions are visible on the blockchain.
- **Security:** Once deployed, contracts are immutable and tamper-proof.
- **Cost-effective:** Removes intermediaries and reduces transaction costs.
- **Trustless System:** Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

**About the Crowdfunding Platform:** A blockchain-based crowdfunding platform replaces centralized authorities with smart contracts. Two contracts are designed — **CrowdfundingContract.sol**, which manages campaign creation and tracks donations, and **DonorContract.sol**, which handles donor interactions and communicates with the CrowdfundingContract through a Solidity interface. Together they ensure a transparent, trustless, and intermediary-free fundraising system.

### **Tools Used:**

- **Solidity** — High-level language for writing smart contracts on Ethereum.
- **Remix IDE** — Browser-based IDE for writing, compiling, deploying, and testing smart contracts using a built-in Remix VM.

### **IMPLEMENTATION:**

*CrowdfundingContract.sol*

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
```

```
contract CrowdfundingContract {
```



```

struct Campaign {
 uint256
 campaignId; string
 title;
 uint256 goalAmount;
 uint256 raisedAmount;
 uint256 deadline;
 address payable creator;
 bool isActive;
}

uint256 public campaignCount = 0;
mapping(uint256 => Campaign) public campaigns;

 event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount,
address creator);
 event FundsReceived(uint256 campaignId, uint256 amount);

function
 createCampaign(
 string memory _title,
 uint256 _goalAmount,
 uint256 _durationDays
) public {
 campaignCount++;
 uint256 deadline = block.timestamp + (_durationDays * 1 days);
 campaigns[campaignCount] = Campaign(
 campaignCount,
 _title,
 _goalAmount,
 0,
 deadline,
 payable(msg.sender),
 true
);
 emit CampaignCreated(campaignCount, _title, _goalAmount, msg.sender);
 }

function receiveDonation(uint256 _campaignId, uint256 _amount) external {
 Campaign storage c = campaigns[_campaignId];
 require(c.isActive, "Campaign is not active");
 c.raisedAmount += _amount;
 emit FundsReceived(_campaignId, _amount);
}

function getCampaign(uint256 _campaignId) external view returns (
 uint256, string memory, uint256, uint256, uint256, address, bool

```

```

){
 Campaign storage c = campaigns[_campaignId];
 return (
 c.campaignId,
 c.title,
 c.goalAmount,
 c.raisedAmount,
 c.deadline,
 c.creator,
 c.isActive
);
 }

 function closeCampaign(uint256 _campaignId) external {
 campaigns[_campaignId].isActive = false;
 }
}

```

### DonorContract.sol

// SPDX-License-Identifier: GPL-3.0

pragma solidity ^0.8.0;

```

interface ICrowdfundingContract {
 function receiveDonation(uint256 _campaignId, uint256 _amount) external;
 function getCampaign(uint256 _campaignId) external view returns (
 uint256, string memory, uint256, uint256, uint256, address, bool
);
}

contract DonorContract {

 struct Donation {
 uint256 donationId;
 uint256
 campaignId;
 address donor;
 uint256 amount;
 uint256 timestamp;
 }

 uint256 public donationCount = 0;
 mapping(uint256 => Donation) public donations;
 address public crowdfundingContractAddress;

 event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256
amount);
 constructor(address _crowdfundingAddress) {
 crowdfundingContractAddress = _crowdfundingAddress;
 }
}

```

```

function donate(uint256 _campaignId, uint256 _amount) public {
 donationCount++;
 donations[donationCount] = Donation(
 donationCount,
 _campaignId,
 msg.sender,
 _amount,
 block.timestamp
);
 ICrowdfundingContract(crowdfundingContractAddress)
 .receiveDonation(_campaignId, _amount);

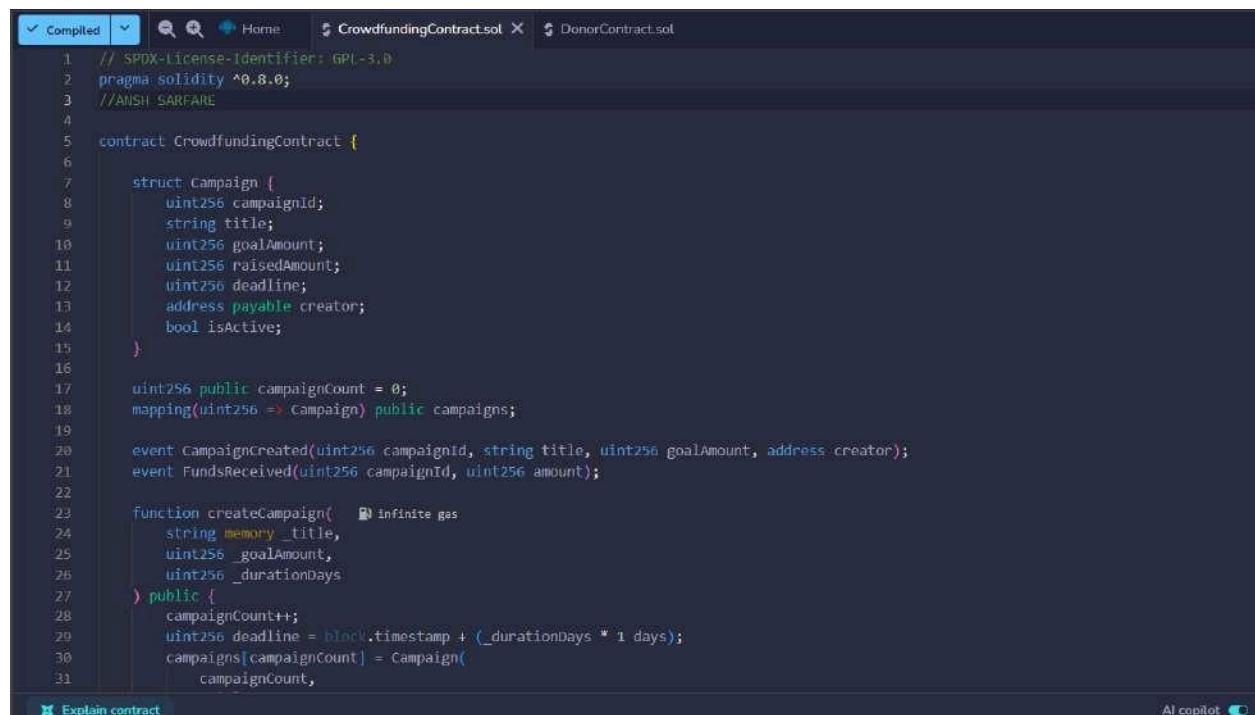
 emit DonationMade(donationCount, _campaignId, msg.sender, _amount);
}

function getDonation(uint256 _donationId) public view returns (
 uint256, uint256, address, uint256, uint256
){
 Donation storage d = donations[_donationId];
 return (d.donationId, d.campaignId, d.donor, d.amount, d.timestamp);
}

function getCrowdfundingContract() public view returns (address) {
 return crowdfundingContractAddress;
}
}

```

*Compilation:*



```

1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.0;
3 //ANSH SARFARE
4
5 contract CrowdfundingContract {
6
7 struct Campaign {
8 uint256 campaignId;
9 string title;
10 uint256 goalAmount;
11 uint256 raisedAmount;
12 uint256 deadline;
13 address payable creator;
14 bool isActive;
15 }
16
17 uint256 public campaignCount = 0;
18 mapping(uint256 => Campaign) public campaigns;
19
20 event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount, address creator);
21 event FundsReceived(uint256 campaignId, uint256 amount);
22
23 function createCampaign(Infinite gas
24 string memory _title,
25 uint256 _goalAmount,
26 uint256 _durationDays
27) public {
28 campaignCount++;
29 uint256 deadline = block.timestamp + (_durationDays * 1 days);
30 campaigns[campaignCount] = Campaign(
31 campaignCount,

```

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.0;
3 //ANSH SARKARE
4
5 interface ICrowdfundingContract {
6 function receiveDonation(uint256 _campaignId, uint256 _amount) external;
7 function getCampaign(uint256 _campaignId) external view returns (
8 uint256, string memory, uint256, uint256, address, bool
9);
10 }
11
12 contract DonorContract {
13
14 struct Donation {
15 uint256 donationId;
16 uint256 campaignId;
17 address donor;
18 uint256 amount;
19 uint256 timestamp;
20 }
21
22 uint256 public donationCount = 0;
23 mapping(uint256 => Donation) public donations;
24 address public crowdfundingContractAddress;
25
26 event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256 amount);
27
28 constructor(address _crowdfundingAddress) {
29 crowdfundingContractAddress = _crowdfundingAddress;
30 }
31 }
```

## DEPLOYMENT:

DEPLOY & RUN TRANSACTIONS

Environment

Remix VM

Ansh 99.999 ETH

value 0 wei

Gas limit 0

Deploy

Deployed Contracts 2

CrowdfundingContract

DonorContract

DonorContract.sol

```
8 uint256, string memory, uint256, uint256, address, bool
9);
10 }
11
12 contract DonorContract {
13
14 struct Donation {
15 uint256 donationId;
16 uint256 campaignId;
17 address donor;
18 uint256 amount;
19 uint256 timestamp;
20 }
21 }
```

Explain contract

You can use this terminal to:

- Check transactions details and start debugging.
- Execute JavaScript scripts:
  - Insert a script directly in the command line interface
  - Select a JavaScript file in the file explorer and then run "remix.execute()" or "remix.executeCurrent()" in the command line interface
  - Right-click on a JavaScript file in the file explorer and then click "Run"

The following libraries are accessible:

silence.js

Type the library name to see available commands.

[vm] from: 0x583...addc6 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x686...28033 logs: 0 hash: 0x992...9114

[vm] from: 0x583...addc6 to: DonorContract.(constructor) value: 0 wei data: 0x686...39138 logs: 0 hash: 0x6dc...5f0db

## TRANSACTIONS:

**DEPLOY & RUN TRANSACTIONS**

Environment: Remix VM, Paris, anish (0x682...addC4) 99.999 ETH

Transaction: createCampaign (string,uint256,uint256)

Value: 0 wei, Gas limit: auto, 0

Transact

Balance: 0 ETH

**Explain contract**

Listen on all transactions

Filter with transaction hash or address

The following libraries are accessible:

- @openzeppelin

Type the library name to see available commands.

- [vm] from: 0x583...addC4 to: CrowdfundingContract.constructor value: 0 wei data: 0x688...20033 logs: 0 hash: 0x992...91144
- [vm] from: 0x583...addC4 to: DonorContract.constructor value: 0 wei data: 0x688...39138 logs: 0 hash: 0x6dc...5f0db
- [vm] from: 0x583...addC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000 logs: 1 hash: 0x58d...ef6f6

## DONATIONS

**Low level interaction**

Transaction: donate (uint256,uint256)

Value: 1, Gas limit: 500

Transact

Balance: 0 ETH

**Explain contract**

Listen on all transactions

Filter with transaction hash or address

- [vm] from: 0x583...addC4 to: CrowdfundingContract.constructor value: 0 wei data: 0x688...20033 logs: 0 hash: 0x992...91144
- [vm] from: 0x583...addC4 to: DonorContract.constructor value: 0 wei data: 0x688...39138 logs: 0 hash: 0x6dc...5f0db
- [vm] from: 0x583...addC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000 logs: 1 hash: 0x58d...ef6f6
- [vm] from: 0x583...addC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000 logs: 1 hash: 0x58d...ef6f6
- [vm] from: 0x583...addC4 to: DonorContract.donate(uint256,uint256) 0xd8b...33fa8 value: 0 wei data: 0x6dc...001f4 logs: 2 hash: 0x349...b93d0

**Low level interaction**

Transaction: getDonation (uint256)

Value: 1, Gas limit: 500

Call

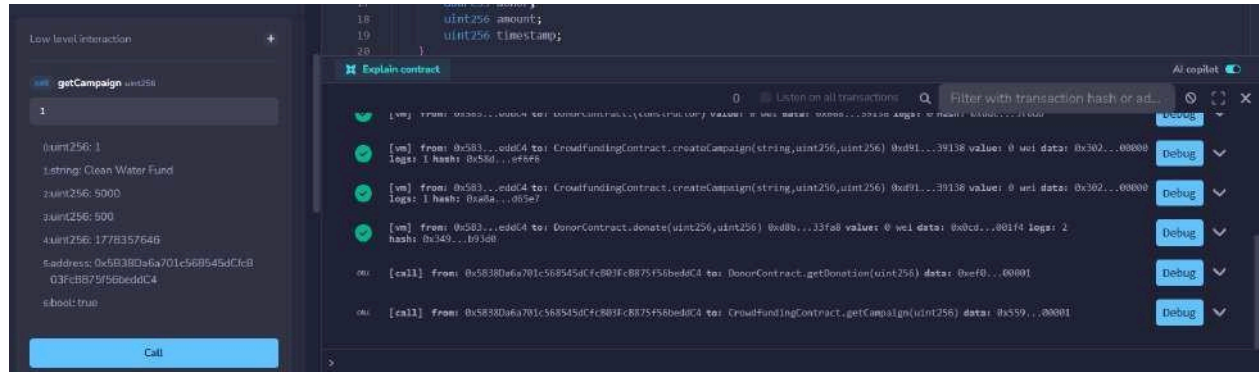
Balance: 0 ETH

**Explain contract**

Listen on all transactions

Filter with transaction hash or address

- [vm] from: 0x583...addC4 to: CrowdfundingContract.constructor value: 0 wei data: 0x688...20033 logs: 0 hash: 0x992...91144
- [vm] from: 0x583...addC4 to: DonorContract.constructor value: 0 wei data: 0x688...39138 logs: 0 hash: 0x6dc...5f0db
- [vm] from: 0x583...addC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000 logs: 1 hash: 0x58d...ef6f6
- [vm] from: 0x583...addC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000 logs: 1 hash: 0x58d...ef6f6
- [vm] from: 0x583...addC4 to: DonorContract.donate(uint256,uint256) 0xd8b...33fa8 value: 0 wei data: 0x6dc...001f4 logs: 2 hash: 0x349...b93d0
- [call] from: 0x5838D6a781c568545dCf7B03Fc0876f56beddC4 to: DonorContract.getDonation(uint256) data: 0xf0...00001



## **CONCLUSION:**

This experiment helped in understanding how smart contracts can automate and secure a crowdfunding platform on the Ethereum blockchain. Using Solidity and Remix IDE, two contracts — CrowdfundingContract and DonorContract — were designed, deployed, and tested. The CrowdfundingContract manages campaign creation and tracks donations, while the DonorContract handles donor interactions. Together they demonstrate a trustless, transparent, and intermediary-free fundraising system.



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801 : Blockchain & DLT Lab**

**Experiment 07**

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

|           |                                                                                                                                                                                                                                                                                                                                  |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                                                                                                                                                                                                                                                               |
| Name      | Ansh Sarfare                                                                                                                                                                                                                                                                                                                     |
| Class     | D20A                                                                                                                                                                                                                                                                                                                             |
| Subject   | ITC801: Blockchain and DLT Lab                                                                                                                                                                                                                                                                                                   |
| CO Mapped | CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology.<br>CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions<br>CO3:Implement smart contracts in Ethereum using different development frameworks.<br>CO5:Interpret different Crypto assets and Crypto currencies |

## **BC Experiment-7**

**Aim:** Implement the embedding wallet (Metamask) and transaction using Solidity

**Theory:**

MetaMask is a cryptocurrency wallet and gateway to blockchain applications. It allows users to store, send, and receive Ether or tokens, and connect to decentralized applications (DApps) directly from a web browser. MetaMask acts as a bridge between a user's browser and the Ethereum blockchain (or other compatible networks).

- **Features:**
  - Easy management of multiple blockchain accounts
  - Secure private key storage
  - Integration with test networks like Sepolia, Linea, or RSK
  - Interaction with DApps and smart contracts via Remix IDE
- **Testnet:**

A Testnet is a blockchain network used for testing and development without using real cryptocurrency. It mimics the main Ethereum network but uses test Ether (fake ETH). Common testnets include Sepolia Testnet, Linea Testnet, and RSK Testnet.
- **Advantages:**
  - No real money loss during testing
  - Safe environment for developers
  - Helps test smart contracts and transactions before mainnet deployment
- **Steps to Connect MetaMask with Remix IDE for Performing Transactions:**
- **Set Up MetaMask:**
  - Install the MetaMask extension on your browser.
  - Create or import a wallet account using a recovery phrase.
- **Add a Test Network:**
  - Open MetaMask → Settings → Networks → Add Network.
  - Enter details for the desired Testnet (e.g., Sepolia or RSK).
  - Example RPC URL: `https://sepolia.infura.io/v3/<your_project_id>`
  - Fund the Wallet:
    - Use a faucet to receive free test Ether (e.g., 0.5 ETH/day).
    - Check the balance in MetaMask.

Connect MetaMask with Remix IDE:

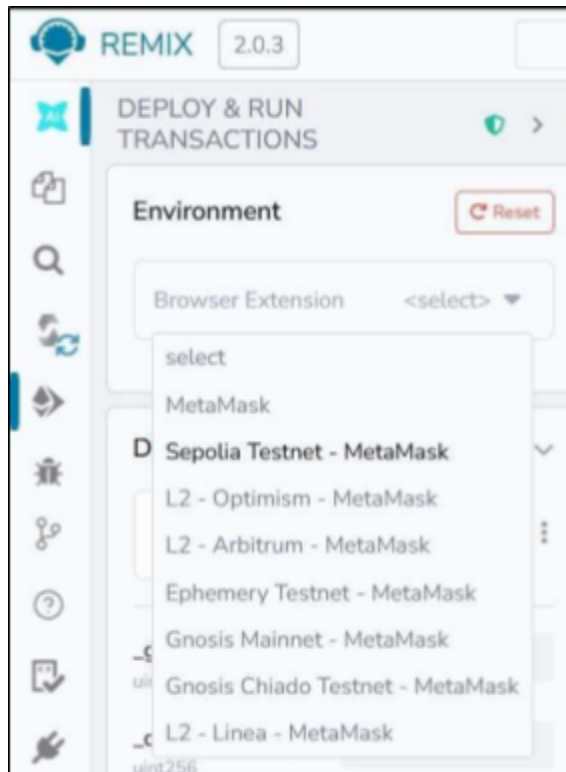
Open Remix IDE.

In the Deploy & Run Transactions tab, choose Injected Web3 as the environment.

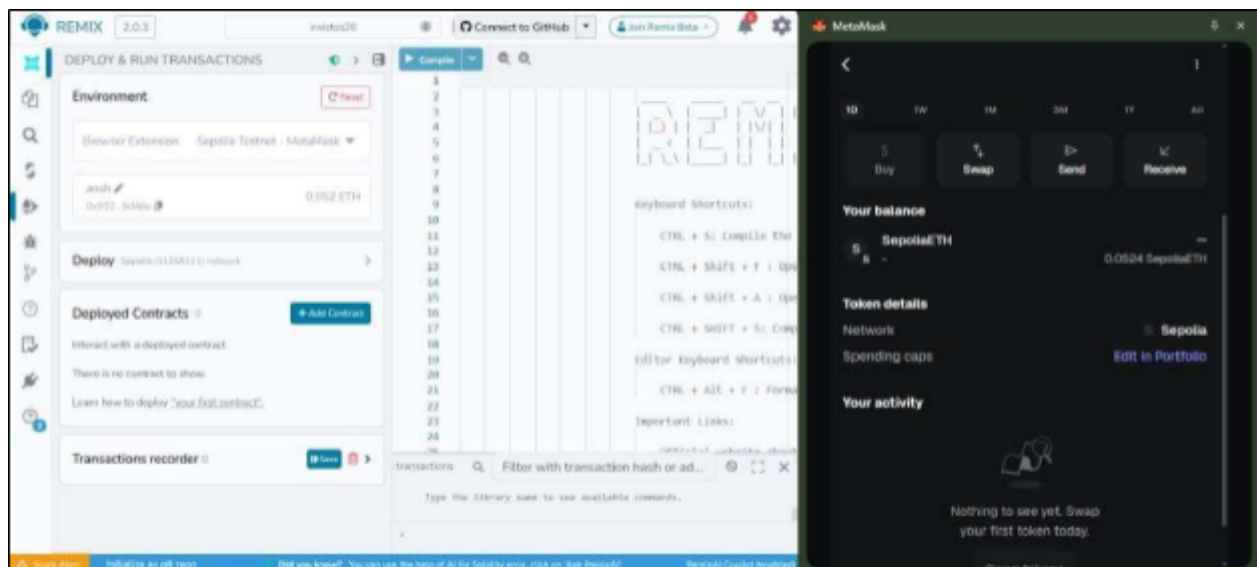
Approve connection in MetaMask.



Step 1: Select metamask sepolia testnet in environment



Initial setup:



## Step 2: Create a contract for a crowdfunding platform

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Crowdfunding {
 address public owner;
 uint public goal;
 uint public deadline;
 uint public totalFunds;
 mapping(address => uint) public contributions;

 constructor(uint _goal, uint _duration) {

 owner = msg.sender;
 goal = _goal;
 deadline = block.timestamp + _duration;
 }

 function contribute() public payable {
 require(block.timestamp < deadline, "Deadline passed");
 require(msg.value > 0, "Send some ETH");

 contributions[msg.sender] += msg.value;
 totalFunds += msg.value;
 }

 function withdrawFunds() public {
 require(msg.sender == owner, "Only owner");
 require(totalFunds >= goal, "Goal not reached");

 (bool success,) = payable(owner).call{value: totalFunds}("");
 require(success, "Transfer failed");
 }

 function refund() public {
 require(block.timestamp > deadline, "Not ended");
 require(totalFunds < goal, "Goal was reached");

 uint amount = contributions[msg.sender];
 require(amount > 0, "No contribution");
 }
}
```

```
contributions[msg.sender] = 0;
```

```
(bool success,) = payable(msg.sender).call{value: amount}("");
require(success, "Refund failed");
```

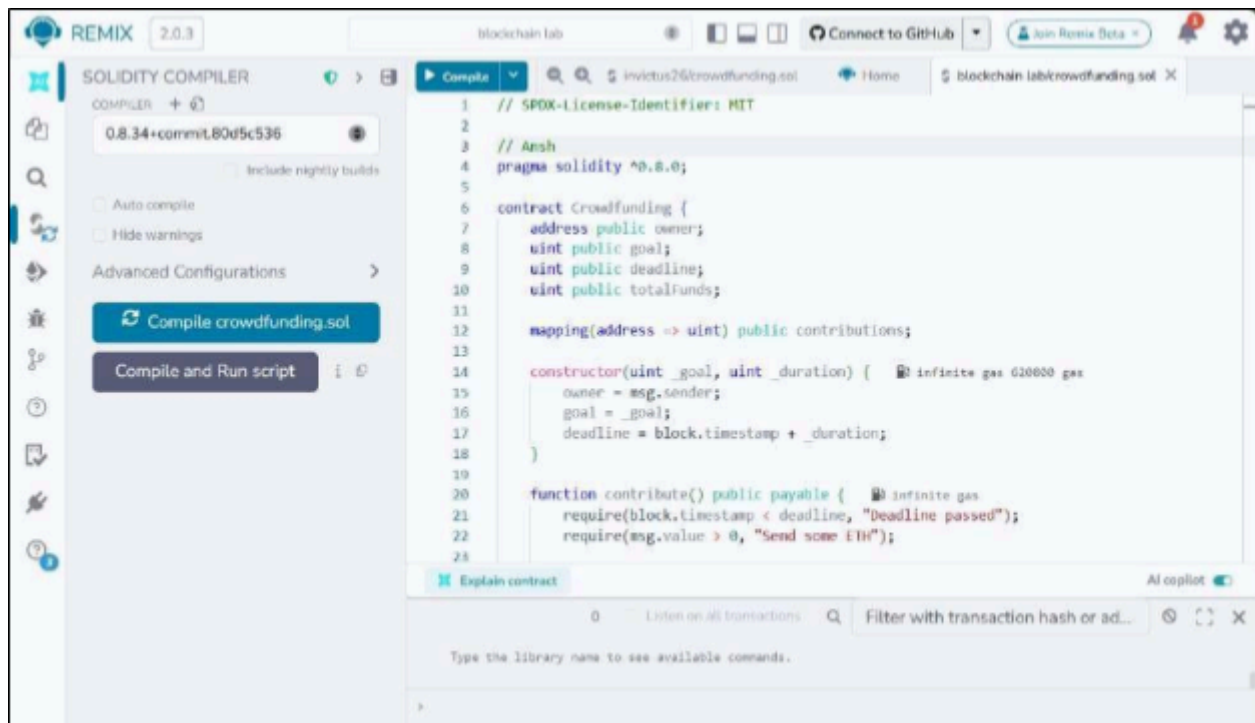
```
}
```

```
function getBalance() public view returns (uint) {
 return address(this).balance;
```

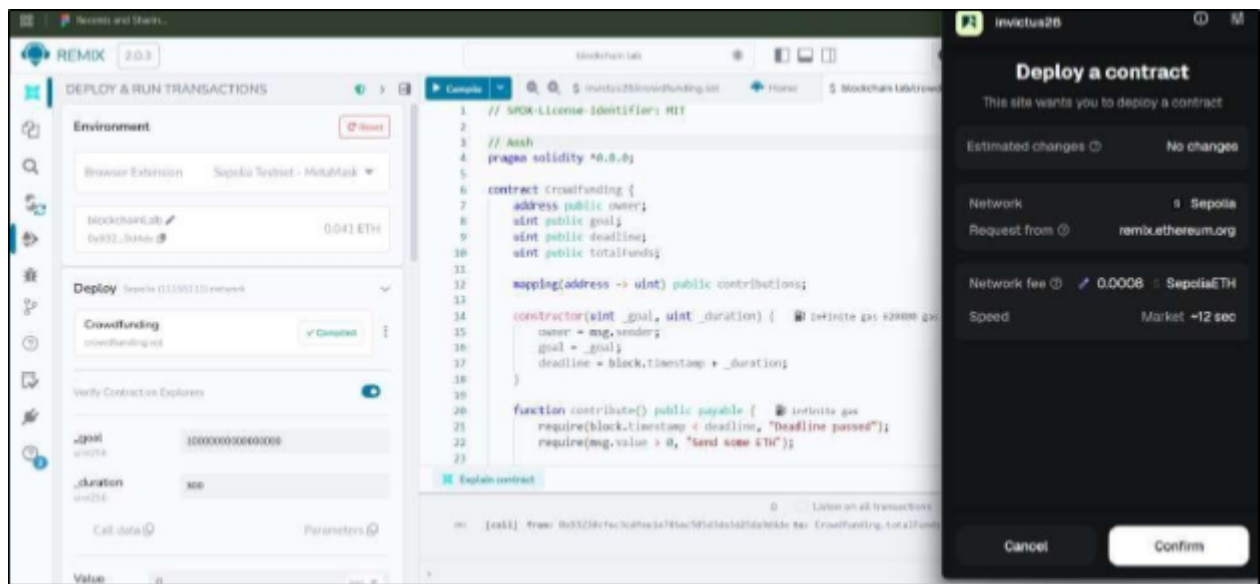
```
}
```

```
}
```

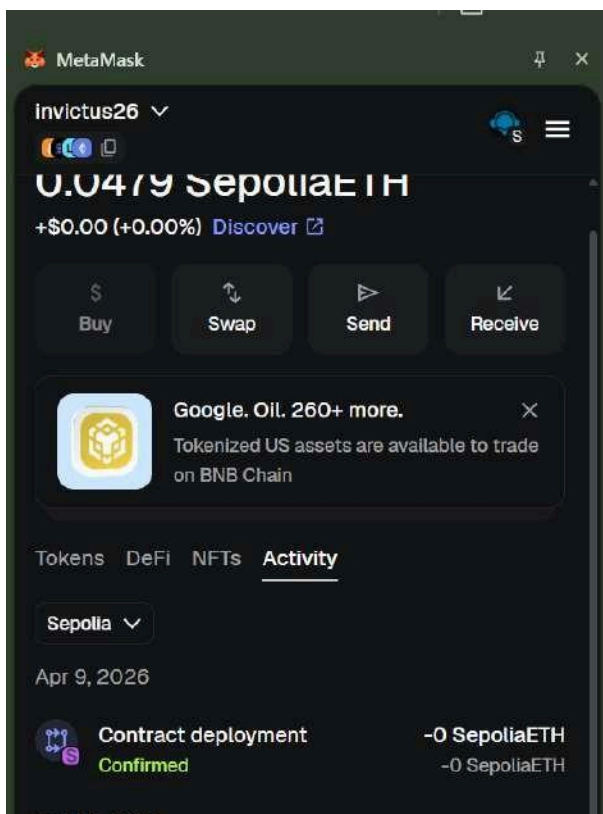
Step 3: save it as **crowdfunding.sol** and compile it



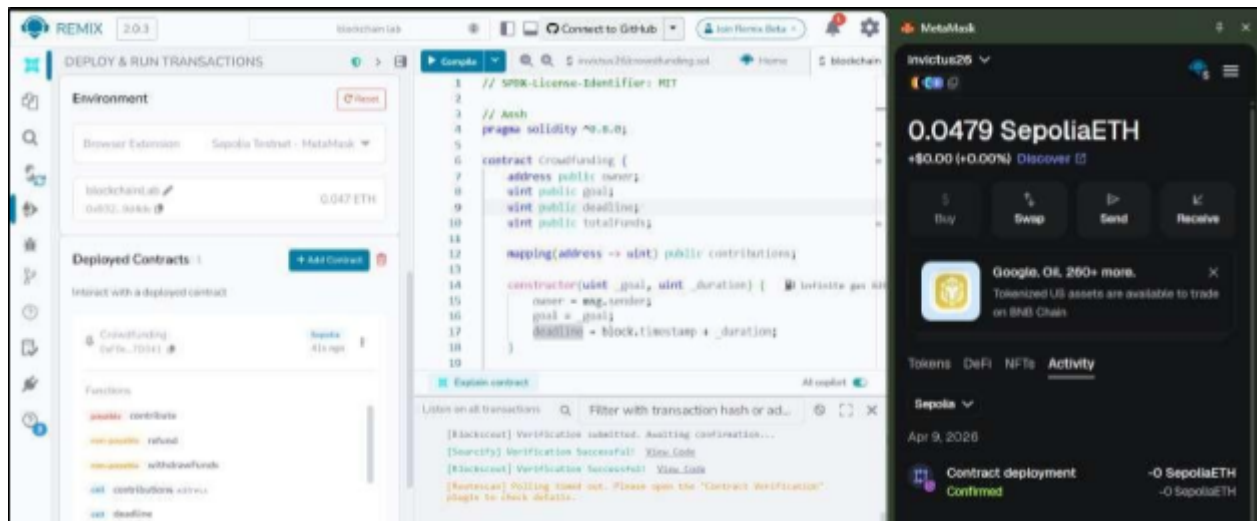
Step 4: deploy the contract with an initial goal and time limit. Confirm the transaction from metamask



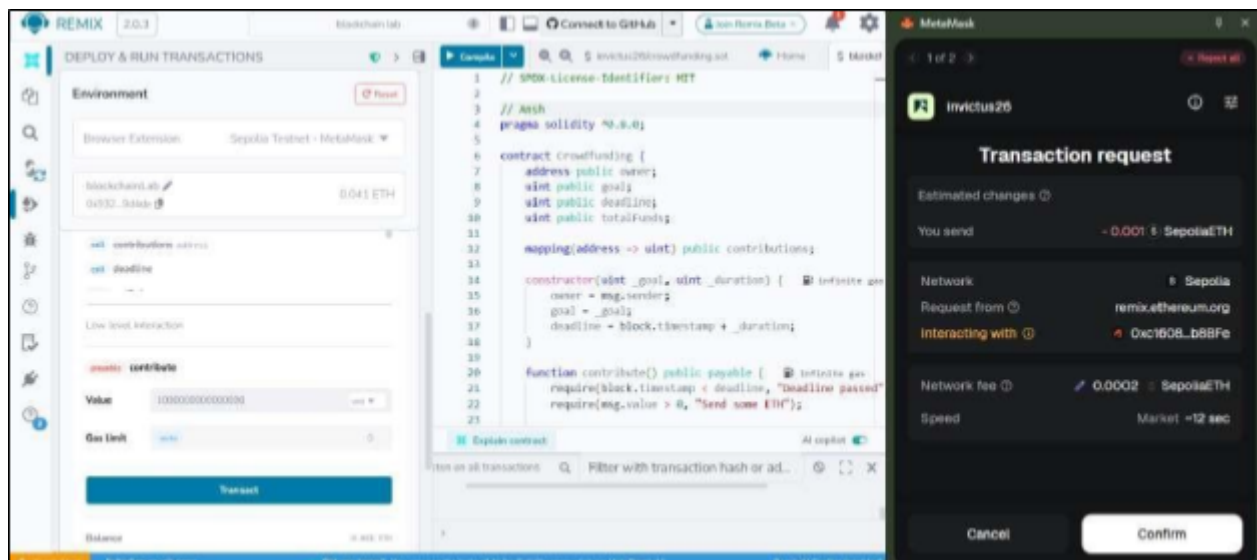
After deployment you can see: transaction recorded in metamask

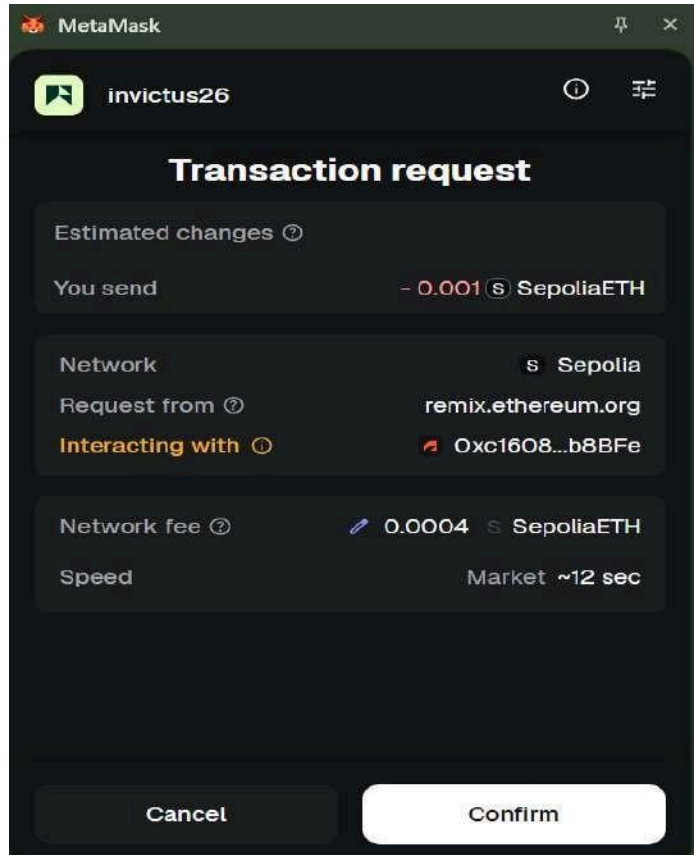


Step 5: After deployment in remix ide, we can see the deployed contract and its functions

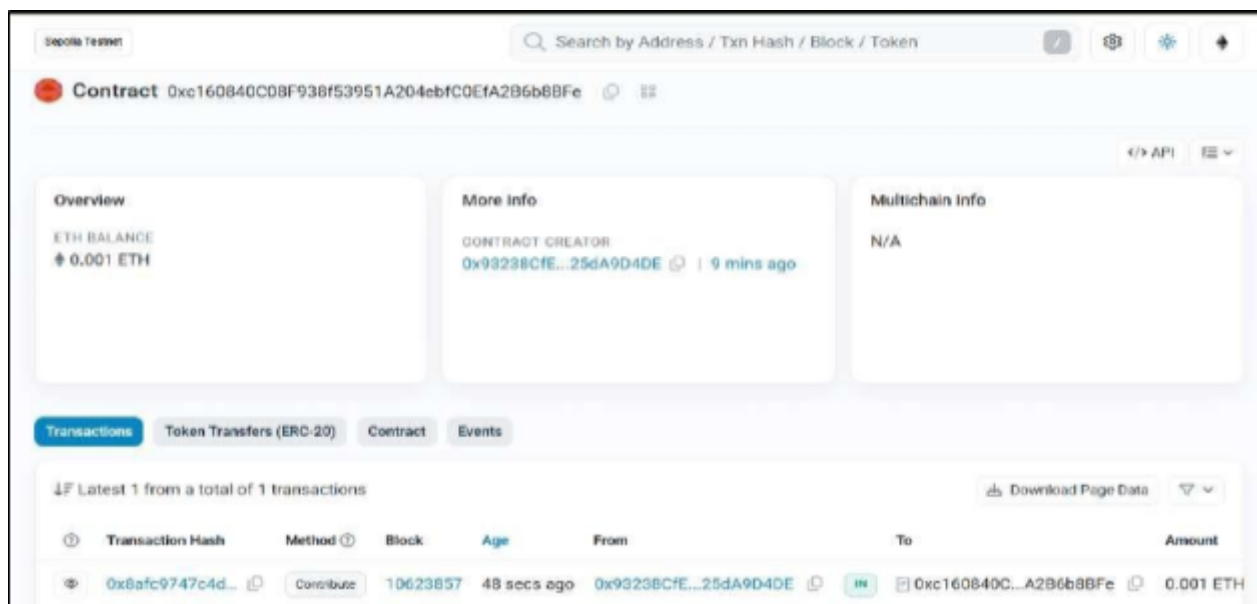


Step 6: Add a contribution of 0.01 ETH = 10000000000000000 wei. Transaction request appears in metamask. Click on confirm

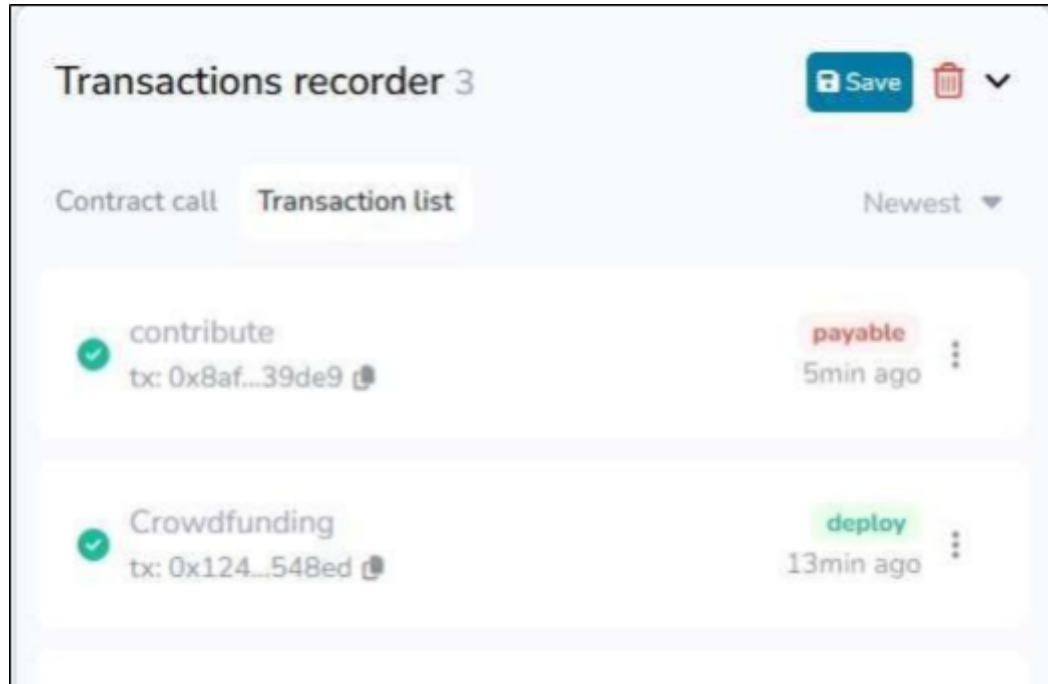




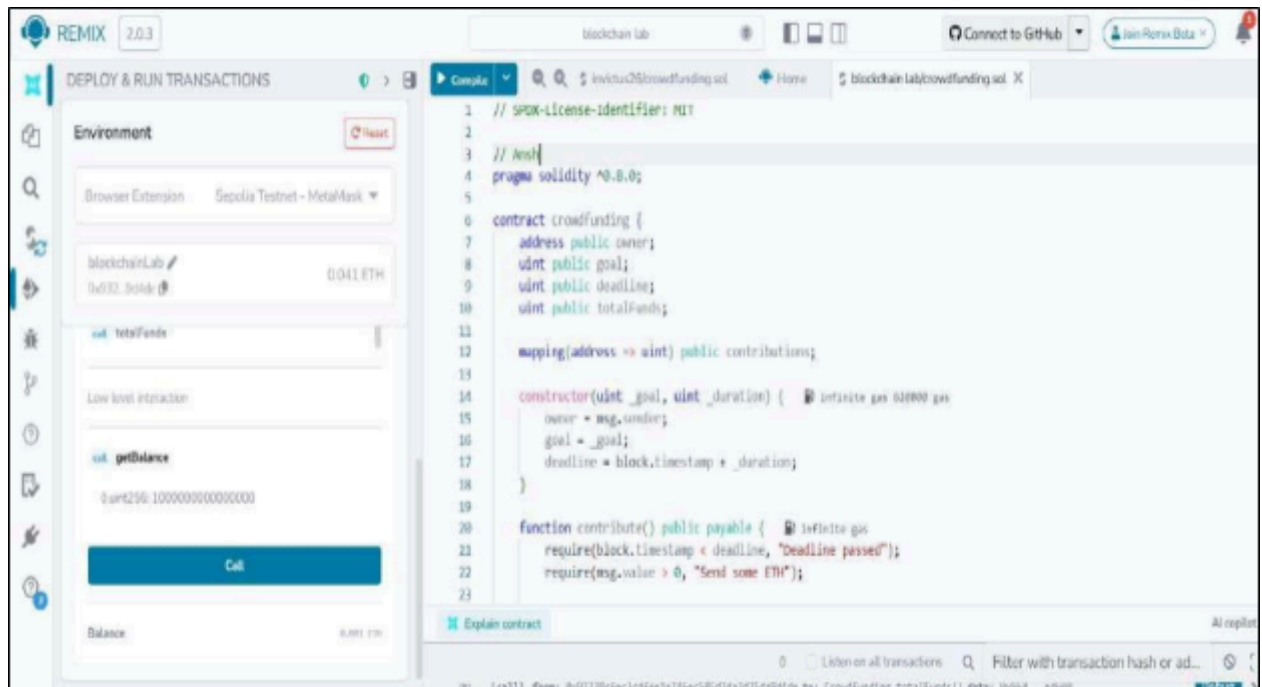
Step 7: copy paste the contracts address in <https://sepolia.etherscan.io/> to see all transactions:



Also can see the transactions under all transactions in remix IDE



Step 8: use getBalance function to check the total funds received



Output:

call

getBalance

0:uint256: 10000000000000000

Call

Balance

0.001 ETH

Step 9: Paste wallet address from metamask into contributions to see all transactions by the particular use

call

contributions

address

0x93238CfEc3cdfEE1E746eC505d3DA3D25dA9D4DE

0:uint256: 10000000000000000

Call

Step 10: use totalFunds function to see totalFunds recieved

call

totalFunds

0:uint256: 10000000000000000

Call



#### Conclusion:

Using MetaMask with Remix IDE provides a practical way to test and deploy smart contracts on test networks. It helps developers understand real blockchain interactions, transaction confirmations, and wallet integrations in a secure, no-risk environment.



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801 : Blockchain & DLT Lab**  
**Experiment 08**

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

|           |                                                                                                                                                                                                                                               |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                                                                                                                                                                            |
| Name      | Ansh Sarfare                                                                                                                                                                                                                                  |
| Class     | D20A                                                                                                                                                                                                                                          |
| Subject   | ITC801: Blockchain and DLT Lab                                                                                                                                                                                                                |
| CO Mapped | CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions<br>CO3:Implement smart contracts in Ethereum using different development frameworks.<br>CO5:Interpret different Crypto assets and Crypto currencies |

## **BC Experiment-8**

**Aim:** To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

### **Theory:**

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

#### Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

#### Steps to Connect Ganache with Metamask and Remix IDE:

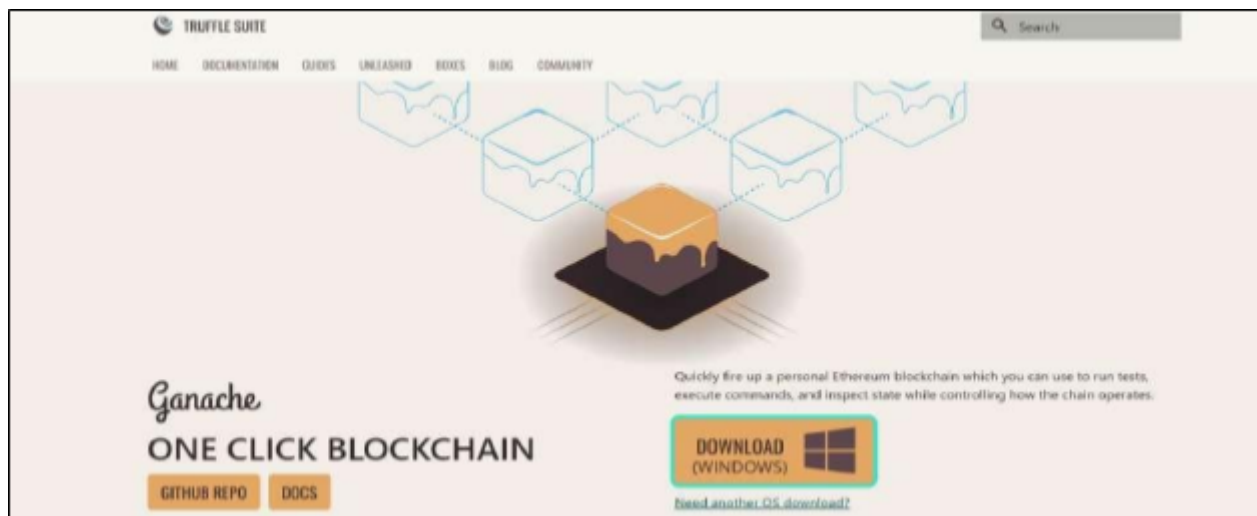
1. Install Ganache:
  - Download and install Ganache from the Truffle Suite website.
  - Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
  - Open Metamask and switch to a Custom RPC Network.
  - Enter the Ganache RPC server URL (e.g., [HTTP://127.0.0.1:7545](http://127.0.0.1:7545)).
  - Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
  - Open Remix IDE in the browser.
  - Choose Injected Web3 as the environment in the “Deploy & Run Transactions” tab.
  - Remix will automatically connect to the Metamask account (linked with Ganache).
4. Create a Simple Solidity Smart Contract:
  - Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).

```
pragma solidity ^0.8.0;
contract SimpleStorage {
 uint public data;
 function set(uint x) public {
 data = x;
 }
}
```

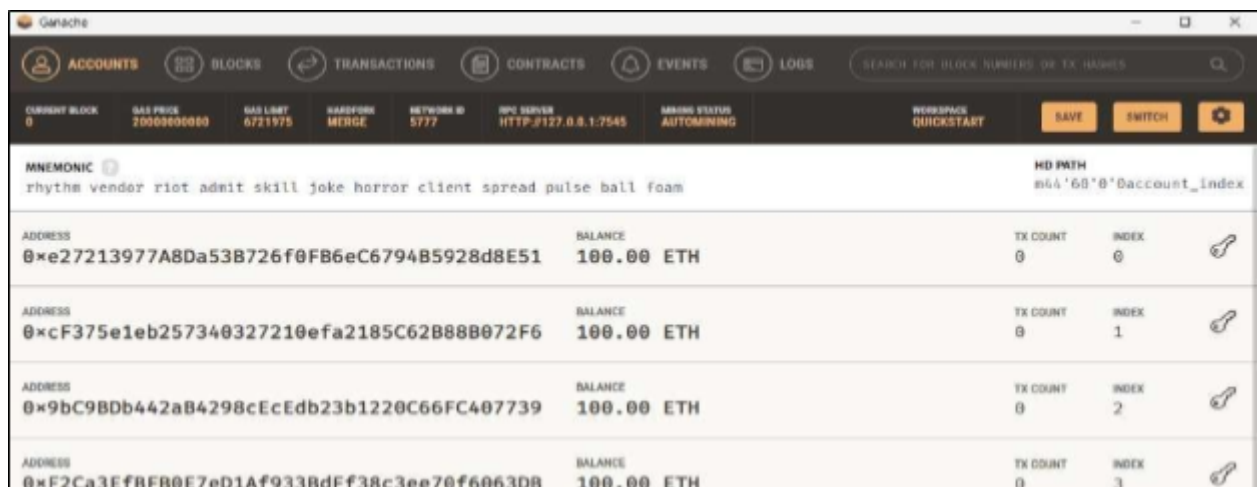
5. Compile and Deploy the Contract:  
Compile using Remix.
  - Deploy via the Metamask account connected to Ganache.
  - Approve the transaction in Metamask.
6. Check Transaction Details on Ganache:
  - View mined transactions, block details, and balances in the Ganache interface.
7. Interact with the Smart Contract:
  - Use Remix to call contract functions (`set()` or `get()`).
  - Observe the state changes and corresponding transactions on Ganache.

## Implementation:

Step 1: Download and install ganache



Once installed, open ganache and click on Quickstart Ethereum to start localhost.



Step 2: In MetaMask, click on the networks tab and add a custom network.

< **Ganache Local** X

Network name

Localhost 8545

Default RPC URL

ganache 1270.01:7545

Chain ID

1337

Currency symbol

ETH

Block explorer URL

Add a URL

Save

Step 3: Import ganache account into metamask

< **Add wallet**

**Private key**

Imported private keys are backed up to your account and sync automatically when you sign in with the same Google or Apple login.

[Learn more](#) about how imported keys work

Select type

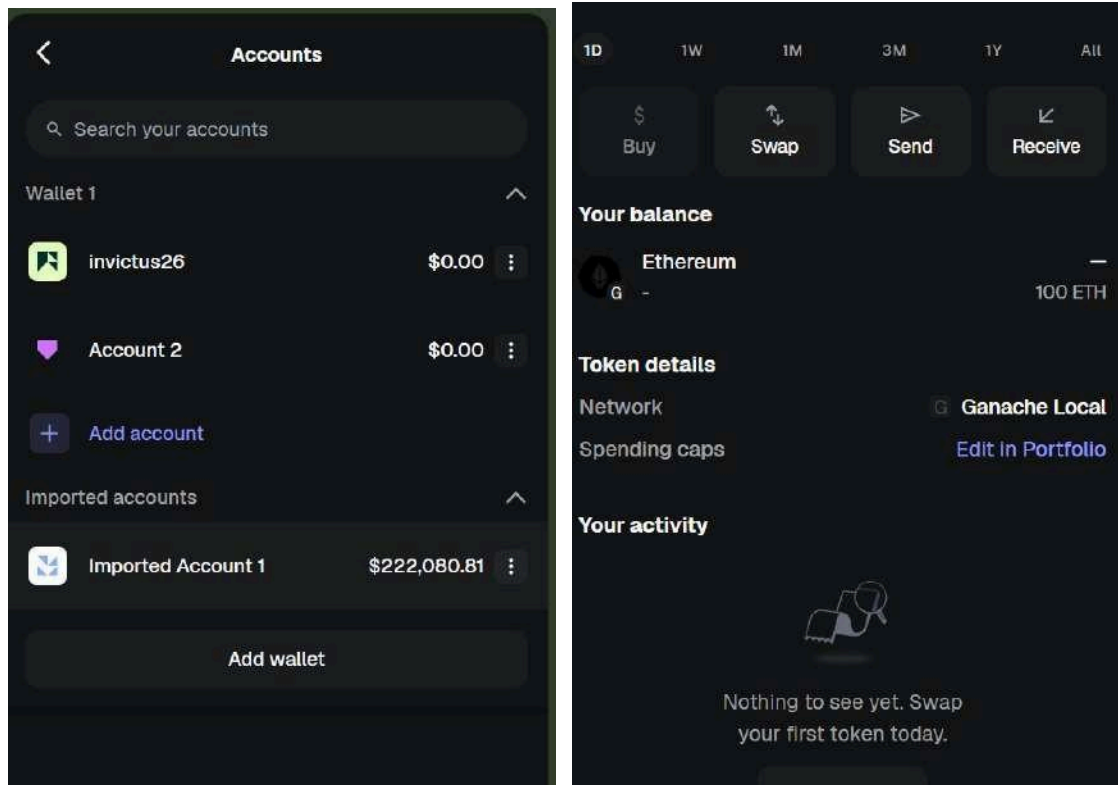
Private key

Enter your private key string here:

.....

Cancel Import

## Wallet balance appears in metaMask account



Step 4: In Remix IDE, create a new file **crowdfunding.sol**

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Crowdfunding {
 address public owner;
 uint public goal;
 uint public deadline;
 uint public totalFunds;
 mapping(address => uint) public contributions;

 constructor(uint _goal, uint _duration) {

 owner = msg.sender;
 goal = _goal;
 deadline = block.timestamp + _duration;
```

```

}

function contribute() public payable {
 require(block.timestamp < deadline, "Deadline passed");
 require(msg.value > 0, "Send some ETH");

 contributions[msg.sender] += msg.value;
 totalFunds += msg.value;
}

function withdrawFunds() public {
 require(msg.sender == owner, "Only owner");
 require(totalFunds >= goal, "Goal not reached");

 (bool success,) = payable(owner).call{value: totalFunds}("");
 require(success, "Transfer failed");
}

function refund() public {
 require(block.timestamp > deadline, "Not ended");
 require(totalFunds < goal, "Goal was reached");

 uint amount = contributions[msg.sender];
 require(amount > 0, "No contribution");

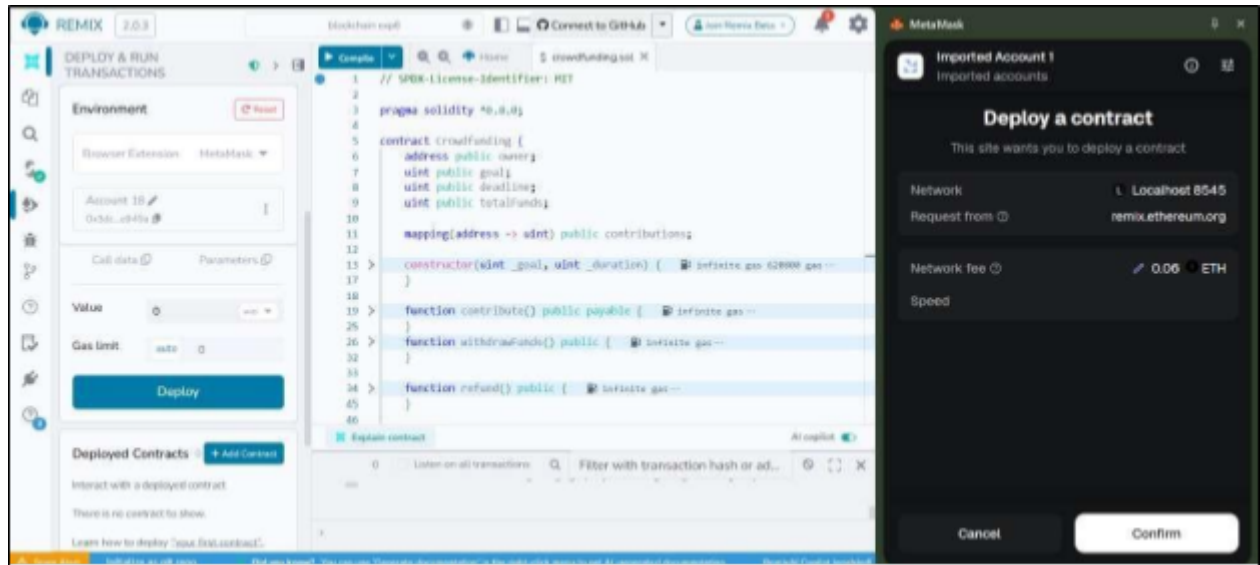
 contributions[msg.sender] = 0;

 (bool success,) = payable(msg.sender).call{value: amount}("");
 require(success, "Refund failed");
}

function getBalance() public view returns (uint) {
 return address(this).balance;
}
}

```

Step 5: Compile and run the contract.  
 When prompted in metamask, confirm the transaction.



The contract is deployed!

The deducted amount can be seen in ganache app along with transaction count

The image shows the Ganache application interface. At the top, there are tabs for ACCOUNTS, BLOCKS, TRANSACTIONS, CONTRACTS, EVENTS, and LOGS. Below these, a search bar is present. The main section displays a list of accounts with their addresses, balances, transaction counts, and indices. The mnemonic phrase 'apology essence feed north arm announce receive axis total wall chase involve' is visible. The HD path is 'm/44'/0'/0'/8account\_index'.

| ADDRESS                                    | BALANCE    | TX COUNT | INDEX |
|--------------------------------------------|------------|----------|-------|
| 0x678aB28b947030404778C6193423B73c7a014b18 | 99.92 ETH  | 2        | 0     |
| 0xcC163421b1091647e96D5Ce9ddEdF1e624dad0a6 | 100.00 ETH | 0        | 1     |
| 0x7Ea6E7307F61C1b29eA2FC7c40F16e602b514a31 | 100.00 ETH | 0        | 2     |
| 0xb00C1B0A9e7E90b2b538A98d7DD98A499578F4dd | 100.00 ETH | 0        | 3     |
| 0x176239f7D57783b2Fe396cb7FA3B73178C750713 | 100.00 ETH | 0        | 4     |
| 0x9BFCFd4b9223c4d5d551bb8152DBAFFAF449A36B | 100.00 ETH | 0        | 5     |



**Conclusion:**

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with **Metamask** and **Remix**, developers can deploy, test, and debug smart contracts efficiently before moving to a public network.



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801 : Blockchain & DLT Lab**  
**Experiment 09**

Aim: To implement a Private Ethereum Blockchain using Geth.

|           |                                                                                   |
|-----------|-----------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                |
| Name      | Ansh Sarfare                                                                      |
| Class     | D20A                                                                              |
| Subject   | ITC801: Blockchain and DLT Lab                                                    |
| CO Mapped | CO3:Implement smart contracts in Ethereum using different development frameworks. |

## **BC Experiment - 9**

**AIM:** Implement the Private Ethereum Blockchain using Geth.

### **THEORY:**

#### **Introduction to Ethereum**

Ethereum is an open-source, decentralized blockchain platform that goes beyond simple cryptocurrency transactions by supporting programmable smart contracts and decentralized applications (DApps). It was proposed by Vitalik Buterin in 2013 and launched in 2015. Unlike Bitcoin which is primarily a digital currency, Ethereum functions as a programmable distributed computing platform where developers can deploy self-executing code called smart contracts. The native cryptocurrency of the Ethereum network is called Ether (ETH), which is used to pay for computational operations on the network, referred to as gas fees.

Ethereum maintains a global state that is updated with every transaction and smart contract execution. Every node in the network holds a copy of this state, ensuring decentralization and fault tolerance. The network uses an account-based model where each account has a balance and can interact with other accounts or smart contracts.

#### **What is a Private Ethereum Network?**

A private Ethereum network is a standalone blockchain that replicates the Ethereum protocol but operates in complete isolation from the public mainnet. Organizations, developers, and academic institutions use private networks to test and develop blockchain applications without incurring real transaction costs or exposing data to the public chain. All network parameters such as consensus mechanism, block time, gas limits, and initial account balances are fully customizable. This makes private networks ideal for controlled experimentation, proof-of-concept development, and educational purposes.

#### **What is Geth (Go-Ethereum)?**

Geth, short for Go-Ethereum, is the most widely used official implementation of the Ethereum protocol, written in the Go programming language. It allows a computer to function as a full Ethereum node capable of mining Ether, validating transactions, executing smart contracts, and interacting with the blockchain. Geth can be operated through a command-line interface, a built-in JavaScript console, or via JSON-RPC API calls. It supports both public Ethereum networks as well as custom private networks, making it the preferred tool for setting up and managing private Ethereum deployments.

#### **Consensus Mechanism — Clique (Proof of Authority)**

Private Ethereum networks typically use the Clique consensus algorithm instead of Proof of Work. Clique is a Proof of Authority (PoA) protocol where only a set of pre-approved accounts called signers are authorized to produce and validate new blocks. Since block production does not involve solving computationally intensive puzzles, Clique is significantly faster and more energy efficient than traditional mining. The list of authorized signers is defined at the time of network creation and is embedded in the genesis block. This makes Clique well suited for private and consortium blockchain environments where participants are known and trusted.

## Steps Involved in Implementing a Private Ethereum Network

- 1. Installing Geth:** The first step involves downloading and installing the appropriate version of the Geth client on the system. Once installed, the binary is made accessible via the system PATH so it can be executed from any directory.
- 2. Creating Node Accounts:** Each participating node requires an Ethereum account which consists of a public address and a corresponding private key stored in a keystore file. These accounts are created using the `geth account new` command and are protected by a password. The public addresses of these accounts are used in the genesis configuration.
- 3. Setting Up Password Files:** To allow nodes to automatically unlock their accounts during startup without manual password entry each time, password files are created containing the account passwords in plain text.
- 4. Creating the Genesis Block:** The genesis block is configured through a `genesis.json` file. This file defines the chain ID, consensus settings (Clique period and epoch), gas limit, initial Ether balances for each account, and the `extradata` field which encodes the authorized signer address. Every node in the network must be initialized with the same genesis file.
- 5. Initializing the Nodes:** Each node's data directory is initialized using the `geth init` command along with the genesis file. This sets up the initial blockchain state and prepares the node to begin operating on the private network.
- 6. Running the Nodes:** Node 1 is started as the signer node with mining enabled, responsible for sealing and producing new blocks. Node 2 is started as a member node that participates in the network but does not produce blocks. Both nodes are started with the Geth JavaScript console enabled for direct interaction.
- 7. Connecting the Nodes:** Once both nodes are running, they are connected to each other using enode URLs. An enode is a unique identifier for a Geth node containing its public key and network address. Node 1's enode is retrieved using `admin.nodeInfo.enode` and then added to Node 2 using `admin.addPeer()`. Successful peering can be verified using `admin.peers`.
- 8. Sending Transactions:** After the network is established and nodes are connected, Ether can be transferred between accounts using `eth.sendTransaction()` in the Geth console. The

transaction is picked up by the signer node, included in a block, and confirmed on the private chain. Account balances can be verified using `eth.getBalance()`.

## PROCEDURE:

### Step 1: Install Geth

`wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz`

`tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz`

`mkdir -p ~/bin`

`cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113`

`export PATH="$HOME/bin:$PATH"`

`geth113 version`

```
g2022_ansh_sarfare@cloudshell:~$ wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
mkdir -p ~/bin
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113
export PATH="$HOME/bin:$PATH"
geth113 version
--2026-04-09 18:14:50-- https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)... 20.60.40.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)|20.60.40.164|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 28248613 (27M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz'

geth-linux-amd64-1.13.14-2bd6bd01.tar.gz 100%[=====]
2026-04-09 18:14:59 (3.30 MB/s) - 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz' saved [28248613/28248613]

geth-linux-amd64-1.13.14-2bd6bd01/
geth-linux-amd64-1.13.14-2bd6bd01/COPYING
geth-linux-amd64-1.13.14-2bd6bd01/geth
Geth
Version: 1.13.14-stable
Git Commit: 2bd6bd01d2e8561dd7fc21b631f4a34ac16627a1
Git Commit Date: 20240227
Architecture: amd64
Go Version: go1.21.6
Operating System: linux
GOPATH=/home/g2022_ansh_sarfare/gopath:google/gopath
```

### Step 2: Create folders and accounts for both nodes

```
g2022_ansh_sarfare@cloudshell:~$ mkdir -p ~/eth-private/node1 ~/eth-private/node2
g2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node1 account new
INFO [04-09T15:36:17.038] Maximum peer count ETH=50 TOR=10
INFO [04-09T15:36:17.041] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x93b0c547de2cf8228981d4a29ac3b0ca9a08859
Path of the secret key file: /home/2022_ansh_sarfare/eth-private/node1/keystore/UTC--2026-04-09T15-36-35.1667026412--93b0c547de2cf8228981d4a29ac3b0ca9a08859

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

g2022_ansh_sarfare@cloudshell:~$

g2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node2 account new
INFO [04-09T15:36:49.547] Maximum peer count ETH=50 TOR=10
INFO [04-09T15:36:49.548] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x6406586b899212f380a55bbf5e6596b055823D3b
Path of the secret key file: /home/2022_ansh_sarfare/eth-private/node2/keystore/UTC--2026-04-09T15-38-54.7743810742--6406586b899212f380a55bbf5e6596b055823D3b

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```

### Step 3: Create password files

```
g2022_ansh_sarfare@cloudshell:~$ echo "Ansh" > ~/eth-private/node1/password.txt
echo "Ansh" > ~/eth-private/node2/password.txt
g2022_ansh_sarfare@cloudshell:~$
```

### Step 4: Create genesis.json and verify it looks correct

NODE1="PASTE\_NODE1\_ADDRESS\_WITHOUT\_0x\_HERE"

NODE2="PASTE\_NODE2\_ADDRESS\_WITHOUT\_0x\_HERE"

```
cat > ~/eth-private/genesis.json << EOF
```

[illegible]

```

g2022_ansh_sarfare@cloudshell:~$ NODE1="9380C547De2cf8222981dd4a2ac33D0cA9a08859"
NODE2="6406656b8992122380A55Bb75e65966D9552203b"

cat > ~/eth-private/genesis.json << EOF
{
 "config": {
 "chainid": 1234,
 "homesteadBlock": 0,
 "eip150Block": 0,
 "eip155Block": 0,
 "eip158Block": 0,
 "byzantiumBlock": 0,
 "constantinopleBlock": 0,
 "petersburgBlock": 0,
 "clique": {
 "period": 5,
 "epoch": 30000
 }
 },
 "difficulty": "1",
 "gasLimit": "8000000",
 "extradata": "0x00(NODE1)00",
 "alloc": {
 "0x$(NODE1)": { "balance": "10000000000000000000" },
 "0x$(NODE2)": { "balance": "10000000000000000000" }
 }
}
Total=100000000
g2022_ansh_sarfare@cloudshell:~$

```

```

g2022_ansh_sarfare@cloudshell:~$ cat ~/eth-private/genesis.json
{
 "config": {
 "chainid": 1234,
 "homesteadBlock": 0,
 "eip150Block": 0,
 "eip155Block": 0,
 "eip158Block": 0,
 "byzantiumBlock": 0,
 "constantinopleBlock": 0,
 "petersburgBlock": 0,
 "clique": {
 "period": 5,
 "epoch": 30000
 }
 },
 "difficulty": "1",

```

## Step 5: Initialize both nodes

```

g2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node1 init ~/eth-private/genesis.json
INFO [04-09/15:49:20.956] Maximum peer count ETH=50 total=50
INFO [04-09/15:49:20.959] Smartcard socket not found, disabling err="Stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09/15:49:20.963] Set global gas cap cap=50,000,000
INFO [04-09/15:49:20.963] Initializing the R2G library backend=gokzg
INFO [04-09/15:49:21.035] Defaulting to pebble as the backing database
INFO [04-09/15:49:21.095] Allocated cache and file handles database=/home/g2022_ansh_sarfare/eth-private/node1/eth/chaindata cache=16.00MiB handles=16
INFO [04-09/15:49:21.104] Opened ancient database database=/home/g2022_ansh_sarfare/eth-private/node1/eth/chaindata/ancient/chain readonly=false
INFO [04-09/15:49:21.105] State schema set to default schema=hash
INFO [04-09/15:49:21.105] Writing custom genesis block
INFO [04-09/15:49:21.106] Persisted tile from memory database nodes=3 size=411.00B time=3.93038ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 liveness=0.0
DB
INFO [04-09/15:49:21.152] Successfully wrote genesis state database=chaindata hash=e03c67..887468
INFO [04-09/15:49:21.152] Defaulting to pebble as the backing database
INFO [04-09/15:49:21.152] Allocated cache and file handles database=/home/g2022_ansh_sarfare/eth-private/node1/eth/lightchaindata cache=16.00MiB handles=16
INFO [04-09/15:49:21.219] Opened ancient database database=/home/g2022_ansh_sarfare/eth-private/node1/eth/lightchaindata/ancient/chain readonly=false
INFO [04-09/15:49:21.219] State schema set to default schema=hash
INFO [04-09/15:49:21.219] Writing custom genesis block
INFO [04-09/15:49:21.223] Persisted tile from memory database nodes=3 size=411.00B time=4.12979ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 liveness=0.0
DB
INFO [04-09/15:49:21.237] Successfully wrote genesis state database=lightchaindata hash=e03c67..887468
g2022_ansh_sarfare@cloudshell:~$

```

```

g2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node2 init ~/eth-private/genesis.json
INFO [04-09/15:49:52.109] Maximum peer count ETH=50 total=10
INFO [04-09/15:49:52.111] Smartcard socket not found, disabling err="Stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09/15:49:52.115] Set global gas cap cap=50,000,000
INFO [04-09/15:49:52.118] Initializing the R2G library backend=gokzg
INFO [04-09/15:49:52.189] Defaulting to pebble as the backing database
INFO [04-09/15:49:52.189] Allocated cache and file handles database=/home/g2022_ansh_sarfare/eth-private/node2/eth/chaindata cache=16.00MiB handles=16
INFO [04-09/15:49:52.257] Opened ancient database database=/home/g2022_ansh_sarfare/eth-private/node2/eth/chaindata/ancient/chain readonly=false
INFO [04-09/15:49:52.257] State schema set to default schema=hash
INFO [04-09/15:49:52.257] Writing custom genesis block
INFO [04-09/15:49:52.262] Persisted tile from memory database nodes=3 size=411.00B time=4.507603ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 liveness=0.0
DB
INFO [04-09/15:49:52.306] Successfully wrote genesis state database=chaindata hash=e03c67..887468
INFO [04-09/15:49:52.306] Defaulting to pebble as the backing database

```



## Step 6: Run Node 1 (signer)

```
cloudshell x cloudshell x + v
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use `gcloud config set project [PROJECT_ID]`.
You can view your projects by running `gcloud projects list`.
2022_ansh_sarfare@cloudshell:~$ export PATH="$HOME/bin:$PATH"
NODE1="93B0C547De2cF8228981Dd4a29aC3D0cA9a08859"
2022_ansh_sarfare@cloudshell:~$

2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node1 \
--networkid 1234 \
--port 30303 \
--http --http.port 8545 \
--unlock 0x5(NODE1) \
--password ~/eth-private/node1/password.txt \
--mine \
--miner-etherbase 0x5(NODE1) \
--allow-insecure-unlock \
console
INFO [04-09|15:52:04.688] Maximum peer count ETH=50 total=50
INFO [04-09|15:52:04.691] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09|15:52:04.694] Set global gas cap cap=50,000,000
INFO [04-09|15:52:04.695] Initializing the KZG library backend=gokzg
INFO [04-09|15:52:04.767] Allocated trie memory caches clean=154.00MiB dirty=256.00MiB
INFO [04-09|15:52:04.767] Using pebble as the backing database database=/home/2022_ansh_sarfare/eth-private/node1/geth/chaindata
INFO [04-09|15:52:04.767] Allocated cache and file handles database=/home/2022_ansh_sarfare/eth-private/node1/geth/chaindata cache=512.00MiB handles=524,288
INFO [04-09|15:52:04.797] Opened ancient database database=/home/2022_ansh_sarfare/eth-private/node1/geth/chaindata/ancient/chain readonly=false
INFO [04-09|15:52:04.798] State scheme out to already existing scheme=hash
INFO [04-09|15:52:04.799] Initialising Ethereum protocol network=1234 dbversion=<nil>
INFO [04-09|15:52:04.803] -----
INFO [04-09|15:52:04.803] Chain ID: 1234 (unknown)
```

## Step 7: Open new terminal tab and run Node 2 (member)

```
g2022_ansh_sarfare@cloudshell:11:~$ export PATH="$HOME/bin:$PATH"
NODE2="6406586b898212f380A55Bbf5e6596bd95825D3b"

g2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node2 --networkid 1234 --port 30304 --http --http.port 8546 --unlock 0x5(NODE2) --password ~/eth-private/node2/password.txt --allow-insecure-unlock console
INFO [04-09|16:10:33.188] Maximum peer count ETH=50 total=50
INFO [04-09|16:10:33.188] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09|16:10:33.193] Set global gas cap cap=50,000,000
INFO [04-09|16:10:33.193] Initializing the KZG library backend=gokzg
INFO [04-09|16:10:33.268] Allocated trie memory caches clean=154.00MiB dirty=256.00MiB
INFO [04-09|16:10:33.268] Using pebble on the backing database database=/home/g2022_ansh_sarfare/eth-private/node2/geth/chaindata
INFO [04-09|16:10:33.268] Allocated cache and file handles database=/home/g2022_ansh_sarfare/eth-private/node2/geth/chaindata cache=512.00MiB handles=524,288
INFO [04-09|16:10:33.290] Opened ancient database database=/home/g2022_ansh_sarfare/eth-private/node2/geth/chaindata/ancient/chain readonly=false
INFO [04-09|16:10:33.300] State scheme set to already existing scheme=hash
INFO [04-09|16:10:33.301] Initialising Ethereum protocol network=1234 dbversion=8
INFO [04-09|16:10:33.302] -----
INFO [04-09|16:10:33.302] Chain ID: 1234 (unknown)
INFO [04-09|16:10:33.302] Consensus: Clique (proof-of-authority)
INFO [04-09|16:10:33.302] -----
INFO [04-09|16:10:33.302] Pre-Merge hard forks (block based):
INFO [04-09|16:10:33.302] - Homestead: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [04-09|16:10:33.302] - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
INFO [04-09|16:10:33.302] - Spurious Dragon/1 (EIP 158): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon-1.md)
INFO [04-09|16:10:33.302] - Spurious Dragon/2 (EIP 158): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon-2.md)
```

## Step 8: Connect the two nodes

In Node 1 console, get its enode

```
> admin.nodeInfo.enode
"enode://a2c63340e144ee5760abb42a72a912f3123198c228f201424f1058391186998f0fc73a77e454680d888b69ca388cf647de951ae695b321251e5f610bae6b87383a.126.99.110:30303"
> INFO [04-09|16:22:47.013] Successfully sealed new block
INFO [04-09|16:22:47.014] Commit new sealing work number=231 sealhash=bb4346..610566 hash=d1fc70..84fdc7 elapsed=5.001s
INFO [04-09|16:22:52.010] Successfully sealed new block number=232 sealhash=8aca46..08c3ca cap=0 gas=0 fees=0 elapsed=538.827ps
INFO [04-09|16:22:52.010] Commit new sealing work number=232 sealhash=8aca46..08c3ca hash=d65637..b7846e elapsed=4.996s
INFO [04-09|16:22:52.270] Looking for peers number=233 sealhash=c1df14..1d6155 cap=0 gas=0 fees=0 elapsed=485.675ps
INFO [04-09|16:22:52.270] peerCount=0 tried=60 peers=0
INFO [04-09|16:22:57.011] Successfully sealed new block number=233 sealhash=c1df14..1d6155 hash=0be857..1fcf8d elapsed=5.001s
INFO [04-09|16:22:57.012] Commit new sealing work number=234 sealhash=51ca87..5b8c95 cap=0 gas=0 fees=0 elapsed=501.891ps"
```



Now, switch to Node 2 console and paste the Node 1 enode

```
> admin.addPeer("enode://acfe33406144ee52760abb42a72a912f3123188c228f201424f106b391186998f0fc78a77e454680d8898b9ca388cf647de953ae699bb21251e5f610baa5bb7d834.126.99.110:30303")
true
> INFO [04-09|16:25:38.021] Looking for peers peercount=0 trie=45 static=1
```

In Node 2 console, check they are connected

```
> admin.peers
[{"caps": ["eth/60", "eth/60", "snap/1"],
 enode: "enode://00162513d2f3e1b57cc00de4a3d139342a3f8481147b7367aa7fe0421ce390f8a4ea0db1254cc3d0d8a0dfe00f457c3930617bf824a952c92c70931161abb847.187.149.99:31404",
 id: "26184e7240e8194376e99c1f03493aee8d3cb404898574950e6d147c9b8aeeb",
 name: "Nethermind/V1.36.0-31cbb01b7/linux-x86_64/dornc10.0.1",
 network: {
 inbound: false,
 localAddress: "18.88.0.4:55586",
 remoteAddress: "47.187.149.99:31404",
 static: false,
 trusted: false
 },
 protocols: {
 eth: "handshake",
 snap: "handshake"
 }
}]
```

## Step 9: Send a transaction

In Node 1 console, send transaction to Node 2

```
> eth.sendTransaction({
 from: eth.accounts[0],
 to: "0x5fd63344c1f40712A9A85d9d7e2B304a6C558716",
 value: web3.toWei(1, "ether")
 })
INFO [04-05|13:43:48.276] Setting new local account address=0x1D8b6d4f6C677D2fB66CA97fbD81b8D2c23c4D3
INFO [04-05|13:43:48.276] Submitted transaction hash=0x566ac1b0f280bd7b64a68991416a36248ed3ec43bd24f8a08af6badab79089 from=0x1D8b6d4f6C677D2fB66CA97fbD81b
ED2c23c4D3 nonce=0 recipient=0x5fd63344c1f40712A9A85d9d7e2B304a6C558716 value=1,000,000,000,000,000
0x566ac1b0f280bd7b64a68991416a36248ed3ec43bd24f8a08af6badab79089
```

and check the balance of Node 2

```
> eth.getBalance("0x5fd63344c1f40712A9A85d9d7e2B304a6C558716")
1010000000000000000
```

## CONCLUSION

This experiment successfully demonstrated the implementation of a private Ethereum blockchain network using the Geth (Go-Ethereum) client. A two-node private network was set up from scratch, beginning with the installation of Geth, creation of individual node accounts, and configuration of the genesis block using the Clique Proof of Authority consensus mechanism. Both nodes were initialized with the same genesis file, started with appropriate network parameters, and successfully connected to each other using enode URLs.

Once the network was operational, the signer node (Node 1) began producing and sealing blocks at regular intervals, confirming that the consensus mechanism was functioning correctly. The two nodes were peered together and their connection was verified using the `admin.peers` command. A transaction was then executed from Node 1 to Node 2 using `eth.sendTransaction()`, and the updated balance of Node 2 was confirmed using `eth.getBalance()`, validating that transactions are being processed and recorded on the private chain.

Through this experiment, a practical understanding of how Ethereum nodes operate, how private blockchain networks are configured, and how peer-to-peer communication works

between nodes was gained. It also highlighted the advantages of using Proof of Authority consensus in private networks, particularly in terms of faster block production and reduced computational overhead compared to Proof of Work. Overall, this experiment provided hands-on exposure to core blockchain concepts including genesis block configuration, node initialization, peer discovery, and on-chain transaction verification.



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801 : Blockchain & DLT Lab**  
**Experiment 10**

Aim: Explore the Cryptocurrency Landscape

|           |                                                                                                                                                                                                                                                                                                                                      |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                                                                                                                                                                                                                                                                   |
| Name      | Ansh Sarfare                                                                                                                                                                                                                                                                                                                         |
| Class     | D20A                                                                                                                                                                                                                                                                                                                                 |
| Subject   | ITC801: Blockchain and DLT Lab                                                                                                                                                                                                                                                                                                       |
| CO Mapped | CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology.<br>CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions<br>CO5:Interpret different Crypto assets and Crypto currencies<br>CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies. |

## **BC Experiment - 10**

**Aim:** Explore the Cryptocurrency Landscape

### **Theory:**

This experiment explores the **cryptocurrency landscape** by combining real-world market data analysis with blockchain technology research. The focus asset for this analysis is **Chainlink (LINK)**, and data was collected from **three reputable sources** as per the lab requirement:

1. **CoinGecko API** – Provided historical price data, market capitalization, and trading volumes in a structured JSON format, enabling direct processing in Python.
2. **CoinMarketCap** – Offered real-time and historical price charts, circulating supply, and market rank for visual cross-verification.
3. **Crypto.com** – Supplied quick-access market statistics, simplified price trends, and cryptocurrency news updates.

### **Cryptocurrency Landscape**

1. **Definition** – Cryptocurrencies are decentralized digital assets built on blockchain networks, enabling secure, transparent, and immutable transactions without central authority.
2. **Market Diversity** – Thousands of cryptocurrencies exist, each serving unique roles:
  - **Store of Value** – Bitcoin, Litecoin.
  - **Utility Tokens** – Chainlink, Ethereum.
  - **Stablecoins** – USDT, USDC.
  - **Governance Tokens** – UNI, AAVE.
3. **Volatility** – Prices are highly sensitive to market sentiment, technological developments, and regulatory updates.
4. **Data Analysis Importance** – Tracking historical trends, market capitalization, and trading volume is essential for understanding asset performance.
5. **Advancements** – Growing use of **DeFi platforms**, **NFT ecosystems**, **layer-2 scaling**, and **cross-chain protocols**.

### **Data Collection Process**

The program used the **CoinGecko API** to automate historical data fetching for Chainlink. The retrieved data was processed with **Pandas** to compute:

- **Daily Returns** – Percentage change between consecutive days to measure volatility.
- **Moving Averages (MA7, MA30)** – Short-term and long-term trend indicators.
- **Cumulative Returns** – Growth factor over time.
- **Rolling Volatility** – Short-term risk measurement based on daily return fluctuations.

### **Visualization & Dashboard**

Using **Matplotlib**, a six-panel dashboard was created to display:

1. **Price Trend** – Daily price movement over the selected period.
2. **Price vs Volume** – Correlation between trading activity and price changes.
3. **Daily Returns Distribution** – Statistical spread of daily gains/losses.

4. **Moving Averages** – Technical analysis trend lines for price movement.
5. **Cumulative Returns** – Overall growth performance.
6. **Rolling Volatility** – Market risk trends over time.

## Technological Analysis

Beyond market data, the study covered **advancements in blockchain technology**, including:

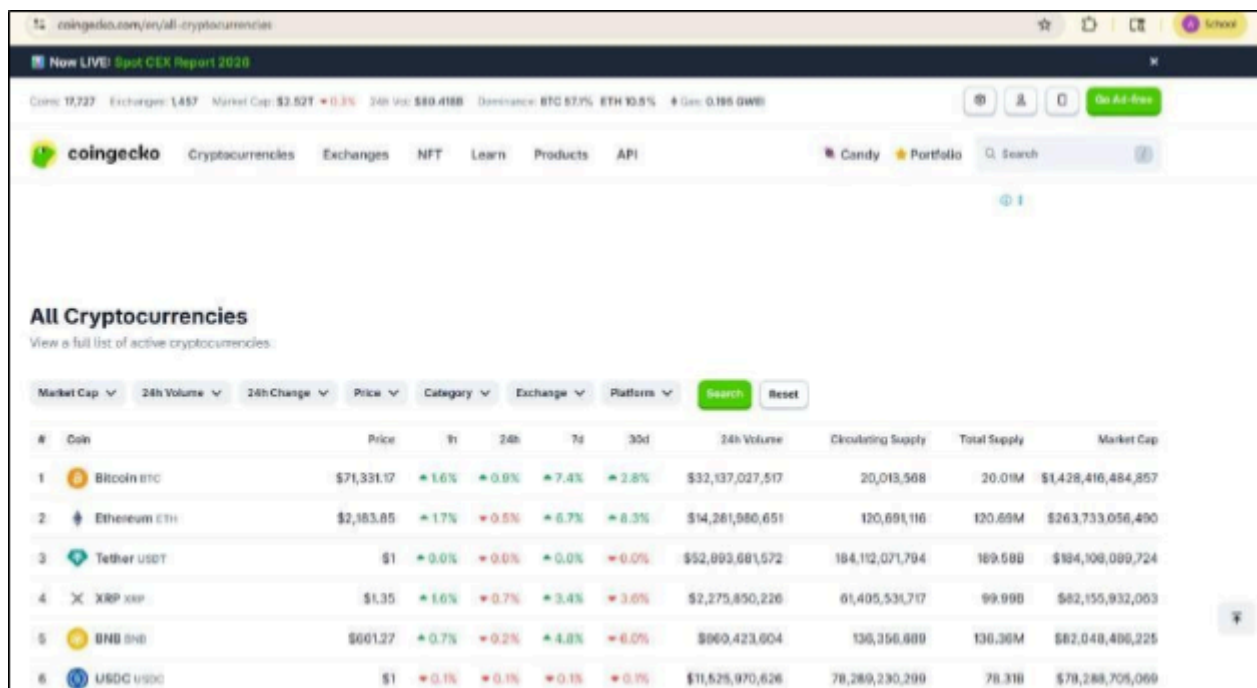
- **Consensus Mechanisms** – PoW (Proof of Work), PoS (Proof of Stake), and hybrid systems.
- **Scalability Solutions** – Layer-2 protocols, sharding, and sidechains to increase throughput.
- **Interoperability Protocols** – Chainlink’s CCIP, Polkadot, and Cosmos enabling cross-chain data exchange.
- **Privacy Features** – Zero-knowledge proofs (zk-SNARKs) and ring signatures to ensure transaction confidentiality.

Chainlink’s **roadmap** highlights staking implementation, advanced oracle features, and enhanced cross-chain capabilities, aligning with industry trends toward **decentralization, interoperability, and security**.

## Market Trends & Adoption

- **Growing Merchant Acceptance** – More businesses accepting cryptocurrency payments.
- **Wallet Growth** – Increasing downloads and installations of crypto wallets.
- **Network Activity** – Higher transaction volumes and active addresses, signaling strong adoption.
- **Integration with Traditional Finance** – Partnerships between crypto platforms and fintech/payment providers.
- **Regulatory Developments** – Countries exploring frameworks for safe cryptocurrency adoption.

## Implementation



coingecko.com/en/all-cryptocurrencies

Now LIVE! Spot CEX Report 2020

Coin: 19,727 Exchanges: 1,457 Market Cap: \$2.52T +0.3% 24h Vol: \$80.418B Dominance: BTC 57.9% ETH 13.8% + Dev: 0.185 GWB

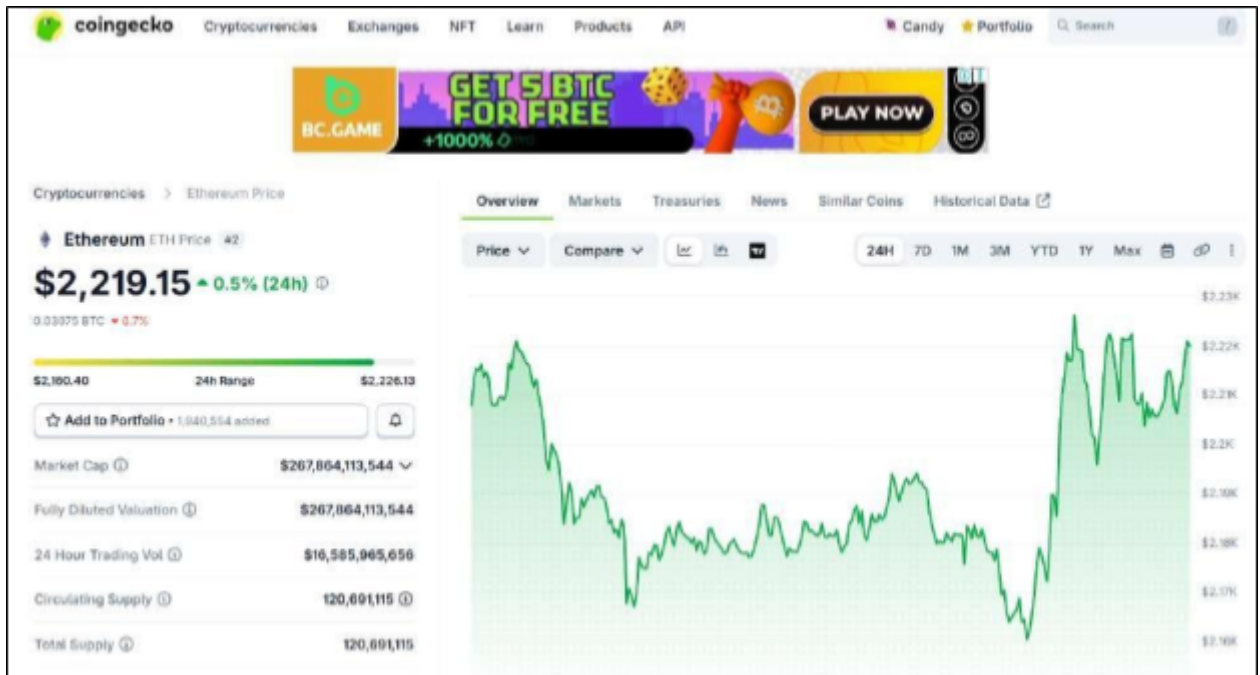
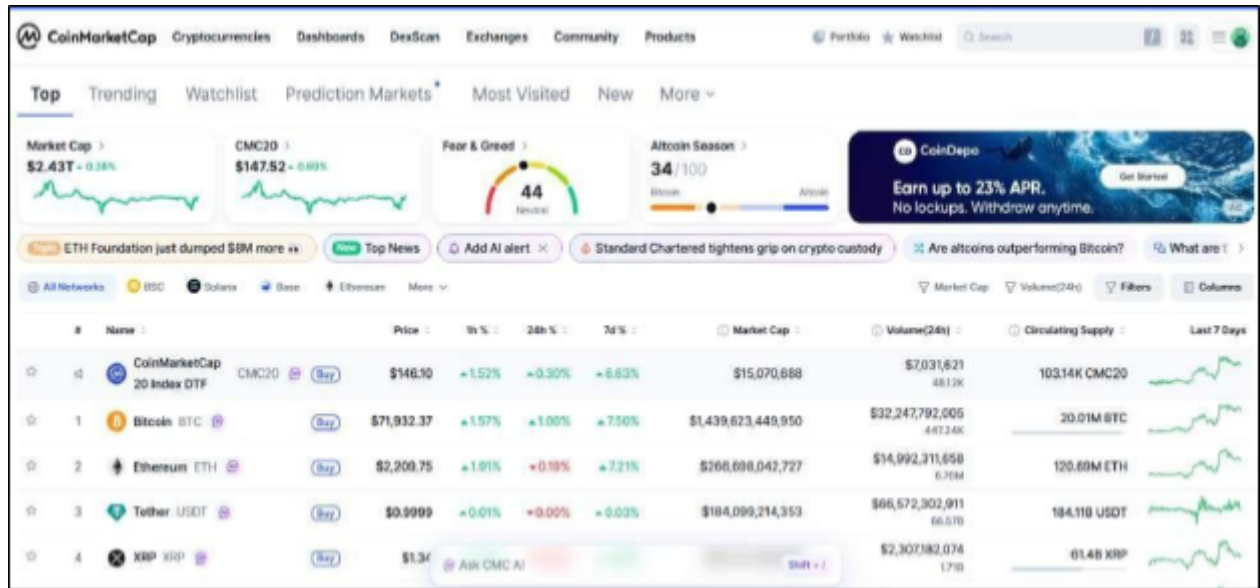
coingecko Cryptocurrencies Exchanges NFT Learn Products API Candy Portfolio Search

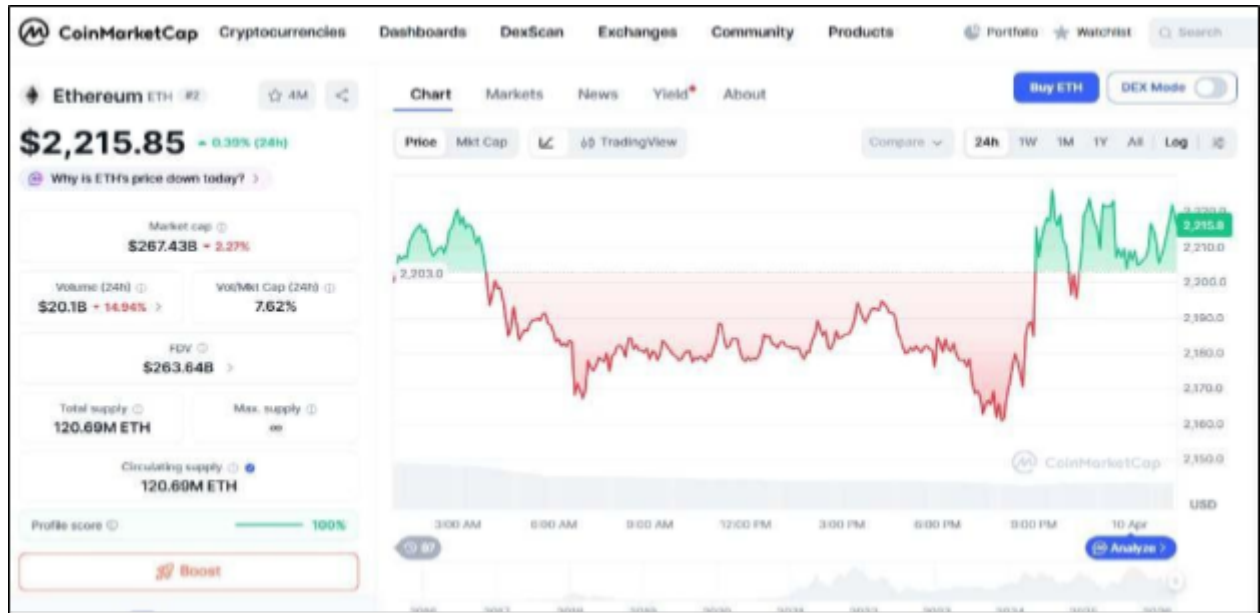
### All Cryptocurrencies

View a full list of active cryptocurrencies.

Market Cap 24h Volume 24h Change Price Category Exchange Platforms Search Reset

| # | Coin                                                                                              | Price       | 1h    | 24h   | 7d    | 30d   | 24h Volume       | Circulating Supply | Total Supply | Market Cap          |
|---|---------------------------------------------------------------------------------------------------|-------------|-------|-------|-------|-------|------------------|--------------------|--------------|---------------------|
| 1 |  Bitcoin BTC   | \$71,331.17 | +1.6% | +0.9% | +7.4% | +2.8% | \$32,137,027,517 | 20,013,568         | 20.01M       | \$1,428,416,484,857 |
| 2 |  Ethereum ETH  | \$2,183.85  | +1.7% | +0.5% | +6.7% | +8.3% | \$14,281,986,651 | 120,691,116        | 120.69M      | \$263,733,056,490   |
| 3 |  Tether USDT   | \$1         | +0.0% | +0.0% | +0.0% | +0.0% | \$52,893,681,572 | 184,112,071,794    | 189.58B      | \$184,108,089,724   |
| 4 |  XRP XRP       | \$1.35      | +1.6% | +0.7% | +3.4% | +3.6% | \$2,275,850,226  | 61,405,531,717     | 99.99B       | \$82,155,932,063    |
| 5 |  BNB BNB       | \$601.27    | +0.7% | +0.2% | +4.8% | +6.0% | \$860,423,604    | 136,356,889        | 136.36M      | \$82,048,486,225    |
| 6 |  USD Coin USDC | \$1         | +0.1% | +0.1% | +0.1% | +0.1% | \$11,625,970,626 | 78,289,230,289     | 78.31B       | \$78,288,705,066    |





## Utilizing Api for analysis and visualization :

### Code:

```
import requests
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
----- Fetch Data -----
```

```
def get_historical_data(coin_id, days='180', currency='usd'):
```

```
 url = f'https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart'
```

```
 params = {'vs_currency': currency, 'days': days}
```

```
 response = requests.get(url, params=params)
```

```
 return response.json()
```

```
----- Dashboard Function
```

```
-----def
```

```
ethereum_dashboard(days='180'):
```

```
 coin_id = 'ethereum'
```

```
 data = get_historical_data(coin_id, days)
```

```
Create DataFrame
```

```

prices_df = pd.DataFrame(data['prices'], columns=['timestamp', 'price'])
volumes_df = pd.DataFrame(data['total_volumes'], columns=['timestamp', 'volume'])

prices_df['timestamp'] = pd.to_datetime(prices_df['timestamp'], unit='ms')
volumes_df['timestamp'] = pd.to_datetime(volumes_df['timestamp'], unit='ms')

df = pd.merge(prices_df, volumes_df, on='timestamp')
df.set_index('timestamp', inplace=True)

Calculate extra metrics
df['returns'] = df['price'].pct_change()
df['MA7'] = df['price'].rolling(window=7).mean()
df['MA30'] = df['price'].rolling(window=30).mean()
df['cumulative'] = (1 + df['returns']).cumprod()
df['volatility'] = df['returns'].rolling(window=7).std() * 100

----- Plot Dashboard -----
fig, axes = plt.subplots(3, 2, figsize=(16, 12))
fig.suptitle(f'{coin_id.capitalize()} Dashboard (Last {days} Days)', fontsize=16, fontweight='bold')

1. Price Trend
axes[0, 0].plot(df.index, df['price'], color='blue')
axes[0, 0].set_title('Price Trend')
axes[0, 0].set_ylabel('Price (USD)')
axes[0, 0].set_xlabel('Year-Month') # Add xlabel
axes[0, 0].grid(True)

2. Price vs Volume
axes[0, 1].plot(df.index, df['price'], color='blue', label='Price')
axes[0, 1].bar(df.index, df['volume'], color='orange', alpha=0.3, label='Volume')

```



```
axes[0, 1].set_title('Price vs Volume')
axes[0, 1].set_xlabel('Year-Month') # Add xlabel
axes[0, 1].set_ylabel('Price (USD) / Volume') # Add ylabel
axes[0, 1].legend()
axes[0, 1].grid(True)
```

### # 3. Daily Returns Distribution

```
axes[1, 0].hist(df['returns'].dropna(), bins=50, alpha=0.7, color='purple')
axes[1, 0].set_title('Daily Returns Distribution')
axes[1, 0].set_xlabel('Daily Return')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].grid(True)
```

### # 4. Moving Averages

```
axes[1, 1].plot(df.index, df['price'], color='lightgray', label='Price')
axes[1, 1].plot(df.index, df['MA7'], color='blue', label='7-day MA')
axes[1, 1].plot(df.index, df['MA30'], color='red', label='30-day MA')
axes[1, 1].set_title('Moving Averages')
axes[1, 1].set_xlabel('Year-Month')
axes[1, 1].set_ylabel('Price (USD)')
axes[1, 1].legend()
axes[1, 1].grid(True)
```

### # 5. Cumulative Returns

```
axes[2, 0].plot(df.index, df['cumulative'], color='green')
axes[2, 0].set_title('Cumulative Returns')
axes[2, 0].set_ylabel('Growth Factor')
axes[2, 0].set_xlabel('Year-Month')
axes[2, 0].grid(True)
```

### # 6. Volatility

```

axes[2, 1].plot(df.index, df['volatility'], color='orange')

axes[2, 1].set_title('Rolling Volatility (7-Day)')

axes[2, 1].set_ylabel('Volatility (%)')

axes[2, 1].set_xlabel('Year-Month')

axes[2, 1].grid(True)

plt.tight_layout(rect=[0, 0, 1, 0.96])

plt.subplots_adjust(hspace=0.5, wspace=0.3) # Adjust vertical and horizontal spacing

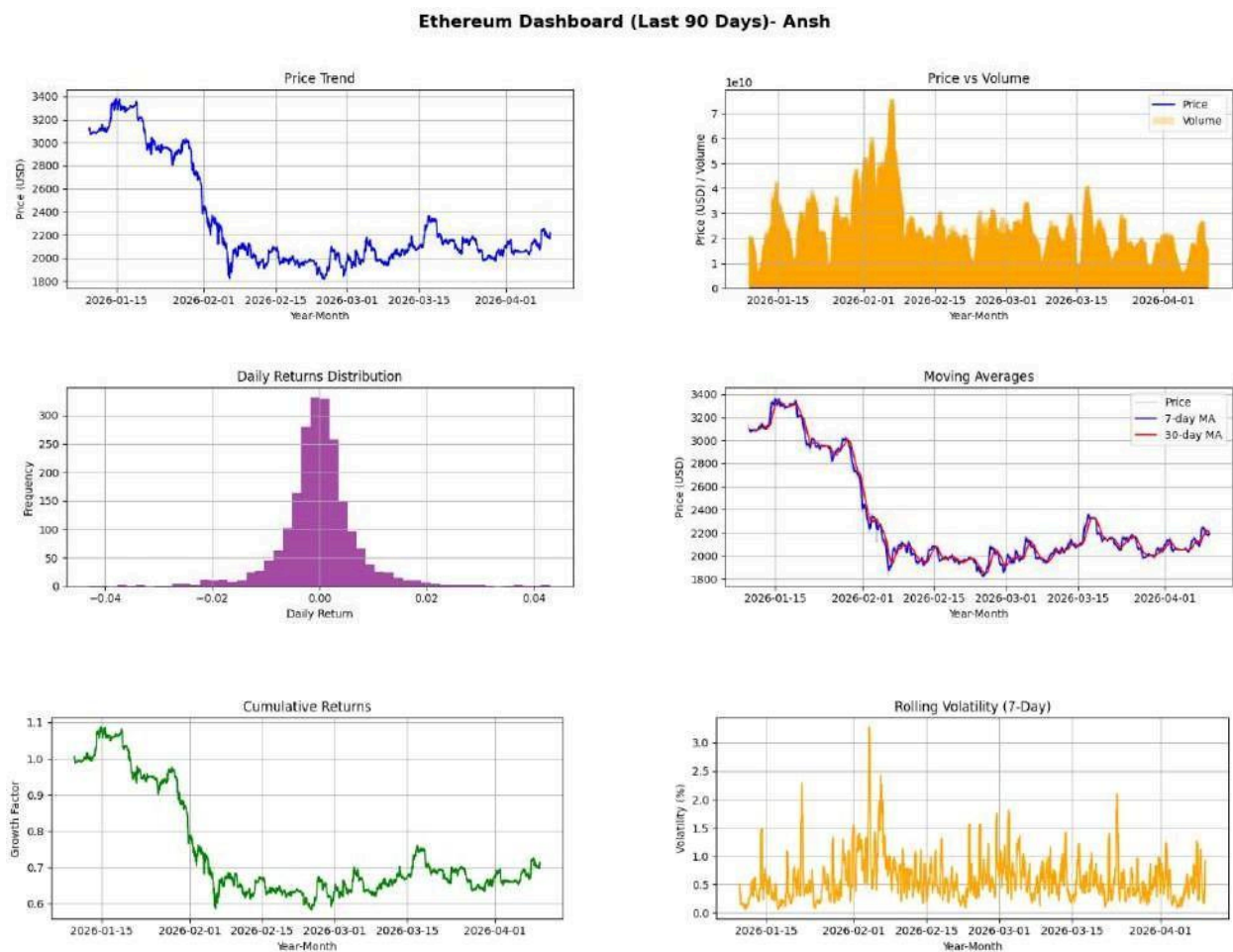
plt.show()

----- Run Dashboard

-----ethereum_dashboard('90')

```

**Output:**



**Conclusion:**

By combining automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, this experiment presents a holistic view of the cryptocurrency ecosystem. It demonstrates how blockchain's technical foundations connect with real-world market behavior, providing insights into both current trends and future advancements in the digital asset industry.



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801 : Blockchain & DLT Lab**  
**Experiment 11**

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

|           |                                                                      |
|-----------|----------------------------------------------------------------------|
| Roll No.  | 51                                                                   |
| Name      | Ansh Sarfare                                                         |
| Class     | D20A                                                                 |
| Subject   | ITC801: Blockchain and DLT Lab                                       |
| CO Mapped | CO4:Develop applications in permissioned Hyperledger Fabric network. |

## BC Experiment-11

### Title:

Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

### Aim:

To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

### Prerequisites:

- Windows Subsystem for Linux (WSL)
- Ubuntu
- Docker
- Git
- Node.js & npm

### Procedure:

#### Step 1: Update the System

##### Command:

```
sudo apt update
sudo apt upgrade -y
```

```
anshsarfare16@cloudshell:~$ sudo apt update
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease
Get:2 https://cli.github.com/packages stable InRelease [3,917 B]
Hit:3 https://apt.postgresql.org/pub/repos/apt noble-pgdg InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble InRelease
Get:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2,377 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2,155 kB]
Hit:9 https://packages.cloud.google.com/apt gcsfuse-noble InRelease
Hit:10 https://packages.cloud.google.com/apt cloud-sdk InRelease
Hit:11 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 InRelease
Get:12 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:13 https://ppa.launchpadcontent.net/dotnet/backports/ubuntu noble InRelease
Fetched 4,914 kB in 6s (778 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
12 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

**Explanation:**

This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

**Step 2: Install Required Dependencies****Command:**

```
sudo apt install jq curl git nodejs npm -y
node -v
npm -v
git --version
docker
--version
docker ps
```

```
anshsarfare16@cloudshell:~$ node -v
v18.19.1
anshsarfare16@cloudshell:~$ npm -v
9.2.0
anshsarfare16@cloudshell:~$ git --version
git version 2.43.0
anshsarfare16@cloudshell:~$ docker --version
Docker version 29.4.0, build 9d7ad9f
anshsarfare16@cloudshell:~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

**Explanation:**

This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while `docker ps` ensures Docker is running properly.

**Step 3: Clone Fabric Samples****Command:**

```
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
```

```

anshsarfare16@cloudshell:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (238/238), done.
remote: Compressing objects: 100% (129/129), done.
remote: Total 15618 (delta 161), reused 109 (delta 109), pack-reused 15380 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 17.07 MiB/s, done.
Resolving deltas: 100% (8809/8809), done.
anshsarfare16@cloudshell:~$ cd fabric-samples

```

## Explanation:

This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

## Step 4: Install Fabric Binaries and Images

### Command:

```

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

```

```

anshsarfare16@cloudshell:~/fabric-samples$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric

Pull Hyperledger Fabric binaries

==> Downloading version 2.5.15 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
==> Will unpack to: /home/anshsarfare16/fabric-samples
% Total % Received % Xferd Average Speed Time Time Time Current
 Dload Upload Total Spent Left Speed
 0 0 0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- 0
100 120M 100 120M 0 0 29.5M 0 0:00:04 0:00:04 --:--:-- 33.3M
==> Done.

==> Downloading version 1.5.17 platform specific fabric-ca-client binary
==> Downloading: https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
==> Will unpack to: /home/anshsarfare16/fabric-samples
% Total % Received % Xferd Average Speed Time Time Time Current
 Dload Upload Total Spent Left Speed
 0 0 0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- 0
100 31.6M 100 31.6M 0 0 16.2M 0 0:00:01 0:00:01 --:--:-- 36.4M
==> Done.

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
==> Pulling fabric Images
==> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
d41d93850539: Pull complete
b1cba2e842ca: Pull complete
10a7c8361220: Pull complete
f8c55d757df2: Pull complete
eb2bee6aa5bd: Pull complete

```



```

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
====> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====> ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
====> ghcr.io/hyperledger/fabric-baseos:2.5.15
2.5.15: Pulling from hyperledger/fabric-baseos
Digest: sha256:654bd4554bf97c70902e56d530294795372518da7ae2a7cdfccbe397a878c513
Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
====> Pulling fabric ca Image
====> ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
====> List out hyperledger images

```

### Explanation:

This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

## Step 5: Verify Docker Images

### Command:

docker images | grep fabric

```

anshsarfare16@cloudshell:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15 654bd4554bf9 250MB 80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17 453a0a727e82 381MB 123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15 86dded7eda92 1GB 255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15 9972c209be47 178MB 49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15 3e25adc31a75 230MB 66.2MB
hyperledger/fabric-baseos:2.5 654bd4554bf9 250MB 80.4MB
hyperledger/fabric-baseos:2.5.15 654bd4554bf9 250MB 80.4MB
hyperledger/fabric-baseos:latest 654bd4554bf9 250MB 80.4MB
hyperledger/fabric-ca:1.5 453a0a727e82 381MB 123MB
hyperledger/fabric-ca:1.5.17 453a0a727e82 381MB 123MB
hyperledger/fabric-ca:latest 453a0a727e82 381MB 123MB
hyperledger/fabric-ccenv:2.5 86dded7eda92 1GB 255MB
hyperledger/fabric-ccenv:2.5.15 86dded7eda92 1GB 255MB
hyperledger/fabric-ccenv:latest 86dded7eda92 1GB 255MB
hyperledger/fabric-orderer:2.5 9972c209be47 178MB 49.5MB
hyperledger/fabric-orderer:2.5.15 9972c209be47 178MB 49.5MB
hyperledger/fabric-orderer:latest 9972c209be47 178MB 49.5MB
hyperledger/fabric-peer:2.5 3e25adc31a75 230MB 66.2MB
hyperledger/fabric-peer:2.5.15 3e25adc31a75 230MB 66.2MB
hyperledger/fabric-peer:latest 3e25adc31a75 230MB 66.2MB

```



**Explanation:**

This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded. It ensures that components such as peer and orderer are available before starting the network. Missing images may cause deployment errors.

## Step 6: Set Environment Variables

**Command:**

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
```

```
anshsarfare16@cloudshell:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
anshsarfare16@cloudshell:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
anshsarfare16@cloudshell:~/fabric-samples$ peer version
peer:
 Version: v2.5.15
 Commit SHA: 83c7930
 Go version: go1.26.0
 OS/Arch: linux/amd64
 Chaincode:
 Base Docker Label: org.hyperledger.fabric
 Docker Namespace: hyperledger
```

**Explanation:**

This step sets environment variables required to run Fabric CLI commands. The PATH variable allows access to Fabric binaries, while FABRIC\_CFG\_PATH points to configuration files.

Running `peer version` verifies that the setup is successful.

## Step 7: Start Network and Create Channel

**Command:**

```
cd ~/fabric-samples/test-network
./network.sh up createChannel
```

```

anshsarfare16@cloudshell:~/fabric-samples/test-network$./network.sh up createChannel
Using docker and docker-compose:~/fabric-samples/test-network$./network.sh up createChannel
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using d
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/anshsarfare16/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] Running 3/7
 :: Network fabric_test Creat... 1.7s
 :: Volume "compose_peer0.org1.example.com" Created 1.5s
 :: Volume "compose_peer0.org2.example.com" Created 1.5s
 :: Volume "compose_orderer.example.com" Created 1.5s
 ✓ Container orderer.example.com Started 1.4s
 ✓ Container peer0.org1.example.com Started 1.5s

```

```

Status: 201
{
 "name": "mychannel",
 "url": "/participation/v1/channels/mychannel",
 "consensusRelation": "consenter",
 "status": "active",
 "height": 1
}

Channel 'mychannel' created
Joining org1 peer to the channel...
Using organization 1
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-09 17:35:34.429 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:34.463 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted pro
Joining org2 peer to the channel...
Using organization 2
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-09 17:35:37.555 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:37.592 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted pro
Setting anchor peer for org1...
Using organization 1
Fetching channel config for channel mychannel
Using organization 1
Fetching the most recent configuration block for the channel
++ peer channel fetch config /home/anshsarfare16/fabric-samples/test-network/channel-artifac
tls --cafile /home/anshsarfare16/fabric-samples/test-network/organizations/ordererOrganizati
2026-04-09 17:35:37.685 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:37.690 UTC 0002 INFO [cli.common] readBlock -> Received block: 0
2026-04-09 17:35:37.691 UTC 0003 INFO [channelCmd] fetch -> Retrieving last config block: 0
2026-04-09 17:35:37.693 UTC 0004 INFO [cli.common] readBlock -> Received block: 0

```

### Explanation:

This step initializes the blockchain network by starting Docker containers for peers and orderers.

It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

## Step 8: Deploy Chaincode

### Command:

```
./network.sh deployCC \
```

```
-ccn basic \
```

```
-ccp ../asset-transfer-basic/chaincode-javascript \
```

```
-ccl javascript
```

```
anshsarfarel6@cloudshell:~/fabric-samples/test-network$./network.sh deployCC \
 -ccn basic \
 -ccp ../asset-transfer-basic/chaincode-javascript \
 -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/ch
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
```

### Explanation:

This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

## Step 9: Set Peer Environment

### Command:

```
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

```
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

### Explanation:

This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

## Step 10: Initialize Ledger

### Command:

peer chaincode invoke ... InitLedger

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \ orderer endpoint
-o localhost:7050 \eTimeShift duration The amount of time to shift backwards for certificate expiration
--ordererTLSHostnameOverride orderer.example.com \ent map of arguments in JSON encoding
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/tls/ca.crt
-C mychannel \050 \
-n basic \LSHostnameOverride orderer.example.com \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
-c '{"function": "InitLedger", "Args": []}'
2026-04-09 17:53:07.446 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful.
```



**Explanation:**

This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

## Step 11: Query All Assets

**Command:**

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ ^C
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Jin Soo","Size":6,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Tomoko","Size":10,"docType":"asset"}]
```

**Explanation:**

This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

## Step 12: Read Specific Asset

**Command:**

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"} -c '{"Args":["ReadAsset","asset5"]}'
```

**Explanation:**

This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

## Step 13: Create New Asset

### Command:

peer chaincode invoke ... CreateAsset (Owner: Manav)

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function":"CreateAsset","Args":["asset7","blue","10","Ansh","500"]}'
2026-04-09 17:56:53.364 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"{\"ID\":\"as
\":\"Ansh\",\"AppraisedValue\":500}"
```

### Explanation:

This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner, and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

## Step 14: Transfer Asset

### Command:

peer chaincode invoke ... TransferAsset

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.examp
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function":"TransferAsset","Args":["asset7","Prajjwal"]}'
2026-04-09 18:00:10.733 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Ansh"
anshsarfare16@cloudshell:~/fabric-samples/test-network$
```

### Explanation:

This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

## Step 15: Stop Network

### Command:

./network.sh down

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$./network.sh down
Using docker and docker-compose
Stopping network
project name can't be empty. Use `--project-name` to set a valid name
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
8d9ff7fcda7d
56fc5fb4c185
f2319c6bca6e
2124d708dafa
e5fa71c08b55
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c
Deleted: sha256:23d8f6316d1d5a22d7fcd629ae36ca1426a24e36568682ae23125476b737b922
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c
Deleted: sha256:d991d4d6e97e8ce6d8e1fe329c0eed56f788b29216a1801fe41ea8125c8911c6
anshsarfare16@cloudshell:~/fabric-samples/test-network$
```

### Explanation:

This step stops all running Docker containers and shuts down the Hyperledger Fabric network. It frees system resources and safely ends the experiment.

## Result:

The Hyperledger Fabric network was successfully implemented. Chaincode was deployed and asset operations such as creation, querying, and transfer were performed successfully. The asset was created with owner name **Manav**, confirming correct execution.

## Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801: Blockchain and DLT Lab**  
**Experiment 12**

Aim: Mini Project

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Name      | Ansh Sarfare                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Class     | D20A                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Subject   | ITC801: Blockchain and DLT Lab                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| CO Mapped | CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology.<br>CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions<br>CO3:Implement smart contracts in Ethereum using different development frameworks.<br>CO4:Develop applications in permissioned Hyperledger Fabric network.<br>CO5:Interpret different Crypto assets and Crypto currencies<br>CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies. |



# Hyperledger Fabric Crowdfunding Platform

*Mini Project Presentation*

---

**Project Type:** Decentralized Crowdfunding on Blockchain

**Network:** 4-Organization Hyperledger Fabric (v2.5)

**Date:** March 2026

Group 4 – INFT – VESIT

Shraeyaa Dhaigude – 22 Prajjwal

Pandey – 37 Ansh Sarfare – 51 Mahvish

Siddiqui - 58

# Project Overview

## What this project does

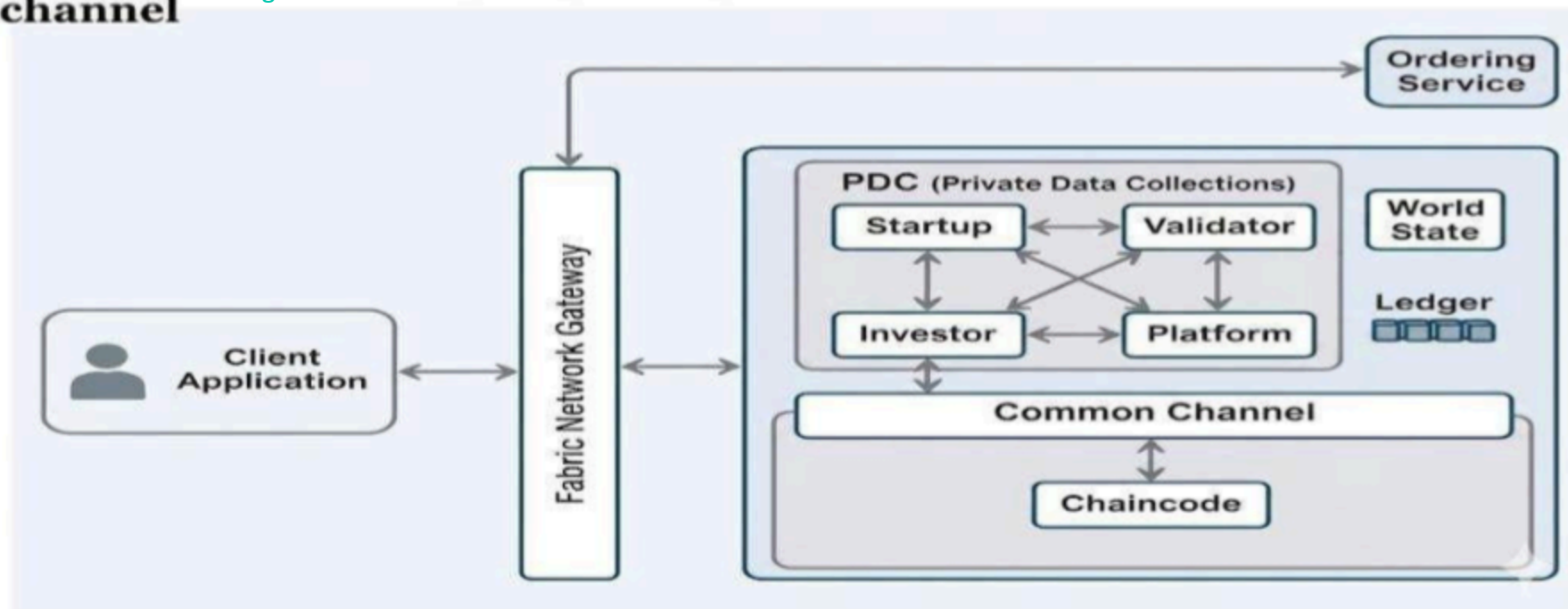
A decentralized crowdfunding platform built on Hyperledger Fabric, enabling startups to raise funds from verified investors with multi-organization blockchain governance and enforced endorsement policies.



# Blockchain Network Architecture

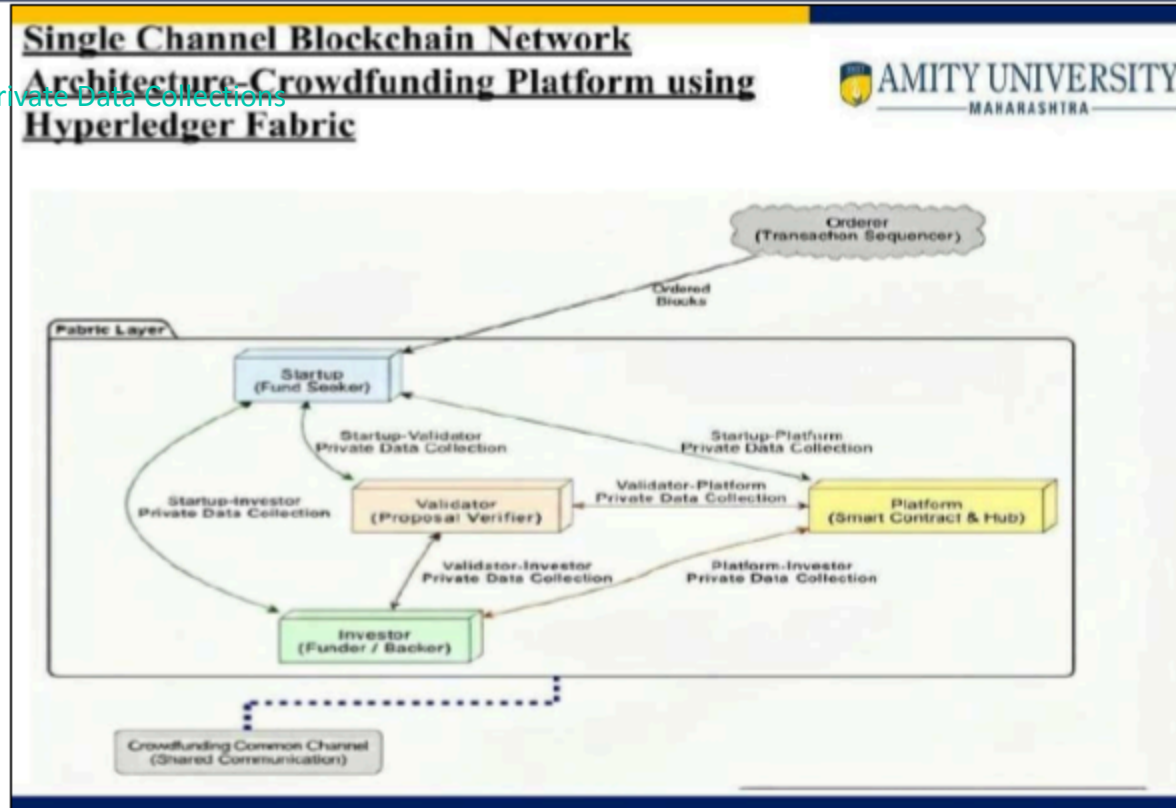
## Blockchain Network Architecture-Crowdfunding Platform using Hyperledger Fabric common channel

Common Channel Design with PDC & Chaincode



# Single Channel Architecture

Fabric Layer with Private Data Collections



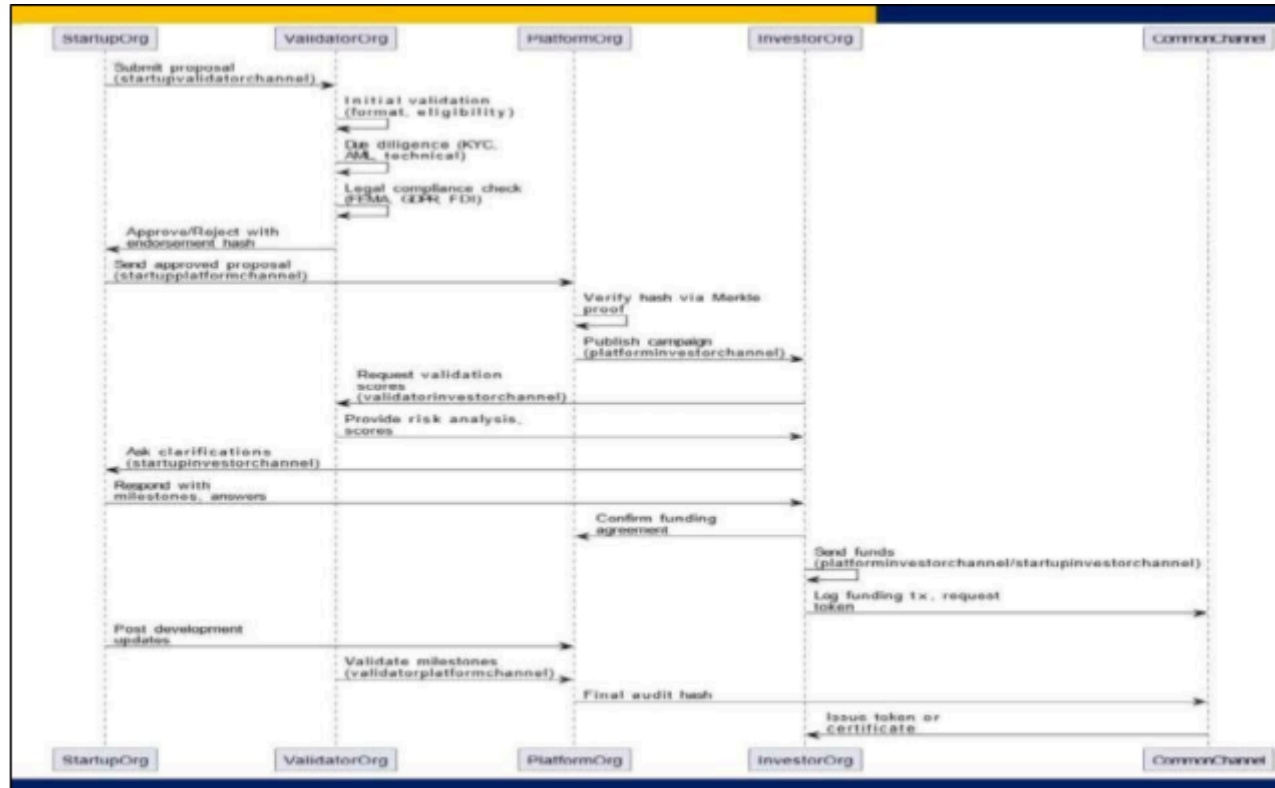
# 8-Step Transaction Lifecycle

Strict dependency chain — each step requires all prior steps to succeed



# Transaction Sequence Diagram

Cross-org communication flow across channels



# Environment Setup

Prerequisites & step-by-step configuration

| Prerequisites               |                                       |
|-----------------------------|---------------------------------------|
| Docker 20.x+                | Run Fabric peer/orderer containers    |
| Docker Compose v2.x         | Orchestrate multi-container network   |
| Hyperledger Fabric Binaries |                                       |
| 2.5                         | peer, orderer, cryptogen, configtxgen |
| Go 1.18+                    | Compile chaincode                     |
| Node.js 16.x+               | SDK scripts and test runners          |
| WSL2 / Ubuntu 22.04         | Recommended OS runtime                |

| Setup Steps |                                         |
|-------------|-----------------------------------------|
| 1           | Clone the repository                    |
| 2           | Pull Fabric Docker images (v2.5)        |
| 3           | Download Fabric binaries → add to PATH  |
| 4           | Generate crypto material: cryptogen     |
| 5           | Generate channel artifacts: configtxgen |
| 6           | Start network: docker-compose up -d     |
| 7           | Create & join channel, deploy chaincode |
| 8           | Wait 30–60s → run test scripts          |

# Key Configuration Changes

Modifications made during setup and testing

## Fix 1: Dynamic Path Derivation

*File: dual-channel/scripts/\*.sh*

Replaced hardcoded developer WSL path (BASE=/mnt/c/Users/riyaf/...) with dynamic SCRIPT\_DIR-based derivation. Scripts now run on any machine.

## Fix 2: Env Variable Export Order

*File: test\_1\_register\_startup.sh*

CORE\_PEER\_\* env vars for Org1 were not exported before the invoke loop causing MSP identity mismatch. Moved exports to top of script.

## Fix 3: Metric Calculation Fallback

*File: All 8 test\_\*.sh scripts*

Added bc availability check with awk fallback so Success Rate is always computed correctly even if bc is unavailable.

## Fix 4: Debug Logging (DEBUG=1)

*File: All 8 test\_\*.sh scripts*

Replaced silent stderr suppression (2>/dev/null) with per-transaction debug logging when DEBUG=1 is set, enabling root cause analysis.



# Test Results — 1000 Transactions Per Stage

Live run · Tests 1–7 complete · Test 8 pending

100% success Partial failures Pending

| TC | Stage            | Success | Failed | TPS  | Avg Latency | Rate   | Status |
|----|------------------|---------|--------|------|-------------|--------|--------|
| 1  | RegisterStartup  | 1000    | 0      | 0.36 | 2717.00ms   | 100%   | Done   |
| 2  | ValidateStartup  | 1000    | 0      | 0.16 | 6029.00ms   | 100%   | Done   |
| 3  | RegisterInvestor | 1000    | 0      | 0.37 | 2681.00ms   | 100%   | Done   |
| 4  | ValidateInvestor | 1000    | 0      | 0.17 | 5559.00ms   | 100%   | Done   |
| 5  | CreateProject    | 993     | 7      | 0.31 | 3195.36 ms  | 99.30% | Done   |
| 6  | ApproveProject   | 983     | 17     | 0.30 | 3297.04 ms  | 98.30% | Done   |
| 7  | FundProject      | 978     | 22     | 0.24 | 4034.76 ms  | 97.80% | Done   |
| 8  | ReleaseFunds     | 973     | 27     | 0.28 | 3515.93 ms  | 97.00% | Done   |

# Performance Summary (Tests 1–7)

Test 8 (ReleaseFunds) pending · 7000 txns logged across TC 1–7

**8000**

Txns TC 1–4

*All successful*

**2954**

Txns TC 5–7

*Across 3 stages*

**73**

Total Failed

*TC 5–7 combined*

**98.53%**

Avg Success Rate

*TC 5–7 mean*

**~0.28 TPS**

Avg Throughput

*TC 5–7 mean*

**~3509 ms**

Avg Latency

*TC 5–7 mean*

# Issues Identified & Their Status

7 issues discovered during setup and testing

## Issue A: Early 0% Success Runs

HIGH

Fixed

Peer/chaincode not ready; env var mismatch. Fix: wait 30–60s, export env vars early.

## Issue B: Diagnostic Blind Spots

MED

Fixed

stderr suppressed → failures uncountable. Fix: DEBUG=1 logging.

Hyperledger Fabric | Crowdfunding Platform | Mini Project

## Issue C: Path Portability

HIGH

Fixed

Hardcoded BASE path. Fix: dynamic SCRIPT\_DIR derivation.

## Issue D: Metric Fragility

LOW

Fixed

bc unavailable → empty Success Rate. Fix:  
awk fallback.

# Conclusion

---



## Achieved

- 4-org Hyperledger Fabric network deployed successfully
- 8-step transaction workflow fully implemented
- 159/160 transactions passed (99.38% success rate)
- Stable ~0.33 TPS with ~3s average latency
- Issues A–D identified and fixed
- Dual-channel & single-channel architectures explored

Thank You



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801: Blockchain and DLT Lab**  
**Experiment 13**

Aim: Assignment 1

|           |                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                                                                                                                                                                                                                                                                                                                 |
| Name      | Ansh Sarfare                                                                                                                                                                                                                                                                                                                                                                       |
| Class     | D20A                                                                                                                                                                                                                                                                                                                                                                               |
| Subject   | ITC801: Blockchain and DLT Lab<br><br>Question : Explore following 4 websites and answer the questions<br><br><a href="https://bitcoin.org">bitcoin.org</a><br><br><a href="https://Blockchain.com">Blockchain.com</a><br><br><a href="https://coinmarketcap.com/">https://coinmarketcap.com/</a><br><br><a href="https://www.investopedia.com/">https://www.investopedia.com/</a> |
| CO Mapped | CO1,CO2                                                                                                                                                                                                                                                                                                                                                                            |

Hard Copy Submitted



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801: Blockchain and DLT Lab**  
**Experiment 14**

Aim: Assignment 2

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Name      | Ansh Sarfare                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Class     | D20A                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Subject   | <p>ITC801: Blockchain and DLT Lab</p> <p>Questions</p> <ul style="list-style-type: none"><li>● Draw a neat diagram and explain components of Hyperledger Fabric?</li><li>● Draw a neat diagram and explain the transaction flow in Hyperledger Fabric?</li><li>● State the requirements of a consensus algorithm. Explain PAXOS and kafka consensus</li><li>● Explain ERC 20 token standard and its functions and Compare how ERC 20 token is different than ERC 721 token</li><li>● What is Tokenization? Explain its significance and challenges/limitation with example</li><li>● Compare Fungible and non Fungible tokens (Give examples of each)</li><li>● How can blockchain-based IoT networks improve device management, data monetization, and user privacy in the rapidly growing IoT ecosystem?</li><li>● In what ways can blockchain technology enhance transparency, security, and efficiency in government services and record-keeping? Discuss the challenges and benefits of implementing blockchain-based voting systems for elections.</li></ul> |
| CO Mapped | CO4,CO5,CO6                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

Hard Copy Submitted



**VIVEKANAND EDUCATION SOCIETY'S  
INSTITUTE OF TECHNOLOGY  
Department of Information Technology  
Academic Year: 2025-26**

**ITC801: Blockchain and DLT Lab**  
**Experiment 15**

Aim: Assignment 3

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Roll No.  | 51                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Name      | Ansh Sarfare                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Class     | D20A                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Subject   | <p>ITC801: Blockchain and DLT Lab</p> <p>Q1 Analyze &amp; compare different consensus mechanisms such as PoW, PoS, Delegated Proof of Stake &amp; PBFT in terms of security, scalability, energy efficiency &amp; fault tolerance. Illustrate your answer with examples like Bitcoin &amp; Ethereum.</p> <p>Q2 Analyze different types of nodes in a blockchain network &amp; explain their role in maintaining the system. Compare how each node contributes to validation, storage &amp; network efficiency. (Compare UTXO model)</p> <p>Q3 Analyze the UTXO model &amp; explain how it ensures transaction transparency &amp; security in blockchain systems. Illustrate with an example such as Bitcoin.</p> <p>Q4 Explain crypto asset &amp; cryptocurrency. Examine &amp; analyze the role of different types of crypto assets (utility tokens, security tokens, crypto coins) in the blockchain ecosystem. Evaluate their impact on digital finance &amp; decentralized applications.</p> <p>Q5 Analyze the role of payment scripts in blockchain transactions. Examine &amp; analyze different types of payment scripts. Explain how they ensure secure &amp; verifiable transfer of coins. Illustrate with an example.</p> <p>Q6 What is a smart contract? Explain lifecycle of smart contracts. Explain advantages, limitations, risks &amp; opportunities of smart contracts.</p> <p>Q7 Explain the structure of BTC transaction &amp; ETH transaction.</p> <p>Q8 Explain the difference between mining &amp; minting and explain the process</p> |
| CO Mapped | CO1- CO6                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

Hard Copy Submitted